PREMIUM HANDWRITTEN STUDY SERIES

KUBERNETES

Zero to Hero

Illustrated interview notebook
from first Pod to production architect

110 ORIGINAL QUESTIONS | 110 ANSWERED FOLLOW-UPS | 22 CHAPTERS | 12 HANDS-ON LABS

```
kubectl apply -f knowledge.yaml
desired-state → controllers → Pods
Explain • Sketch • Run • Verify
```

Java & Spring Boot focus | Training edition | October 2026

THE NOTEBOOK METHOD

Understand the flow. Own the failure.

Learn one question at a time with readable explanations and real command examples.

110
Exact original questions + simple answers

110
Follow-up questions actually answered

12
Cluster labs as separate Bash scripts

The five-part interview answer:

1. Tell the interviewer what it means.
2. Draw the real flow or ownership chain.
3. Show the YAML / kubectl action.
4. Explain what happens when something fails.
5. Answer the follow-up and name the limit of the guarantee.

What is verified?

Original question wording is retained. Lab scripts are checked using **bash -n**; the bundled Spring example is parsed as YAML and checked for the seven expected resources. Kubernetes clusters were **not** available here; no actual Pod, CNI, storage or rollout behavior was executed.

Safety first

Before the labs: inspect **kubectl config current-context**. Run against disposable clusters only. A namespace is not a guarantee against deleting resources in the wrong cluster. Do not use the teaching Secret values in production.

STUDY ROADMAP

22 chapters from zero to hero

01 Kubernetes foundations

02 Control plane and node architecture

03 Pods, containers and lifecycle

04 Deployments, ReplicaSets and rollouts

05 Services, DNS and traffic paths

06 Ingress, Gateway API and external routing

07 ConfigMaps, Secrets and runtime configuration

08 Startup, readiness, liveness and shutdown

09 Requests, limits, QoS and JVM sizing

10 Scheduling, affinity and topology

11 Volumes, claims and storage lifecycle

12 StatefulSets, DaemonSets, Jobs and CronJobs

13 Authentication, RBAC and workload identity

14 NetworkPolicy and layered traffic security

15 Autoscaling and downstream capacity

16 Observability and evidence-driven diagnosis

17 Helm, Kustomize, apply and GitOps

18 Spring Boot and Java workload design

19 Availability, disruption and rollout strategy

20 Cluster operations and recovery boundaries

21 Minikube, image loading and Windows/WSL access

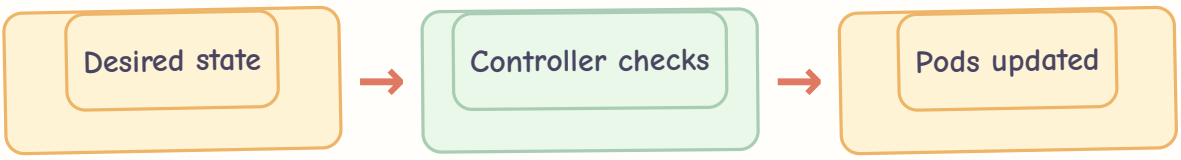
22 Senior interview and architecture scenarios

Practice chapters 01-08 first, 09-17 next, then 18-22 for Java / architect interviews. Labs start after the question bank.

CHAPTER 01 / 22

Kubernetes foundations

Keep desired state separate from application correctness.



THE QUESTIONS IN THIS CHAPTER

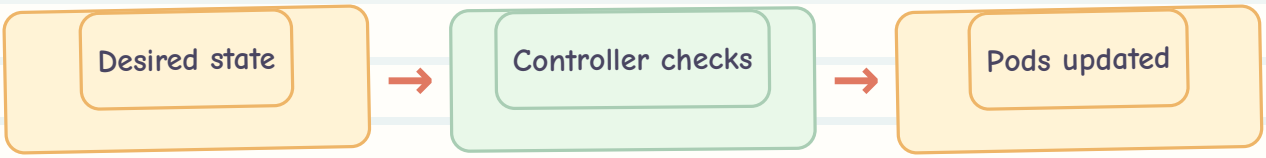
1. What is Kubernetes?
2. How is Kubernetes different from Docker?
3. What does declarative configuration mean?
4. What are labels, selectors and annotations?
5. What does a namespace isolate?

Q001. What is Kubernetes?

20-SECOND ANSWER • IN SIMPLE WORDS

Kubernetes keeps the requested number of containers running and helps them talk to each other.

Interview precision: Kubernetes is a platform for declaring and reconciling containerized workloads, including placement, service discovery and operational lifecycle.



YAML / COMMAND EXAMPLE

```
kubectl apply -f deployment.yaml
kubectl get deployments,pods,services -n interview
# The manifest declares desired state; controllers work toward it.
```

WHAT REALLY HAPPENS · A Deployment requesting three replicas causes controllers to maintain matching Pods. This is a continuing reconciliation process, not a one-time shell script.

INTERVIEWER FOLLOW-UP

Which responsibility belongs to the application rather than the orchestrator?

Answer: The app must still handle business rules, failed payments, safe retries and data correctness.

Watch out: Kubernetes does not automatically create correct retries, secure application code or database backups.

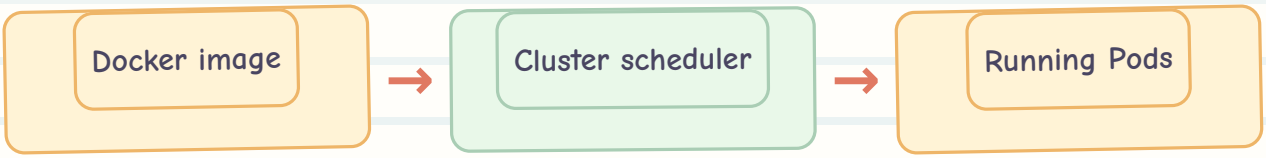
Original guide: page 6 · K01, K02, K47, K39 (source directory).

Q002. How is Kubernetes different from Docker?

20-SECOND ANSWER • IN SIMPLE WORDS

Docker builds and runs containers. Kubernetes manages those containers across a cluster.

Interview precision: Docker tools package and run containers; Kubernetes coordinates workloads across cluster resources through compatible runtimes.



YAML / COMMAND EXAMPLE

```
Source -> image build -> registry
Kubernetes manifest -> node runtime -> running Pod
# Docker-built images remain usable with a compatible runtime.
```

WHAT REALLY HAPPENS · Image format and node runtime integration are separate contracts. Kubernetes uses CRI-compatible runtime integration; it does not require Docker Engine on every node.

INTERVIEWER FOLLOW-UP

What changes when you move a single-host service into a cluster?

Answer: Add scheduling, Services, shared configuration, health checks, security, storage and safe rollout rules.

Watch out: A Docker Compose file is not automatically a Kubernetes deployment specification.

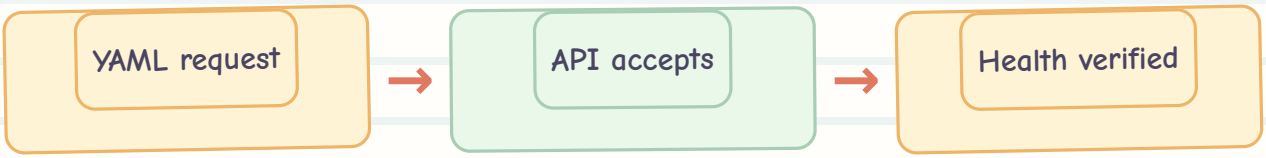
Original guide: page 7 · K01, K02, K47, K39 (source directory).

Q003. What does declarative configuration mean?

20-SECOND ANSWER • IN SIMPLE WORDS

Write what the cluster should look like; controllers keep trying to reach that state.

Interview precision: You describe the desired object state; the platform compares it with observed state and attempts reconciliation.



YAML / COMMAND EXAMPLE

```
spec:
  replicas: 3
# Deleting one owned Pod does not change this desired replica count.
```

WHAT REALLY HAPPENS · The controller can replace an owned Pod to restore the declared count. Changing the Deployment specification changes the goal. Actual state may temporarily differ during scheduling or failures.

INTERVIEWER FOLLOW-UP

How do you distinguish an accepted manifest from a healthy application?

Answer: A successful apply only proves the API accepted the object. Check Pod readiness and real requests too.

Watch out: A declaration can be impossible to satisfy because of capacity, policy or missing dependencies.

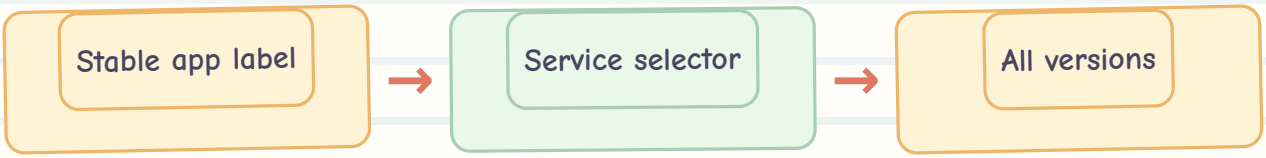
Original guide: page 7 · K01, K02, K47, K39 (source directory).

Q004. What are labels, selectors and annotations?

20-SECOND ANSWER • IN SIMPLE WORDS

Labels identify objects, selectors choose them, and annotations carry extra notes.

Interview precision: Labels identify and group objects, selectors match label sets, and annotations carry metadata not used as ordinary selector criteria.



YAML / COMMAND EXAMPLE

```
metadata:
  labels:
    app: orders
    tier: api
  annotations:
    example.com/source-commit: abc123
```

WHAT REALLY HAPPENS · A Service can select app=orders, while a deployment system records source identity in an annotation. Stable selector design matters because traffic and controller ownership depend on it.

INTERVIEWER FOLLOW-UP

Why should release-version labels be separate from a Service's stable selector?

Answer: Keep Service selectors stable so a new release label does not remove every Pod from service traffic.

Watch out: A label is not an authorization rule. Mislabeling can route traffic to an unintended workload.

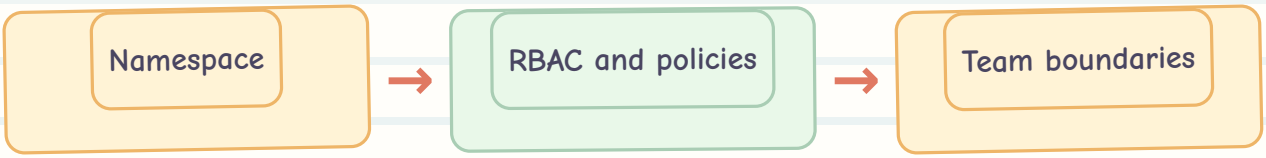
Original guide: page 8 · K01, K02, K47, K39 (source directory).

Q005. What does a namespace isolate?

20-SECOND ANSWER • IN SIMPLE WORDS

A namespace groups Kubernetes resources, but is not a complete security wall.

Interview precision: A namespace scopes many API objects and supports separate policy boundaries; it does not by itself isolate all networking, nodes or privileges.



YAML / COMMAND EXAMPLE

```
kubectl get pods -n orders
kubectl get namespaces
kubectl get nodes
# Nodes and PersistentVolumes are cluster-scoped.
```

WHAT REALLY HAPPENS · Two namespaces can each contain a Service named api. Namespaces help organize ownership, quotas and RoleBindings. NetworkPolicy and runtime controls add other boundaries.

INTERVIEWER FOLLOW-UP

What additional controls would you use for two independent teams?

Answer: Add RBAC, network rules, quotas, Pod Security policies and isolated credentials for each team.

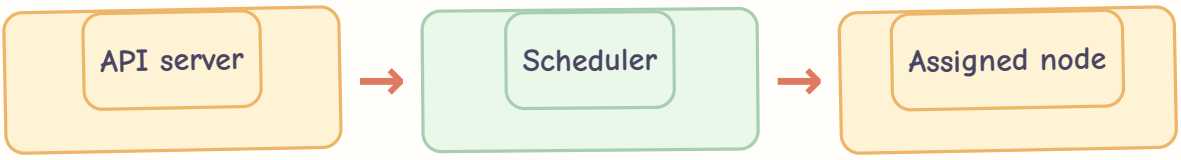
Watch out: Namespace separation alone is insufficient for hostile multi-tenancy. Cluster-scoped resources and administrator authority remain shared.

Original guide: page 8 · K01, K02, K47, K39 (source directory).

CHAPTER 02 / 22

Control plane and node architecture

Follow a request from kubectl to a running process.



THE QUESTIONS IN THIS CHAPTER

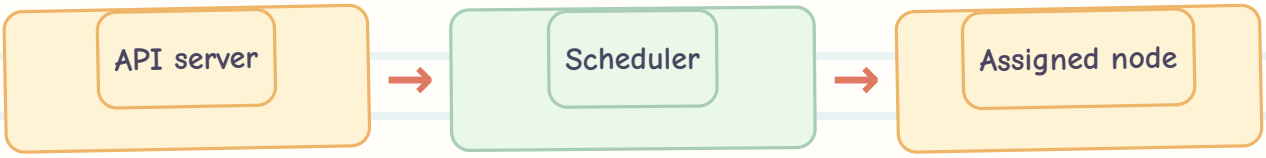
1. What are the main control-plane components?
2. What do kubelet, the runtime, CNI and service networking do?
3. Why is etcd important, and what needs backing up?
4. What happens if the API server is temporarily unavailable?
5. How does a Pod get scheduled and started?

Q006. What are the main control-plane components?

20-SECOND ANSWER • IN SIMPLE WORDS

The API server receives requests, etcd stores state, the scheduler picks nodes, and controllers repair differences.

Interview precision: The API server exposes cluster operations, etcd stores cluster state, the scheduler selects nodes, and controllers reconcile objects.



YAML / COMMAND EXAMPLE

```
kubectl -> API server -> stored desired state
Scheduler -> node assignment
Controller -> reconcile owned workloads
Kubelet -> execute assigned Pod through runtime integration
```

WHAT REALLY HAPPENS · These responsibilities are cooperative. A scheduler does not start the Java process, and etcd does not contain the application's database.

INTERVIEWER FOLLOW-UP

Which component would you investigate for a Pod with no node assignment?

Answer: Inspect the scheduler and Pod Events; verify node resources, selector rules, taints and PVC topology.

Watch out: Distribution-specific packaging can differ. Managed services hide some components but do not remove their architectural roles.

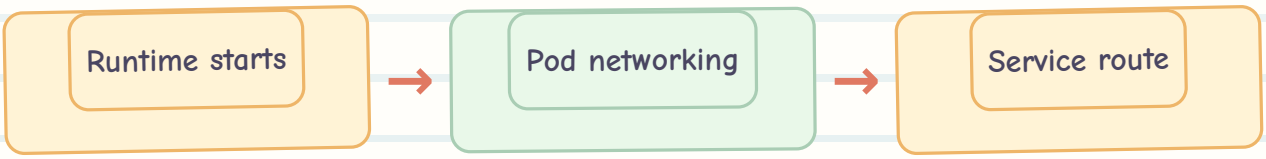
Original guide: page 9 · K01, K41, K51, K52 (source directory).

Q007. What do kubelet, the runtime, CNI and service networking do?

20-SECOND ANSWER • IN SIMPLE WORDS

Kubelet manages a node's Pods, the runtime starts containers, and networking connects them.

Interview precision: Kubelet manages node-side Pod execution, the runtime runs containers, CNI integration supplies Pod networking, and service networking implements Service reachability.



YAML / COMMAND EXAMPLE

```
kubectl get nodes -o wide
kubectl describe node NODE
kubectl get pods -n kube-system
# Service forwarding may use kube-proxy or a supported replacement.
```

WHAT REALLY HAPPENS · A running container with broken Pod networking is a different failure from a container creation error. Network implementations can use different mechanisms, including eBPF-based service handling.

INTERVIEWER FOLLOW-UP

How would you separate a runtime startup failure from a network failure?

Answer: Check waiting reason, events and container ID first; if running, inspect Pod IP, DNS, CNI and connectivity.

Watch out: Do not assume every cluster has an identical kube-proxy datapath or CNI policy capability.

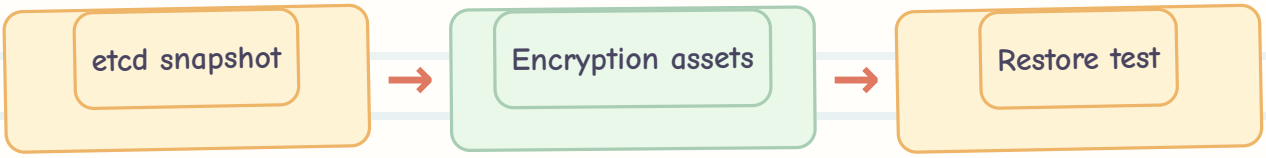
Original guide: page 10 · K01, K41, K51, K52 (source directory).

Q008. Why is etcd important, and what needs backing up?

20-SECOND ANSWER • IN SIMPLE WORDS

etcd stores the cluster's desired and observed API state, not your application's database.

Interview precision: etcd retains Kubernetes control-plane state; recovery requires a supported backup and restoration procedure appropriate to the cluster distribution.



YAML / COMMAND EXAMPLE

```
Cluster objects and control-plane state -> etcd recovery plan
Business database and volume data -> separate data recovery plan
# Test both recovery procedures independently.
```

WHAT REALLY HAPPENS · Restoring cluster objects is not the same as restoring orders, uploaded files or database transaction logs. A managed platform may own etcd operations, but the application team still owns its data guarantees.

INTERVIEWER FOLLOW-UP

Which secrets and encryption configuration must a control-plane restore account for?

Answer: Preserve etcd snapshots, encryption keys/configuration, certificate material and secrets-restoration procedures.

Watch out: A backup without a tested restore does not establish recovery time or consistency.

Original guide: page 10 · K01, K41, K51, K52 (source directory).

Q009. What happens if the API server is temporarily unavailable?

20-SECOND ANSWER • IN SIMPLE WORDS

Existing Pods might keep serving, but new deployments and many cluster repairs may stop.

Interview precision: Existing workloads can often continue running, but API-driven changes and reconciliation that require the unavailable control plane are impaired.



YAML / COMMAND EXAMPLE

```
Existing node process: may continue serving
New Deployment update: cannot rely on API acceptance
Failed-node replacement: may be delayed
```

WHAT REALLY HAPPENS · The impact depends on local node state, network implementation and the specific failure. Availability of existing traffic does not imply the cluster can scale, reschedule or accept deployments.

INTERVIEWER FOLLOW-UP

How would you monitor control-plane health separately from application availability?

Answer: Track API availability and request errors separately from real application success and user-facing latency.

Watch out: Do not promise every application keeps working; control-plane-dependent operations and credentials may have their own requirements.

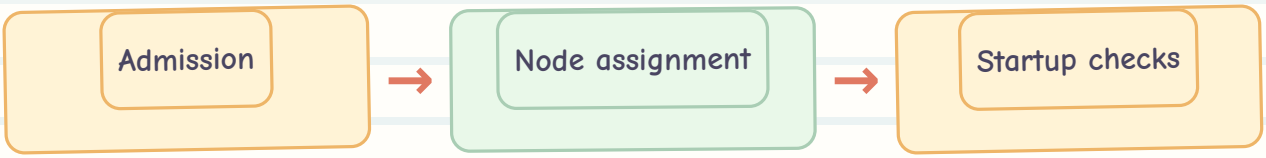
Original guide: page 11 · K01, K41, K51, K52 (source directory).

Q010. How does a Pod get scheduled and started?

20-SECOND ANSWER • IN SIMPLE WORDS

Admission checks a Pod, the scheduler selects a node, and kubelet starts its containers.

Interview precision: Admission accepts or rejects the object, scheduling assigns an eligible node, and the node prepares networking, storage and containers.



YAML / COMMAND EXAMPLE

```
kubectl get pod POD -n interview -o wide
kubectl describe pod POD -n interview
# Look at node assignment, conditions and Events.
```

WHAT REALLY HAPPENS · Pending with no assigned node points toward placement constraints. A node-assigned Pod can still wait for image pulls or volume attachment. Event evidence helps locate the failing step.

INTERVIEWER FOLLOW-UP

Why can a Pod be assigned to a node yet never become Ready?

Answer: Image pull errors, volume mounts, crashing processes or failing probes can block readiness after assignment.

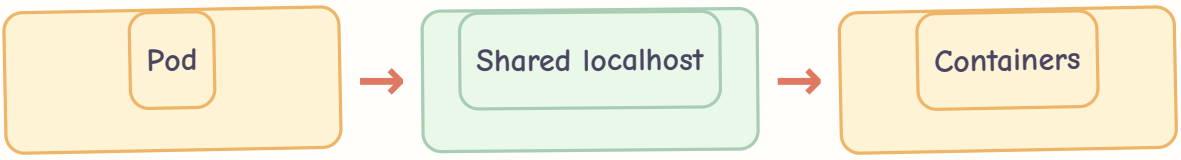
Watch out: Admission, scheduling and application readiness are distinct gates.

Original guide: page 11 · K01, K41, K51, K52 (source directory).

CHAPTER 03 / 22

Pods, containers and lifecycle

Understand Pod identity and shared local context.



THE QUESTIONS IN THIS CHAPTER

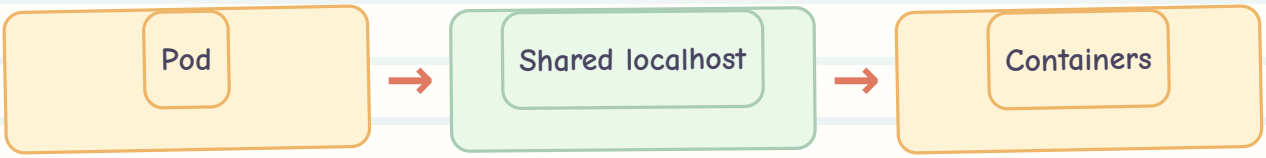
1. What is a Pod?
2. What is the difference between restarting a container and replacing a Pod?
3. What are init containers used for?
4. What is a sidecar, and how is it different from an ordinary init container?
5. What are Pod phases and CrashLoopBackOff?

Q011. What is a Pod?

20-SECOND ANSWER • IN SIMPLE WORDS

A Pod is the smallest scheduled unit: its containers share one network identity and selected volumes.

Interview precision: A Pod groups one or more containers scheduled together, sharing a network identity and declared volumes.



YAML / COMMAND EXAMPLE

```
spec:
  containers:
    - name: api
      image: TEAM/APP:TAG
    - name: helper
      image: TEAM/HELPER:TAG
```

WHAT REALLY HAPPENS · The two containers share the Pod network namespace and can reach each other through localhost. They do not automatically share every filesystem path; a declared volume can be mounted into both.

INTERVIEWER FOLLOW-UP

Why should an API and its separately scalable database usually be different workloads?

Answer: A database needs different scaling, upgrades, storage and recovery rules from an API, so separate them.

Watch out: A Pod should represent tightly coupled execution, not every service in the system.

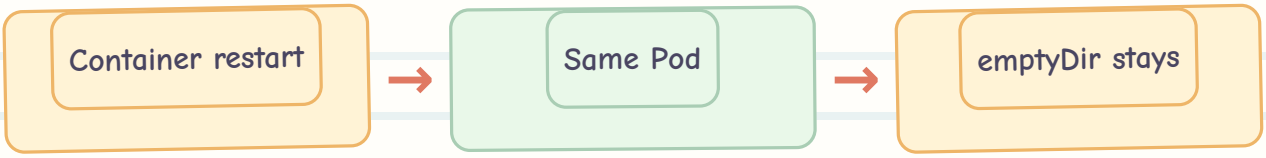
Original guide: page 12 · K02, K22, K34 (source directory).

Q012. What is the difference between restarting a container and replacing a Pod?

20-SECOND ANSWER • IN SIMPLE WORDS

Restarting a container keeps its Pod; replacing the Pod creates a new Pod identity.

Interview precision: A container restart stays within the same Pod identity; replacement creates a new Pod object, normally with a new UID and potentially a different IP or node.



YAML / COMMAND EXAMPLE

```
kubectl get pod POD -n interview -o wide
kubectl get pod POD -n interview -o jsonpath='{.metadata.uid}'
kubectl logs POD -n interview -c api --previous
```

WHAT REALLY HAPPENS · A kubelet restart can retain Pod-level context such as an emptyDir volume. A controller replacement does not preserve emptyDir data or promise the same network address. Use stable discovery and durable storage for those needs.

INTERVIEWER FOLLOW-UP

Which data survives a container crash but not a Pod replacement?

Answer: An emptyDir survives a container restart in the same Pod, but is lost when that Pod is replaced.

Watch out: A Pod's restart count is not a count of every historical replacement.

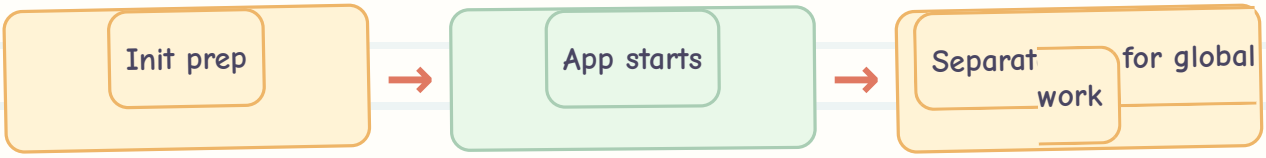
Original guide: page 13 · K02, K22, K34 (source directory).

Q013. What are init containers used for?

20-SECOND ANSWER • IN SIMPLE WORDS

Init containers prepare something before the main application begins.

Interview precision: Ordinary init containers complete sequentially before application containers start, allowing preparation with a separate runtime contract.



YAML / COMMAND EXAMPLE

```
initContainers:
  - name: prepare
    image: busybox:1.37
    command: ["sh", "-c", "echo prepared > /work/state"]
    volumeMounts:
      - {name: work, mountPath: /work}
```

WHAT REALLY HAPPENS · The app can read a prepared file if it mounts the same declared volume. Init work should have bounded retries and observable failure. A cluster-wide migration needs stronger ownership than one init container per replica.

INTERVIEWER FOLLOW-UP

When would a separate Job be better than an init container?

Answer: Use a Job for one shared migration or task that must finish once per planned workflow, not once per replica.

Watch out: This is a fragment requiring a work volume. A forever-waiting init step can leave every replica unavailable.

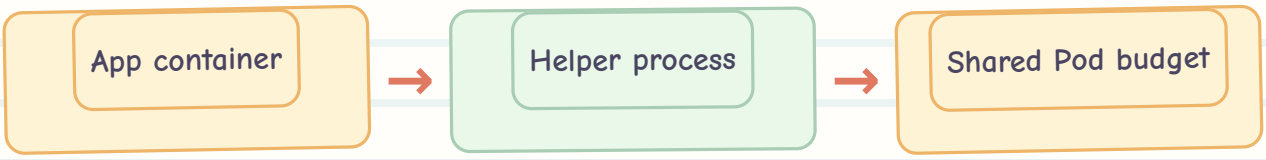
Original guide: page 13 · K02, K22, K34 (source directory).

Q014. What is a sidecar, and how is it different from an ordinary init container?

20-SECOND ANSWER • IN SIMPLE WORDS

A sidecar is a helper that runs alongside the main application.

Interview precision: A sidecar supports the app while it runs; Kubernetes also supports restartable init-container sidecars under the relevant feature/version contract.



YAML / COMMAND EXAMPLE

```
# Native sidecar sketch:
initContainers:
  - name: support
    image: TEAM/SUPPORT:TAG
    restartPolicy: Always
```

WHAT REALLY HAPPENS · The support process can remain active while application containers execute. Legacy sidecar patterns also use regular app containers. Lifecycle and ordering behavior differ, so select a supported pattern deliberately.

INTERVIEWER FOLLOW-UP

How would a helper affect readiness, resource totals and shutdown order?

Answer: Count its memory/CPU, decide whether failure affects readiness, and test process termination order.

Watch out: Do not put restartPolicy: Always on an ordinary init container without understanding native-sidecar support. Job completion behavior also matters.

Original guide: page 14 · K02, K22, K34 (source directory).

Q015. What are Pod phases and CrashLoopBackOff?

20-SECOND ANSWER • IN SIMPLE WORDS

Pod phases are broad states. CrashLoopBackOff means a container keeps failing and restarting.

Interview precision: Phases summarize lifecycle broadly; CrashLoopBackOff is a container waiting reason for repeated restarts, not a Pod phase.



YAML / COMMAND EXAMPLE

```
kubectl get pod POD -n interview -o jsonpath='{.status.phase}'
kubectl describe pod POD -n interview
kubectl logs POD -n interview -c api --previous
```

WHAT REALLY HAPPENS · A Pod can be in Running phase while one container repeatedly crashes or the Pod is not Ready. Inspect per-container state, exit cause, restart count and previous logs.

INTERVIEWER FOLLOW-UP

What evidence distinguishes an application exception from a failed liveness probe?

Answer: Read prior logs and exit reasons; a probe failure appears in events, while an exception appears in app logs.

Watch out: The STATUS column from kubectl is not always the literal phase field.

Original guide: page 14 · K02, K22, K34 (source directory).

CHAPTER 04 / 22

Deployments, ReplicaSets and rollouts

Understand controller ownership and safe releases.



THE QUESTIONS IN THIS CHAPTER

1. How do Deployment, ReplicaSet and Pod relate?
2. What triggers a Deployment rollout?
3. How do maxSurge and maxUnavailable work?
4. Does Kubernetes automatically roll back a failed Deployment?
5. Why are Deployment selectors important?

Q016. How do Deployment, ReplicaSet and Pod relate?

20-SECOND ANSWER • IN SIMPLE WORDS

Deployment manages ReplicaSets; each ReplicaSet creates and replaces matching Pods.

Interview precision: A Deployment manages ReplicaSets for updates; a ReplicaSet maintains matching Pods; Pods run the containers.



YAML / COMMAND EXAMPLE

```
kubectl get deploy,rs,pods -n interview
kubectl rollout status deployment/orders -n interview
kubectl rollout history deployment/orders -n interview
```

WHAT REALLY HAPPENS · A template update creates or reuses an appropriate ReplicaSet and moves replica ownership through a rollout. You normally manage the Deployment rather than hand-editing its ReplicaSets.

INTERVIEWER FOLLOW-UP

Which object should you edit to change the image for every future replica?

Answer: Edit the Deployment's Pod template, not a running Pod or generated ReplicaSet.

Watch out: Direct Pod edits can be replaced by the controller and may not change the desired template.

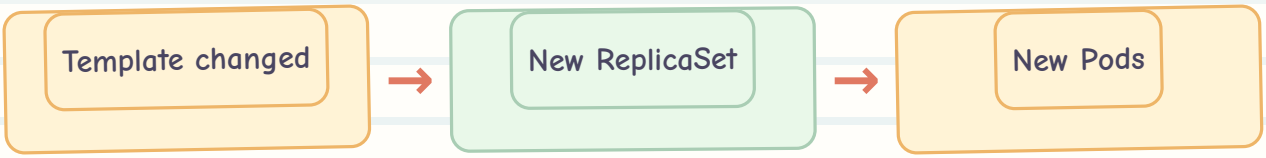
Original guide: page 15 · K03 (source directory).

Q017. What triggers a Deployment rollout?

20-SECOND ANSWER • IN SIMPLE WORDS

Changing the Pod template starts a rollout; changing replicas only scales it.

Interview precision: A change to the Pod template triggers a new rollout; changing the desired replica count alone does not create a new template revision.



YAML / COMMAND EXAMPLE

```
kubectl set image deployment/orders \
  api=TEAM/APP:NEW_TAG -n interview
kubectl scale deployment/orders --replicas=4 -n interview
```

WHAT REALLY HAPPENS · The image change alters the template. Scaling changes how many replicas are requested. Updating an external ConfigMap may leave the template unchanged unless your delivery process deliberately changes a checksum or restarts it.

INTERVIEWER FOLLOW-UP

How do you roll out an environment-variable configuration update?

Answer: Use a controlled restart or a template checksum/version change for environment values that Pods read at startup.

Watch out: Reapplying identical YAML does not force every Pod to restart.

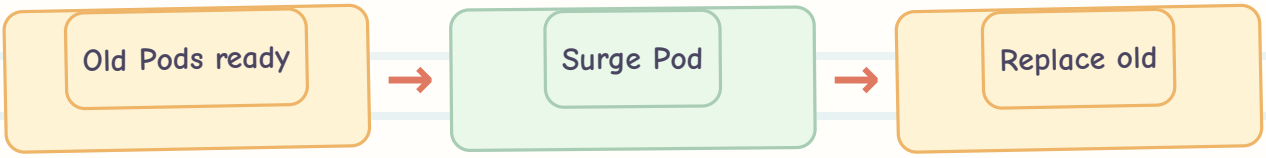
Original guide: page 16 · K03 (source directory).

Q018. How do maxSurge and maxUnavailable work?

20-SECOND ANSWER • IN SIMPLE WORDS

maxSurge allows extra Pods; maxUnavailable sets how many may be unavailable during an update.

Interview precision: They bound extra replicas and unavailable replicas during a rolling update, with percentage rounding rules where percentages are used.



YAML / COMMAND EXAMPLE

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

WHAT REALLY HAPPENS · For a three-replica workload, this policy can temporarily need a fourth replica while retaining available capacity. Readiness and sufficient node resources determine whether the rollout can progress.

INTERVIEWER FOLLOW-UP

What happens if there is no room for the surge Pod?

Answer: The rollout waits for spare capacity, unless you safely adjust surge/unavailability or add capacity.

Watch out: The policy is not an end-to-end zero-error guarantee. Terminating Pods can consume resources beyond a simple replica calculation.

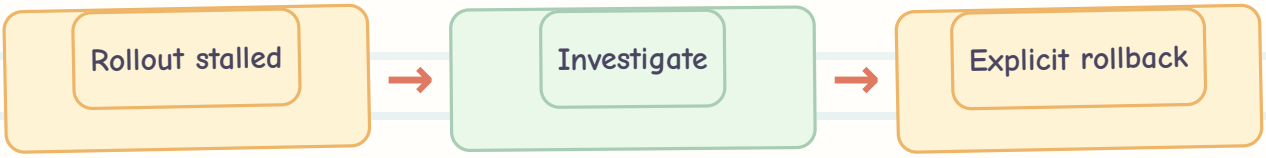
Original guide: page 16 · K03 (source directory).

Q019. Does Kubernetes automatically roll back a failed Deployment?

20-SECOND ANSWER • IN SIMPLE WORDS

Kubernetes marks a stalled rollout; it does not automatically undo every failed release.

Interview precision: A Deployment reports stalled progress under its configured conditions; an automatic rollback requires an external delivery policy or operator action.



YAML / COMMAND EXAMPLE

```
kubectl rollout status deployment/orders -n interview --timeout=120s
kubectl describe deployment orders -n interview
# After reviewing compatibility and intent:
kubectl rollout undo deployment/orders -n interview
```

WHAT REALLY HAPPENS · A progress deadline helps surface a stalled rollout. undo restores a retained template revision; it does not restore database state or all external configuration. Delivery systems can enforce additional rollback rules.

INTERVIEWER FOLLOW-UP

What must be checked before rolling application code back after a migration?

Answer: Check database backward compatibility, the migration's rollback path and whether old code understands new rows.

Watch out: A timeout from your kubectl client is not itself a completed server-side rollback.

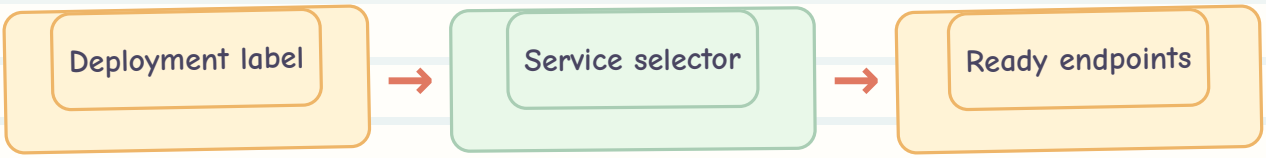
Original guide: page 17 · K03 (source directory).

Q020. Why are Deployment selectors important?

20-SECOND ANSWER • IN SIMPLE WORDS

Selectors tell a Deployment which Pod template it owns and must match the template labels.

Interview precision: The selector defines the Pod set a controller owns and must align with the Pod template labels; apps/v1 Deployment selectors are immutable after creation.



YAML / COMMAND EXAMPLE

```
selector:
  matchLabels:
    app: orders
template:
  metadata:
    labels:
      app: orders
```

WHAT REALLY HAPPENS · Matching labels establish ownership. A stable app selector avoids making release version changes into ownership changes. A changed immutable selector needs a deliberate replacement or new Deployment plan.

INTERVIEWER FOLLOW-UP

How can a correct Deployment still receive no traffic from its Service?

Answer: Service labels may not match, Pods may be unready, or targetPort may not reach the listener.

Watch out: Avoid overlapping ownership selectors between independent controllers. Service selectors have a separate traffic role.

Original guide: page 17 · K03 (source directory).

CHAPTER 05 / 22

Services, DNS and traffic paths

Always trace the caller port to the Java listener.



THE QUESTIONS IN THIS CHAPTER

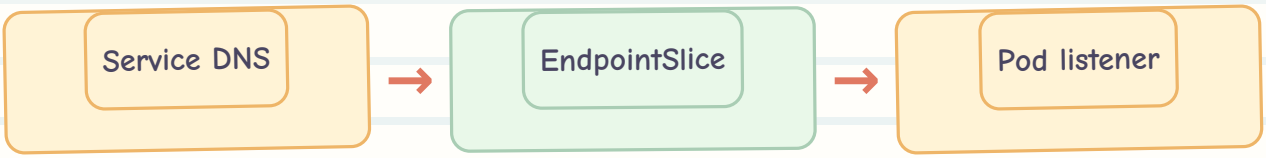
1. What does a Kubernetes Service provide?
2. How do port, targetPort, containerPort and nodePort differ?
3. How do ClusterIP, NodePort, LoadBalancer and headless Services differ?
4. How does Kubernetes Service DNS work?
5. What are EndpointSlices useful for in troubleshooting?

Q021. What does a Kubernetes Service provide?

20-SECOND ANSWER • IN SIMPLE WORDS

A Service gives clients a stable name and port while its matching Pods change.

Interview precision: A Service defines a stable access abstraction for a backend set; its type and networking implementation determine how clients reach that set.



YAML / COMMAND EXAMPLE

```
apiVersion: v1
kind: Service
metadata: {name: orders}
spec:
  selector: {app: orders}
  ports:
    - {port: 80, targetPort: 8080}
```

WHAT REALLY HAPPENS · Clients use orders:80 while selected backend Pods listen on 8080. Pod replacement does not require clients to memorize the new Pod IP.

INTERVIEWER FOLLOW-UP

What would you inspect if the Service exists but connections fail?

Answer: Check selectors, Ready Pods, EndpointSlices, targetPort, listener address, DNS and network rules.

Watch out: A Service does not start an application or make a wrong listener port valid.

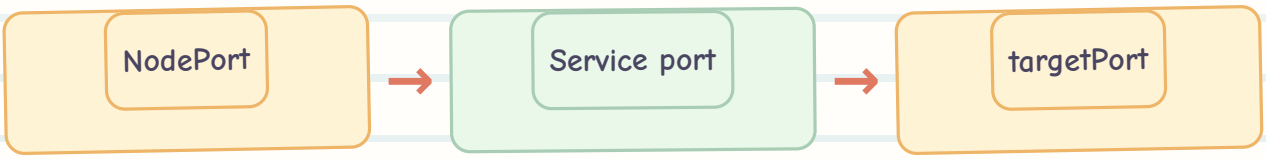
Original guide: page 18 · K04, K20, K21, K35 (source directory).

Q022. How do port, targetPort, containerPort and nodePort differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Service port is the caller's port; targetPort is the Pod's listener; nodePort exposes a node port.

Interview precision: port is the Service endpoint, targetPort is the backend listener target, containerPort is declarative container metadata, and nodePort is an external node-level port when allocated.



YAML / COMMAND EXAMPLE

```
Client -> Service port 80 -> Pod targetPort 8080
NodePort client -> reachable node IP:NODE_PORT -> Service backend
# containerPort: 8080 does not make Java listen on 8080.
```

WHAT REALLY HAPPENS · The application must bind the correct address and port. A named targetPort can refer to a named container port. Host access and cluster-internal access use different endpoints.

INTERVIEWER FOLLOW-UP

Why could port-forward work while a NodePort URL fails on a laptop?

Answer: Port-forward uses an API tunnel. NodePort needs a reachable node address and permitted network path.

Watch out: Do not confuse a Service port with the node-level port or a port-forward's local port.

Original guide: page 19 · K04, K20, K21, K35 (source directory).

Q023. How do ClusterIP, NodePort, LoadBalancer and headless Services differ?

20-SECOND ANSWER • IN SIMPLE WORDS

ClusterIP is internal, NodePort opens a node port, LoadBalancer needs provider support, and headless gives backend DNS.

Interview precision: They provide cluster virtual access, node-level exposure, supported load-balancer integration and DNS-based backend discovery respectively.



YAML / COMMAND EXAMPLE

```
ClusterIP:    cluster-facing virtual endpoint
NodePort:     reachable node address plus allocated port
LoadBalancer: integration with a supporting provider/controller
Headless:     clusterIP: None; backend discovery without a VIP
```

WHAT REALLY HAPPENS · A headless Service is useful for discovering individual StatefulSet members. LoadBalancer provisioning depends on the environment; a local cluster may need tunnel or another integration.

INTERVIEWER FOLLOW-UP

Which type would you use for a private database and why?

Answer: Use ClusterIP for an internal database entry point; use headless DNS when individual stable members are required.

Watch out: A pending external address does not necessarily mean the Pods are broken. Provider policy and internal/external settings matter.

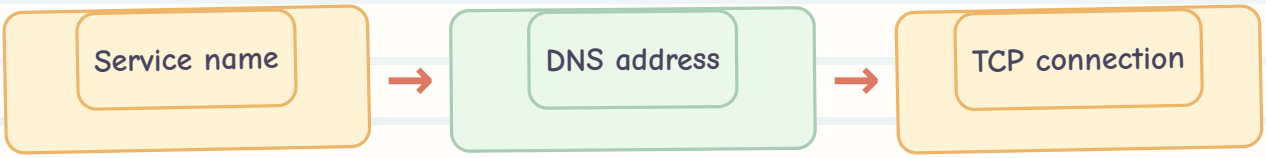
Original guide: page 19 · K04, K20, K21, K35 (source directory).

Q024. How does Kubernetes Service DNS work?

20-SECOND ANSWER • IN SIMPLE WORDS

Cluster DNS translates Service names into network addresses, usually with namespace-based search rules.

Interview precision: Cluster DNS supplies Service names, with short names commonly resolving within the caller's namespace and qualified names identifying other namespaces.



YAML / COMMAND EXAMPLE

```
# Same namespace:
jdbc:postgresql://db:5432/orders
# Another namespace:
jdbc:postgresql://db.data.svc.cluster.local:5432/orders
# cluster.local is a common default, not a universal cluster domain.
```

WHAT REALLY HAPPENS · The Pod's DNS configuration and search domains influence lookup. Resolve the name, then test the required protocol. A successful lookup is not evidence of database readiness or authorization.

INTERVIEWER FOLLOW-UP

How would you distinguish DNS failure from a refused TCP connection?

Answer: Test name resolution first, then TCP connection separately; a refused connection means DNS likely worked.

Watch out: localhost normally refers to the calling Pod, including its sidecars, not a different database Pod.

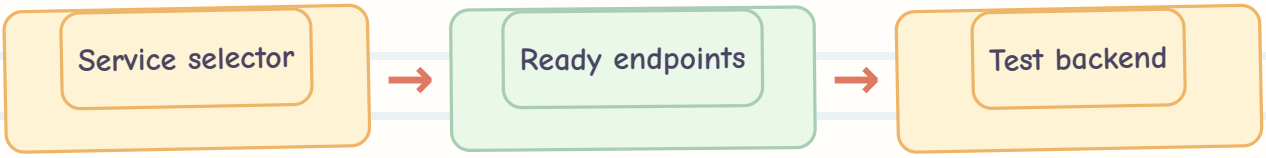
Original guide: page 20 · K04, K20, K21, K35 (source directory).

Q025. What are EndpointSlices useful for in troubleshooting?

20-SECOND ANSWER • IN SIMPLE WORDS

EndpointSlices list backend Pod addresses and their readiness conditions for a Service.

Interview precision: EndpointSlices show Service backend addresses and conditions, helping verify selection and readiness before investigating the network datapath.



YAML / COMMAND EXAMPLE

```
kubectl get svc orders -n interview -o yaml
kubectl get pods -n interview -l app=orders --show-labels
kubectl get endpointslices -n interview \
  -l kubernetes.io/service-name=orders -o yaml
```

WHAT REALLY HAPPENS · An empty backend set can mean the selector matches nothing. Not-ready endpoints point toward Pod readiness. Correct ready endpoints shift attention toward the listener, policy or traffic implementation.

INTERVIEWER FOLLOW-UP

What check comes after a ready endpoint with the correct target port?

Answer: Test the actual listener and network policy from a client Pod, then inspect packet routing if needed.

Watch out: Special settings such as publishNotReadyAddresses change assumptions. Prefer EndpointSlices over relying on legacy Endpoints output for modern diagnosis.

Original guide: page 20 · K04, K20, K21, K35 (source directory).

CHAPTER 06 / 22

Ingress, Gateway API and external routing

An Ingress rule needs a controller and a matching path.



THE QUESTIONS IN THIS CHAPTER

1. What is the difference between a Service and an Ingress?
2. How do Exact and Prefix paths work, and who rewrites paths?
3. Why does a host-based route fail when called by raw IP?
4. What is Gateway API, and how is it organized?
5. How do you troubleshoot an external HTTP 503?

Q026. What is the difference between a Service and an Ingress?

20-SECOND ANSWER • IN SIMPLE WORDS

A Service names backends; an Ingress declares HTTP routes implemented by an installed controller.

Interview precision: A Service exposes a backend set; Ingress expresses HTTP(S) routing rules implemented by an ingress controller.



YAML / COMMAND EXAMPLE

```
External request -> ingress controller -> Service -> ready Pod
# Rules can select a backend by host and path.
kubectl get ingress -n interview
kubectl get ingressclass
```

WHAT REALLY HAPPENS · An ingress object without a supporting controller does not create a working proxy. Its backend references a Service port, which then maps to the Pod listener.

INTERVIEWER FOLLOW-UP

Which controller and class own a specific routing object?

Answer: Inspect ingressClassName/IngressClass and the controller's status, events and logs.

Watch out: Ingress is not a general protocol gateway for every TCP/UDP workload. Its API is frozen; Gateway API supports newer routing models.

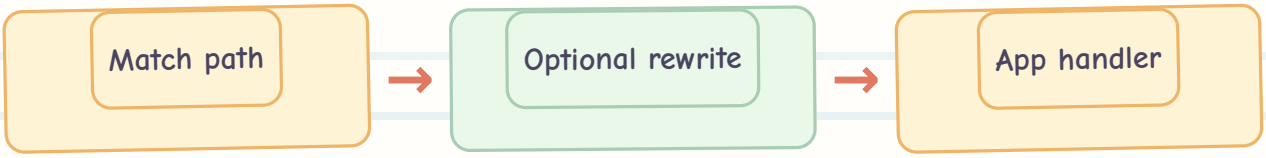
Original guide: page 21 · K05, G01, K35 (source directory).

Q027. How do Exact and Prefix paths work, and who rewrites paths?

20-SECOND ANSWER • IN SIMPLE WORDS

Exact matches a whole path; Prefix matches a path prefix. Rewriting is a separate controller feature.

Interview precision: Path types define matching semantics; URI rewriting is a separate controller or routing-feature decision.



YAML / COMMAND EXAMPLE

```
paths:
  - path: /reviews/v1
    pathType: Prefix
    backend:
      service:
        name: reviews-v1
        port: {number: 80}
```

WHAT REALLY HAPPENS · This routes matching requests but does not universally remove /reviews/v1 before forwarding. The backend must support that path or the chosen controller must apply a supported rewrite policy.

INTERVIEWER FOLLOW-UP

Why could routing reach the right Pod but return an application 404?

Answer: The upstream may receive /reviews/v1 while the app only serves /; verify forwarded path and rewrite rules.

Watch out: Controller-specific annotations are not portable Kubernetes semantics. Avoid assuming all controllers interpret one annotation identically.

Original guide: page 22 · K05, G01, K35 (source directory).

Q028. Why does a host-based route fail when called by raw IP?

20-SECOND ANSWER • IN SIMPLE WORDS

A host rule needs the correct HTTP Host header; requesting by IP may select another route.

Interview precision: The request's host identity may not match the configured rule even if the controller address is reachable.



YAML / COMMAND EXAMPLE

```
curl -H 'Host: reviews.local' http://CONTROLLER_IP/reviews/v1
# Or configure local DNS/hosts so reviews.local reaches the controller.
# HTTPS additionally requires correct SNI and certificate identity.
```

WHAT REALLY HAPPENS · HTTP host matching selects the rule. A raw-IP request can hit a default backend instead. For HTTPS, use a client setup that preserves the intended hostname for TLS validation and routing.

INTERVIEWER FOLLOW-UP

How would curl --resolve help test HTTPS before updating DNS?

Answer: Use curl --resolve to connect to a test IP while preserving the hostname for HTTPS and SNI.

Watch out: Disabling TLS verification is not a proper production fix for a hostname mismatch.

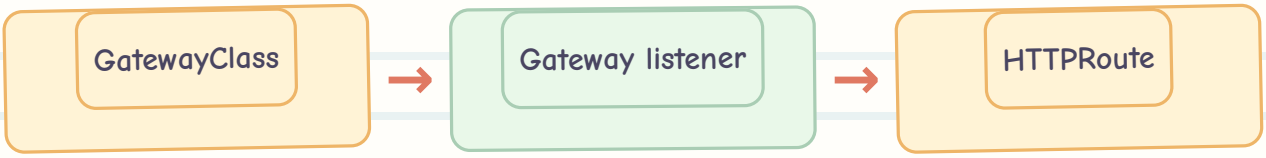
Original guide: page 22 · K05, G01, K35 (source directory).

Q029. What is Gateway API, and how is it organized?

20-SECOND ANSWER • IN SIMPLE WORDS

Gateway API separates traffic infrastructure from application-owned routes.

Interview precision: Gateway API separates infrastructure ownership from route ownership through resources such as GatewayClass, Gateway and HTTPRoute.



YAML / COMMAND EXAMPLE

```
Platform team: GatewayClass and Gateway policy
App team:     HTTPRoute attached to an allowed Gateway
# Requires installed Gateway API CRDs and a supporting controller.
```

WHAT REALLY HAPPENS · The separation helps teams delegate routes while retaining listener and attachment policy. Read resource status to confirm acceptance and backend resolution. Controller support and feature conformance still matter.

INTERVIEWER FOLLOW-UP

What must be allowed before a route can attach across namespaces?

Answer: Check listener allowedRoutes plus cross-namespace ReferenceGrant where required, then inspect route status.

Watch out: These are not automatically installed on every Kubernetes cluster. API presence alone does not prove traffic implementation.

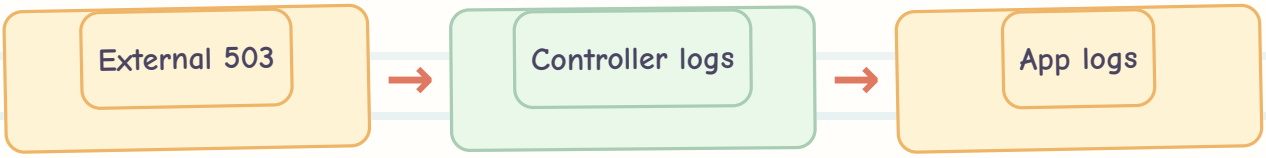
Original guide: page 23 · K05, G01, K35 (source directory).

Q030. How do you troubleshoot an external HTTP 503?

20-SECOND ANSWER • IN SIMPLE WORDS

Trace the 503 from ingress controller through Service and endpoints to the application.

Interview precision: Trace the external controller, route acceptance, Service selection, ready backend set and actual application response.



YAML / COMMAND EXAMPLE

```
kubectl describe ingress ROUTE -n interview
kubectl get svc,pods -n interview
kubectl get endpointslices -n interview
# Inspect controller logs and test backend reachability separately.
```

WHAT REALLY HAPPENS · 503 can arise because no healthy upstream is available, but controller-specific behavior differs. A backend request returning 503 itself is a different cause from a proxy generating it. Use response and log evidence.

INTERVIEWER FOLLOW-UP

How would you compare proxy access logs with application request logs?

Answer: Match request IDs and timestamps; see whether the proxy generated 503 or forwarded an app response.

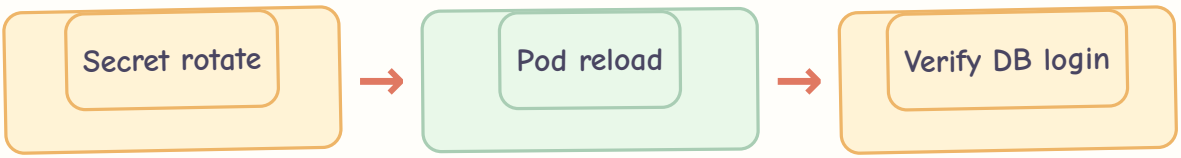
Watch out: Recreating every Pod can erase evidence and worsen an outage. First identify the layer generating the response.

Original guide: page 23 · K05, G01, K35 (source directory).

CHAPTER 07 / 22

ConfigMaps, Secrets and runtime configuration

Deliver configuration without exposing credentials.



THE QUESTIONS IN THIS CHAPTER

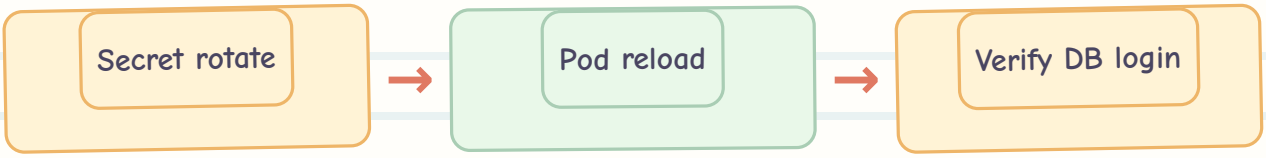
1. When should you use a ConfigMap versus a Secret?
2. Do ConfigMap and Secret updates immediately update application behavior?
3. Why is base64 not encryption, and how should Secrets be protected?
4. What is the Downward API used for?
5. How can Spring Boot read mounted secret files?

Q031. When should you use a ConfigMap versus a Secret?

20-SECOND ANSWER • IN SIMPLE WORDS

ConfigMaps hold regular settings; Secrets hold sensitive values with restricted access.

Interview precision: Use ConfigMaps for ordinary configuration and Secrets for sensitive values, with explicit access and storage protections.



YAML / COMMAND EXAMPLE

```
env:
  - name: SPRING_PROFILES_ACTIVE
    valueFrom:
      configMapKeyRef: {name: app-config, key: profile}
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef: {name: db-auth, key: password}
```

WHAT REALLY HAPPENS · The values enter the container at startup. The Secret object type supports dedicated handling but does not automatically make every consumer or operator safe.

INTERVIEWER FOLLOW-UP

How would you rotate a credential without rebuilding the image?

Answer: Update the Secret through approved delivery, allow credential overlap, then restart/reload Pods safely.

Watch out: Secret data encoded as base64 is not encrypted by that encoding. Restrict API access, logs and backup exposure.

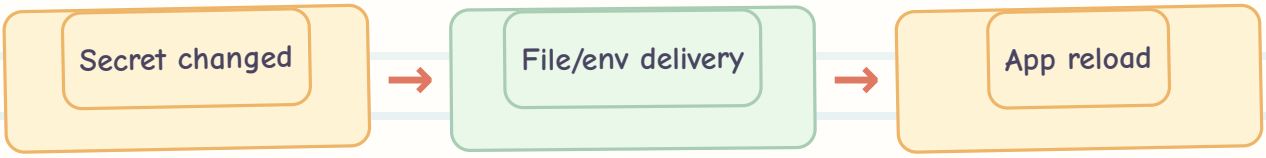
Original guide: page 24 · K10, K11, K42, K48, S03 (source directory).

Q032. Do ConfigMap and Secret updates immediately update application behavior?

20-SECOND ANSWER • IN SIMPLE WORDS

Environment variables are read at startup; mounted files may update later, but apps must reload them.

Interview precision: It depends on injection mode and application reload: environment values are startup snapshots, while projected files can update eventually with important exceptions.



YAML / COMMAND EXAMPLE

```
Environment injection -> recreate/restart workload for new values
Projected volume -> file update can propagate eventually
Application binding -> may still need explicit reload/restart
# subPath mounts do not receive the ordinary projected update behavior.
```

WHAT REALLY HAPPENS · A refreshed file is not proof that a Spring bean re-bound its configuration. A controlled rollout can be simpler than a hot-reload design when changes are infrequent.

INTERVIEWER FOLLOW-UP

What happens if a database revokes the old password before every replica reconnects?

Answer: Replicas still using the old credential may lose connections. Rotate with overlap and confirm all replicas first.

Watch out: Immutable objects and subPath use change the update plan. Coordinate credential overlap with live connection pools.

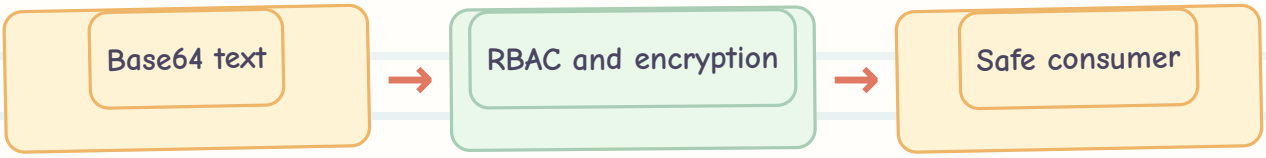
Original guide: page 25 · K10, K11, K42, K48, S03 (source directory).

Q033. Why is base64 not encryption, and how should Secrets be protected?

20-SECOND ANSWER • IN SIMPLE WORDS

Base64 only changes the representation. Secret protection needs real access rules and encryption.

Interview precision: Base64 is reversible representation; protection requires configured at-rest safeguards, secure transport, limited authorization and safe consumption.



YAML / COMMAND EXAMPLE

```
printf 'ZGVtbw==' | base64 -d
# Produces demo with no decryption key.
kubectl auth can-i get secrets -n interview
```

WHAT REALLY HAPPENS · Knowing how values are represented prevents a false security assumption. Verify the cluster's encryption policy and who can read Secrets or create Pods that consume them.

INTERVIEWER FOLLOW-UP

How would you review service-account and developer access separately?

Answer: Review human identities and service-account RBAC independently, plus who can create Pods reading Secrets.

Watch out: Pod-creation authority can expose namespace secrets indirectly. A deny on get secrets is not the entire threat model.

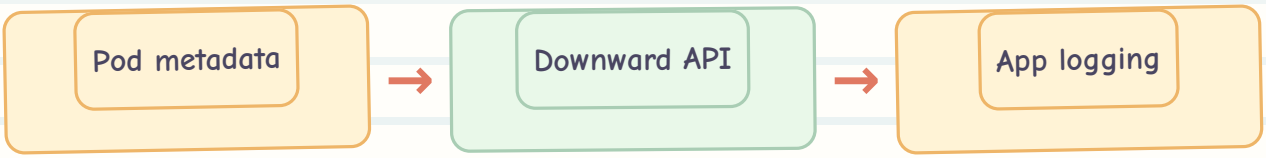
Original guide: page 25 · K10, K11, K42, K48, S03 (source directory).

Q034. What is the Downward API used for?

20-SECOND ANSWER • IN SIMPLE WORDS

Downward API gives a container selected Pod details without broad API permissions.

Interview precision: It supplies supported Pod metadata and resource information to containers without requiring general Kubernetes API access.



YAML / COMMAND EXAMPLE

```
env:
  - name: POD_NAME
    valueFrom:
      fieldRef: {fieldPath: metadata.name}
  - name: POD_NAMESPACE
    valueFrom:
      fieldRef: {fieldPath: metadata.namespace}
```

WHAT REALLY HAPPENS · The app can add instance identity to logs. This avoids giving the process broad API credentials just to learn its own name. Supported field choices and update behavior depend on the injection form.

INTERVIEWER FOLLOW-UP

Which identity should remain stable across Pod replacement?

Answer: Business IDs such as order ID, tenant ID or customer ID must not depend on replaceable Pod names.

Watch out: Pod names are not stable business identifiers. Avoid using them as durable order or customer keys.

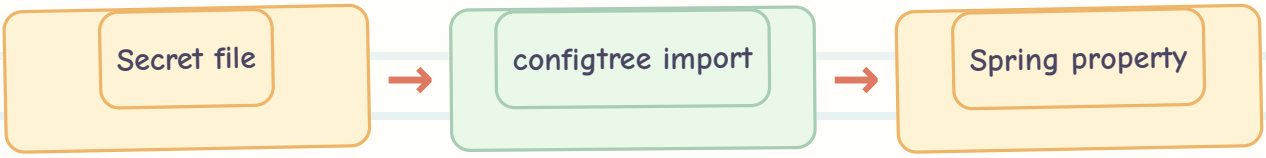
Original guide: page 26 · K10, K11, K42, K48, S03 (source directory).

Q035. How can Spring Boot read mounted secret files?

20-SECOND ANSWER • IN SIMPLE WORDS

Spring can read a directory of mounted secret files as configuration properties.

Interview precision: A supported configuration-tree import maps file names to properties, with required versus optional startup semantics chosen deliberately.



YAML / COMMAND EXAMPLE

```
SPRING_CONFIG_IMPORT=configtree:/run/secrets/
# Example file: /run/secrets/spring.datasource.password
# Use a projected Secret volume at that directory.
```

WHAT REALLY HAPPENS · The app reads the property at startup under the chosen Spring configuration contract. A required import helps detect missing deployment inputs instead of silently starting with incomplete configuration.

INTERVIEWER FOLLOW-UP

When would optional:configtree be appropriate, and when would it hide an error?

Answer: Use optional only for truly optional config; required passwords must fail startup if missing.

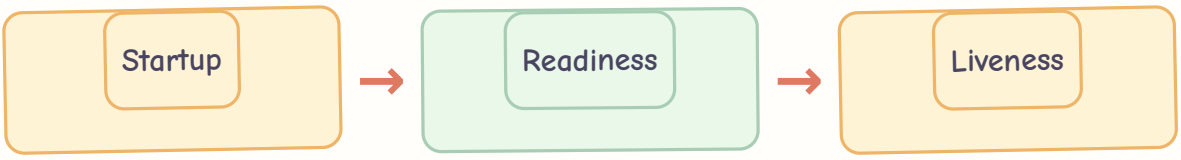
Watch out: File replacement does not automatically refresh all bound values. Secret names, mount permissions and app UID must agree.

Original guide: page 26 · K10, K11, K42, K48, S03 (source directory).

CHAPTER 08 / 22

Startup, readiness, liveness and shutdown

Choose probes by the action Kubernetes should take.



THE QUESTIONS IN THIS CHAPTER

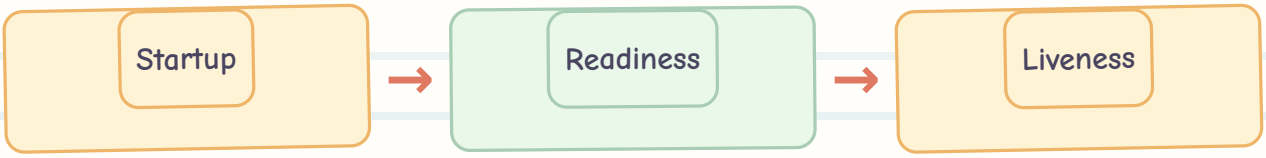
1. How do startup, readiness and liveness probes differ?
2. Why should liveness usually avoid checking a shared database?
3. Does a successful readiness probe guarantee no requests fail?
4. How does terminationGracePeriodSeconds interact with preStop and SIGTERM?
5. How should Spring Boot health endpoints be exposed for probes?

Q036. How do startup, readiness and liveness probes differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Startup gives an app time to boot; readiness controls traffic; liveness decides whether to restart.

Interview precision: Startup gates initial initialization, readiness controls normal traffic admission, and liveness can trigger container restart after repeated failure.



YAML / COMMAND EXAMPLE

```
startupProbe:
  httpGet: {path: /livez, port: 8080}
  periodSeconds: 5
  failureThreshold: 24
readinessProbe:
  httpGet: {path: /readyz, port: 8080}
livenessProbe:
  httpGet: {path: /livez, port: 8080}
```

WHAT REALLY HAPPENS · With a startup probe configured, normal readiness and liveness probing wait for startup success. The example gives about two minutes of failure tolerance at the specified interval; actual timing includes execution details.

INTERVIEWER FOLLOW-UP

Which probe should fail while a service deliberately drains traffic?

Answer: Readiness should fail during drain, while liveness may remain healthy until the process stops.

Watch out: Paths must exist and have a correct access policy. Probe configuration does not create endpoints.

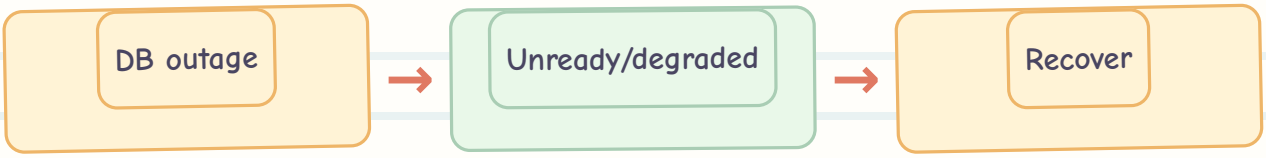
Original guide: page 28 · K06, S01, S02 (source directory).

Q037. Why should liveness usually avoid checking a shared database?

20-SECOND ANSWER • IN SIMPLE WORDS

If the database is down, restarting every healthy JVM usually makes the outage worse.

Interview precision: A shared dependency failure often cannot be repaired by restarting every application process, and mass restarts can worsen the incident.



YAML / COMMAND EXAMPLE

```
Database unavailable -> possibly unready or degraded API
JVM deadlocked -> liveness failure may justify restart
# Decide from the recovery action, not merely the dependency list.
```

WHAT REALLY HAPPENS · Readiness may include dependencies needed for the intended traffic, but that choice must consider whether removing every instance is useful. Liveness should target failures where restarting this instance helps.

INTERVIEWER FOLLOW-UP

How would your API behave during a short database failover?

Answer: Fail or degrade database-dependent requests, retry carefully, and recover connections without a restart storm.

Watch out: A trivial always-200 liveness endpoint may miss a stuck request subsystem. Choose a meaningful but stable contract.

Original guide: page 28 · K06, S01, S02 (source directory).

Q038. Does a successful readiness probe guarantee no requests fail?

20-SECOND ANSWER • IN SIMPLE WORDS

Readiness is a useful sample, not a guarantee that every request succeeds.

Interview precision: No. It is a sampled admission signal, and traffic propagation, application races and downstream failures can occur between samples.



YAML / COMMAND EXAMPLE

```
Probe interval -> observed readiness
Endpoint/controller update -> traffic propagation
Request handling -> current application and dependency state
```

WHAT REALLY HAPPENS · Readiness helps reduce avoidable routing to unsuitable instances. It does not prove all routes, payloads or dependencies work at every instant. Monitor request outcomes and test transitions.

INTERVIEWER FOLLOW-UP

What would you test during rollout beyond waiting for Ready Pods?

Answer: Test HTTP behavior, error rate, latency, graceful drain, database access and new/old version compatibility.

Watch out: Existing connections and external load balancers can have independent drain behavior.

Original guide: page 29 · K06, S01, S02 (source directory).

Q039. How does terminationGracePeriodSeconds interact with preStop and SIGTERM?

20-SECOND ANSWER • IN SIMPLE WORDS

preStop and Spring shutdown share one termination grace period before forced kill.

Interview precision: The grace budget includes termination work; a preStop hook runs before the normal stop signal, and final forced termination follows budget expiry.



YAML / COMMAND EXAMPLE

```
spec:
  terminationGracePeriodSeconds: 45
  containers:
    - name: api
      lifecycle:
        preStop:
          exec: {command: ["sh", "-c", "sleep 5"]}
```

WHAT REALLY HAPPENS · A brief delay can help with traffic propagation only when the image includes sh and the measured deployment needs it. The hook consumes the budget, leaving less time for application drain.

INTERVIEWER FOLLOW-UP

How would you budget a 20-second Spring drain plus propagation time?

Answer: Budget measurable propagation delay, at least 20s drain time, plus margin; e.g. 45s after testing.

Watch out: Do not add a shell-based hook to a shell-free image. A sleep is not proof that traffic has stopped, and exact termination behavior has version details.

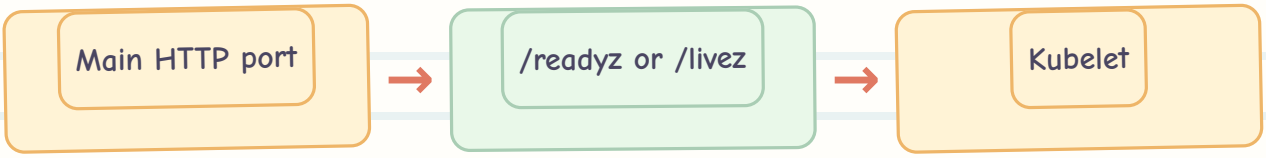
Original guide: page 29 · K06, S01, S02 (source directory).

Q040. How should Spring Boot health endpoints be exposed for probes?

20-SECOND ANSWER • IN SIMPLE WORDS

Expose a real application-port probe that reflects the running Spring HTTP server.

Interview precision: Configure supported probe endpoints on the application path and make their access and listener contract explicit.



YAML / COMMAND EXAMPLE

```
management.endpoint.health.probes.enabled=true
management.endpoint.health.probes.add-additional-paths=true
server.shutdown=graceful
# Supported Boot baseline supplies /livez and /readyz on the main port.
```

WHAT REALLY HAPPENS · Probing the main server can catch failures hidden by a separate healthy management listener. Verify endpoint availability with the actual Actuator dependency, Spring version and Security configuration.

INTERVIEWER FOLLOW-UP

What would a separate management port fail to detect?

Answer: A separate healthy management listener can miss a stuck or unbound main application listener.

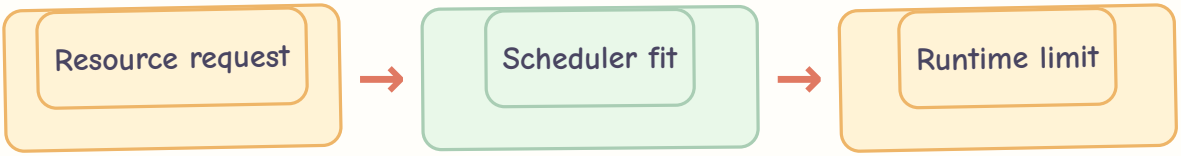
Watch out: Avoid exposing sensitive diagnostic endpoints publicly while enabling probe access. A passing health group is not a full business transaction test.

Original guide: page 30 · K06, S01, S02 (source directory).

CHAPTER 09 / 22

Requests, limits, QoS and JVM sizing

Budget the entire JVM, not only its Java heap.



THE QUESTIONS IN THIS CHAPTER

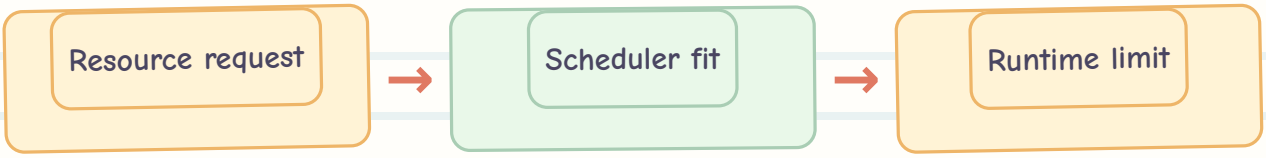
1. What is the difference between resource requests and limits?
2. How do CPU throttling and memory OOM termination differ?
3. What are Guaranteed, Burstable and BestEffort QoS classes?
4. How should Java heap relate to a container memory limit?
5. How do ResourceQuota and LimitRange differ?

Q041. What is the difference between resource requests and limits?

20-SECOND ANSWER • IN SIMPLE WORDS

Requests reserve scheduling capacity; limits cap CPU and memory behavior at runtime.

Interview precision: Requests guide scheduling and resource allocation; limits constrain supported runtime consumption through mechanisms appropriate to each resource.



YAML / COMMAND EXAMPLE

```
resources:
  requests: {cpu: "250m", memory: "256Mi"}
  limits: {cpu: "1", memory: "512Mi"}
```

WHAT REALLY HAPPENS · The scheduler accounts for the requested budget. CPU use can be throttled by a limit, while exceeding memory constraints can cause termination. 250m means a quarter CPU unit; Mi is binary memory sizing.

INTERVIEWER FOLLOW-UP

Why can a Pod remain Pending even while a node looks mostly idle?

Answer: Scheduling uses requested CPU/memory and placement rules, not current dashboard idle percentages alone.

Watch out: Requests are not a promise of unlimited throughput or independent physical cores. Defaults can be injected by namespace policy.

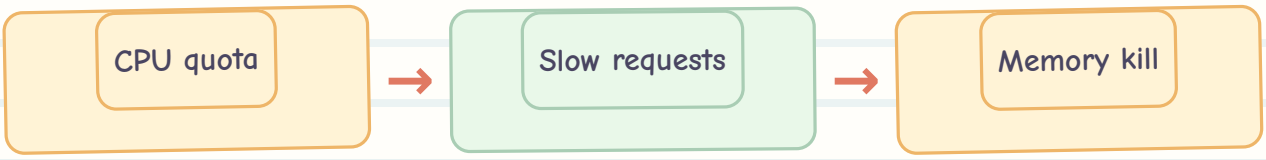
Original guide: page 31 · K07, K29, K30, K31, J01 (source directory).

Q042. How do CPU throttling and memory OOM termination differ?

20-SECOND ANSWER • IN SIMPLE WORDS

CPU throttling slows a process; memory exhaustion can kill it.

Interview precision: CPU limits can delay execution through throttling; memory enforcement can kill a process when the relevant budget is exhausted.



YAML / COMMAND EXAMPLE

```
kubectl describe pod POD -n interview
kubectl top pod POD -n interview
kubectl logs POD -n interview --previous
# Combine with cgroup and JVM time-series telemetry.
```

WHAT REALLY HAPPENS · High tail latency can reflect throttling even without an application exception. A kernel memory kill may prevent Java from emitting a heap dump. Use termination reason and resource history to separate them.

INTERVIEWER FOLLOW-UP

What evidence would distinguish GC pressure from CPU quota pressure?

Answer: Compare CPU throttling time, heap/GC logs, CPU quota and termination state together.

Watch out: Exit 137 is evidence to investigate, not a universal proof of memory exhaustion. Metrics Server snapshots can miss spikes.

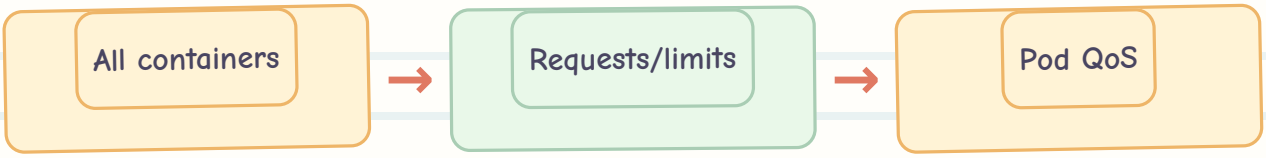
Original guide: page 32 · K07, K29, K30, K31, J01 (source directory).

Q043. What are Guaranteed, Burstable and BestEffort QoS classes?

20-SECOND ANSWER • IN SIMPLE WORDS

QoS depends on how all containers' CPU and memory requests and limits are declared.

Interview precision: They classify Pod resource configuration and influence behavior under resource pressure; they are not application availability guarantees.



YAML / COMMAND EXAMPLE

```
# Traditional per-container Guaranteed pattern:
requests: {cpu: "500m", memory: "512Mi"}
limits:   {cpu: "500m", memory: "512Mi"}
# Every relevant container must meet the class requirements.
```

WHAT REALLY HAPPENS · Without relevant requests or limits, a traditional Pod may be BestEffort. Partial or unequal configuration generally leads to Burstable. Inspect the actual qosClass instead of inferring from one container.

INTERVIEWER FOLLOW-UP

How can a sidecar change the Pod's class and total budget?

Answer: A helper container adds resources and can change the Pod's QoS classification.

Watch out: Newer resource features can affect details. Priority and consumption also matter in eviction decisions; Guaranteed does not mean never killed.

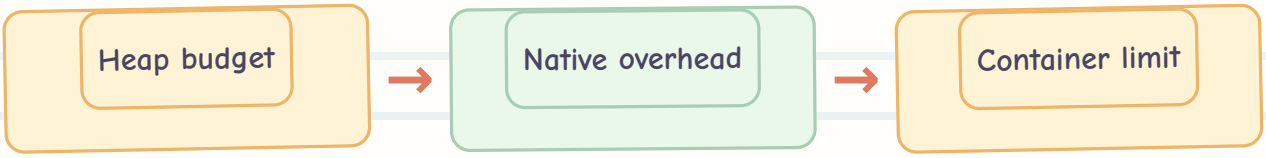
Original guide: page 32 · K07, K29, K30, K31, J01 (source directory).

Q044. How should Java heap relate to a container memory limit?

20-SECOND ANSWER • IN SIMPLE WORDS

Leave memory for JVM heap plus threads, metaspace, direct buffers and native code.

Interview precision: Heap must leave measured room for native allocations, metaspace, direct buffers, thread stacks and other charged memory.



YAML / COMMAND EXAMPLE

```
# Illustrative 512 MiB limit, not a universal sizing rule:
env:
  - {name: JAVA_TOOL_OPTIONS, value: "-Xms128m -Xmx320m"}
# Measure peak total use with realistic traffic and thread counts.
```

WHAT REALLY HAPPENS · The remaining space is a starting hypothesis, not established safety. Modern supported JDKs on Linux detect container limits, but the exact update and cgroup setup must be verified.

INTERVIEWER FOLLOW-UP

When would you choose explicit Xmx rather than a percentage?

Answer: Use explicit Xmx when workloads need a tested fixed heap budget; leave safety room under memory limit.

Watch out: MaxRAMPercentage sizes heap ergonomically; it does not cap total process memory. Probe subprocesses and agents can consume additional memory.

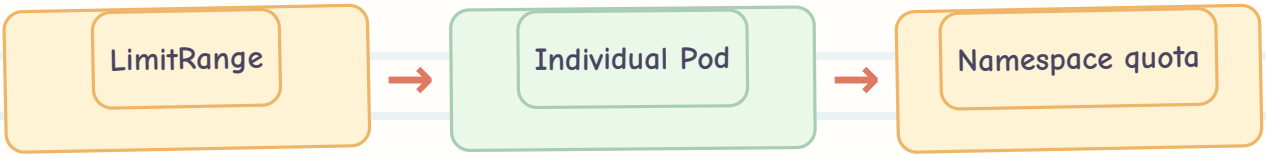
Original guide: page 33 · K07, K29, K30, K31, J01 (source directory).

Q045. How do ResourceQuota and LimitRange differ?

20-SECOND ANSWER • IN SIMPLE WORDS

ResourceQuota limits a namespace total; LimitRange sets defaults and bounds for individual resources.

Interview precision: ResourceQuota restricts aggregate namespace usage or object counts; LimitRange sets supported per-object defaults and bounds during admission.



YAML / COMMAND EXAMPLE

```
kubectl get resourcequota,limitrange -n interview
kubectl describe resourcequota -n interview
# Inspect effective admitted Pod requests and limits.
```

WHAT REALLY HAPPENS · A quota can block a new replica even when nodes have free capacity. A LimitRange can supply missing defaults that change scheduling or runtime behavior. Compare the rendered input with the admitted object.

INTERVIEWER FOLLOW-UP

Why might the same manifest be accepted in one namespace and rejected in another?

Answer: Namespaces can have different quota, security or admission settings, so the same YAML may be rejected.

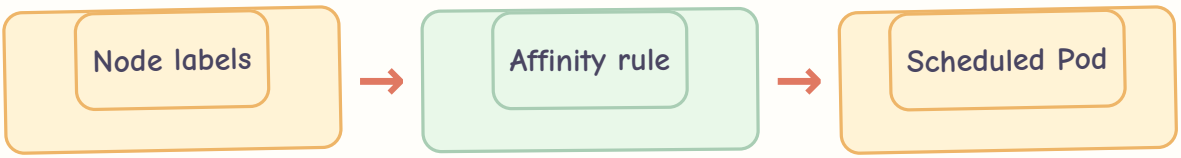
Watch out: These policies do not provision capacity or automatically scale nodes. Runtime memory tuning still belongs to the workload team.

Original guide: page 33 · K07, K29, K30, K31, J01 (source directory).

CHAPTER 10 / 22

Scheduling, affinity and topology

Scheduling rules must match actual node capacity.



THE QUESTIONS IN THIS CHAPTER

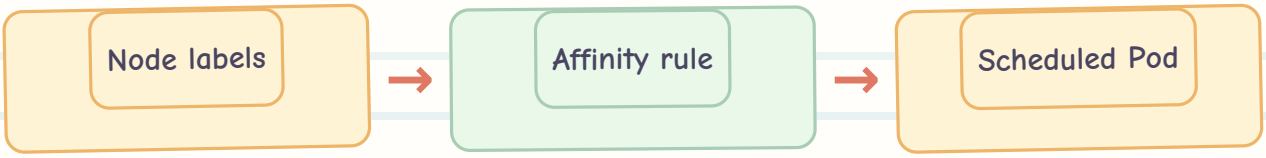
1. How do nodeSelector and node affinity differ?
2. What is the difference between a taint and a toleration?
3. How do Pod affinity, anti-affinity and topology spread differ?
4. Why can a Pod be Pending despite apparent free CPU?
5. What do PriorityClass and preemption provide?

Q046. How do nodeSelector and node affinity differ?

20-SECOND ANSWER • IN SIMPLE WORDS

nodeSelector requires exact labels; node affinity supports richer scheduling rules.

Interview precision: nodeSelector matches simple node labels; node affinity supports richer required and preferred placement expressions.



YAML / COMMAND EXAMPLE

```
nodeSelector:
  workload.example.com/pool: java
# Verify the eligible nodes:
kubectl get nodes --show-labels
```

WHAT REALLY HAPPENS · Only nodes with the declared label are eligible for this selector. Required affinity constrains placement; preferred affinity influences scoring without the same absolute gate. Resources and other constraints still apply.

INTERVIEWER FOLLOW-UP

What happens if every eligible node lacks memory for the request?

Answer: The Pod remains Pending until an eligible node gains enough allocatable memory or constraints change.

Watch out: User-controllable labels are not automatically trusted isolation evidence. Choose secured label practices for sensitive placement.

Original guide: page 34 · K12, K13, K28, K49 (source directory).

Q047. What is the difference between a taint and a toleration?

20-SECOND ANSWER • IN SIMPLE WORDS

Taints repel Pods unless tolerations allow them; tolerations do not force placement.

Interview precision: A node taint repels workloads under its effect; a matching Pod toleration permits placement despite that taint but does not require placement there.



YAML / COMMAND EXAMPLE

```
tolerations:
  - key: dedicated
    operator: Equal
    value: java
    effect: NoSchedule
# Pair with deliberate node selection if a dedicated pool is required.
```

WHAT REALLY HAPPENS · This Pod can tolerate that NoSchedule taint. It may still land elsewhere if no selection rule constrains it. NoExecute has eviction-related behavior beyond scheduling.

INTERVIEWER FOLLOW-UP

How do you make a dedicated node pool both eligible and exclusive?

Answer: Use node affinity to target the pool and taints/tolerations to keep unrelated Pods out.

Watch out: Tolerating all taints can expose workloads to nodes they were not intended to use. Avoid modifying shared node taints as a casual exercise.

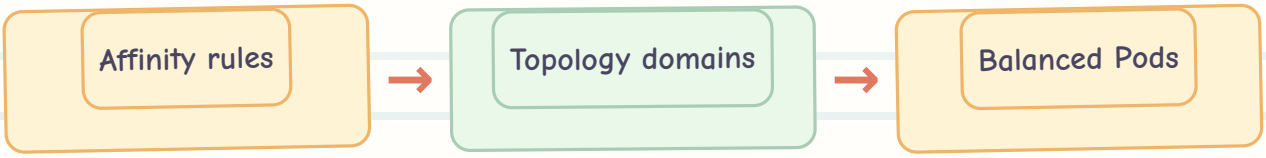
Original guide: page 35 · K12, K13, K28, K49 (source directory).

Q048. How do Pod affinity, anti-affinity and topology spread differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Affinity groups or separates Pods; topology spread distributes them across failure domains.

Interview precision: They express attraction to peer placement, separation from peer placement and balanced distribution across topology domains.



YAML / COMMAND EXAMPLE

```
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: DoNotSchedule
    labelSelector: {matchLabels: {app: orders}}
```

WHAT REALLY HAPPENS · The rule tries to distribute matching Pods across eligible node domains under the specified constraint. Zone-level spreading needs accurate zone labels and enough eligible capacity.

INTERVIEWER FOLLOW-UP

Which failure domain matters for your availability requirement?

Answer: Choose the zone for zone-failure resilience and the node for a single-machine failure.

Watch out: A strict spread rule can deliberately leave replicas Pending. Distribution across nodes is not distribution across independent zones.

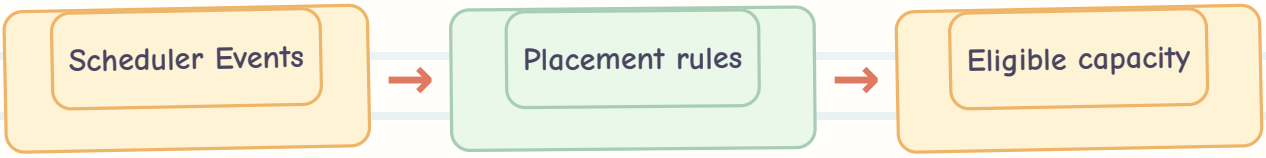
Original guide: page 35 · K12, K13, K28, K49 (source directory).

Q049. Why can a Pod be Pending despite apparent free CPU?

20-SECOND ANSWER • IN SIMPLE WORDS

Free CPU alone is not enough: requests, storage, taints and topology must all fit.

Interview precision: Placement uses declared requests and eligibility, not merely a current CPU utilization snapshot; storage, topology and policy can also block it.



YAML / COMMAND EXAMPLE

```
kubectl describe pod POD -n interview
kubectl get nodes
kubectl describe node CANDIDATE_NODE
kubectl get pvc -n interview
```

WHAT REALLY HAPPENS · Events can report insufficient requested resources, unmatched affinity, taints or unbound storage. Prioritize the actual scheduler message before changing resource settings.

INTERVIEWER FOLLOW-UP

What evidence justifies adding nodes rather than changing the workload?

Answer: Events showing insufficient capacity across eligible nodes justify adding nodes after checking requests.

Watch out: Lowering requests to force scheduling can create unstable oversubscription. A PVC with delayed binding can have an intentional scheduling dependency.

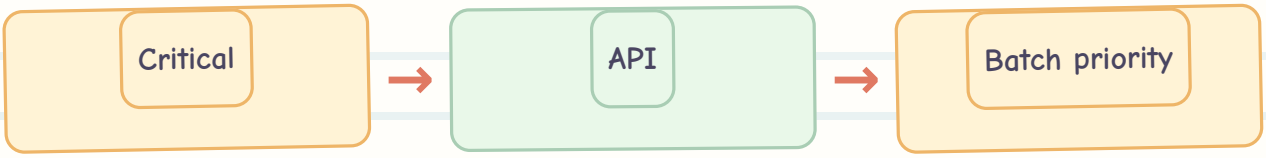
Original guide: page 36 · K12, K13, K28, K49 (source directory).

Q050. What do PriorityClass and preemption provide?

20-SECOND ANSWER • IN SIMPLE WORDS

Priority affects which pending Pods get preference and may allow lower-priority Pod preemption.

Interview precision: Priority influences scheduling order and can allow eligible lower-priority workloads to be displaced, subject to scheduler policy and constraints.



YAML / COMMAND EXAMPLE

```
priorityClassName: TEAM_APPROVED_PRIORITY
# Use an approved class rather than inventing a cluster-wide value.
# Inspect Pending Pod Events for preemption-related evidence.
```

WHAT REALLY HAPPENS · Preemption can free resources but cannot make an incompatible node label or storage constraint disappear. Priority should reflect business importance and team policy.

INTERVIEWER FOLLOW-UP

How would you allocate priority to APIs, batch jobs and critical platform services?

Answer: Protect cluster essentials first, then customer-facing APIs, then batch tasks; avoid unnecessary evictions.

Watch out: Raising every workload's priority does not create capacity and can harm other services. PDB interaction does not make preemption an absolute disruption-free guarantee.

Original guide: page 36 · K12, K13, K28, K49 (source directory).

CHAPTER 11 / 22

Volumes, claims and storage lifecycle

Storage life can be longer than Pod life.



THE QUESTIONS IN THIS CHAPTER

1. How do a PersistentVolume and PersistentVolumeClaim relate?
2. What does a StorageClass control?
3. What do RWO, ROX, RWX and RWOP mean?
4. How do emptyDir, hostPath and PVC-backed storage differ?
5. Does deleting a PVC always preserve or always delete the data?

Q051. How do a PersistentVolume and PersistentVolumeClaim relate?

20-SECOND ANSWER • IN SIMPLE WORDS

A PVC asks for storage; a PV is the allocated storage object to which it binds.

Interview precision: A PV represents available persistent storage; a PVC requests supported capacity and access, then binds to a suitable volume.



YAML / COMMAND EXAMPLE

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests: {storage: 1Gi}
```

WHAT REALLY HAPPENS · An eligible StorageClass can dynamically provision the backend. The Pod mounts the claim, not a guessed node directory. Binding depends on class, capacity, access mode and topology.

INTERVIEWER FOLLOW-UP

What would you inspect before changing a stuck claim's requested size?

Answer: Inspect StorageClass resize support, volume mode, binding state, capacity, events and current data first.

Watch out: An accepted claim can remain Pending if no suitable provisioner or volume is available.

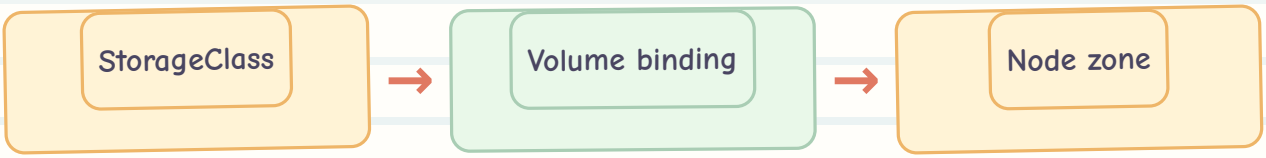
Original guide: page 37 · K08, K24, K25 (source directory).

Q052. What does a StorageClass control?

20-SECOND ANSWER • IN SIMPLE WORDS

StorageClass chooses the provisioner and storage behavior, including when binding waits.

Interview precision: A StorageClass names provisioning policy, including a supported provisioner and settings such as binding mode, reclaim behavior and expansion capability.



YAML / COMMAND EXAMPLE

```
kubectl get storageclass
kubectl describe storageclass CLASS
kubectl describe pvc data -n interview
# WaitForFirstConsumer can delay binding until placement is known.
```

WHAT REALLY HAPPENS · Delayed binding helps choose storage compatible with the Pod's topology. Different classes can have different performance and durability contracts. Select a class intentionally rather than assuming all persistent storage is equivalent.

INTERVIEWER FOLLOW-UP

Why can immediate provisioning place a volume in the wrong zone?

Answer: Immediate binding can create storage before a node/zone is chosen, making the Pod unschedulable.

Watch out: Setting an expansion flag does not make an unsupported backend resize correctly. Provider and filesystem behavior matter.

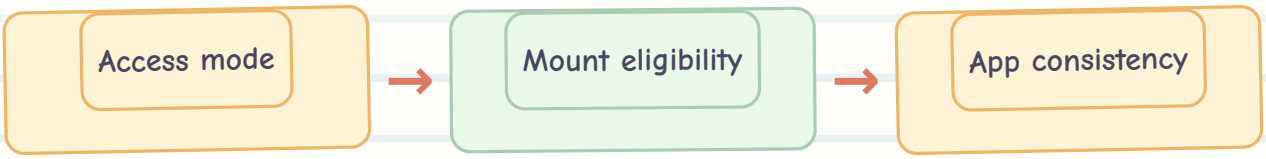
Original guide: page 38 · K08, K24, K25 (source directory).

Q053. What do RWO, ROX, RWX and RWOP mean?

20-SECOND ANSWER • IN SIMPLE WORDS

Access modes describe how a volume can be mounted; they do not make an application multi-writer safe.

Interview precision: They request supported mount-access patterns, with ReadWriteOnce referring to one node rather than necessarily one Pod; ReadWriteOncePod provides a supported single-Pod constraint.



YAML / COMMAND EXAMPLE

```
ReadWriteOnce:     read/write from one node
ReadOnlyMany:      read-only from multiple nodes
ReadWriteMany:     read/write from multiple nodes
ReadWriteOncePod:  supported CSI single-Pod access
```

WHAT REALLY HAPPENS · Multiple Pods on one node can sometimes share an RWO volume. The storage driver must support the requested mode. Access modes do not design database locking or make concurrent writers safe.

INTERVIEWER FOLLOW-UP

Why is a shared database data directory unsafe despite successful mounts?

Answer: Two database writers can corrupt files even when mounted successfully; use a database replication protocol.

Watch out: RWX does not mean arbitrary applications can concurrently modify the same data safely. RWOP has driver/version requirements.

Original guide: page 38 · K08, K24, K25 (source directory).

Q054. How do emptyDir, hostPath and PVC-backed storage differ?

20-SECOND ANSWER • IN SIMPLE WORDS

emptyDir lasts for a Pod, hostPath uses node files, and a PVC follows storage's own lifecycle.

Interview precision: emptyDir follows Pod lifetime, hostPath exposes a node path, and a PVC binds workload storage to a persistent-storage contract.



YAML / COMMAND EXAMPLE

```
volumes:
  - name: scratch
    emptyDir: {sizeLimit: 64Mi}
# Mount scratch at an application temp path.
```

WHAT REALLY HAPPENS · emptyDir survives a container restart within the Pod but disappears with Pod removal. A hostPath ties data and access to a node, creating portability and security concerns. A PVC can preserve data across Pod replacement under its backend policy.

INTERVIEWER FOLLOW-UP

What is a safe temp-storage choice for a replaceable Java API?

Answer: Use size-limited emptyDir for disposable temporary Java files, with a controlled memory/disk budget.

Watch out: Memory-backed emptyDir contributes to memory use; disk-backed scratch still has node and ephemeral-storage limits.

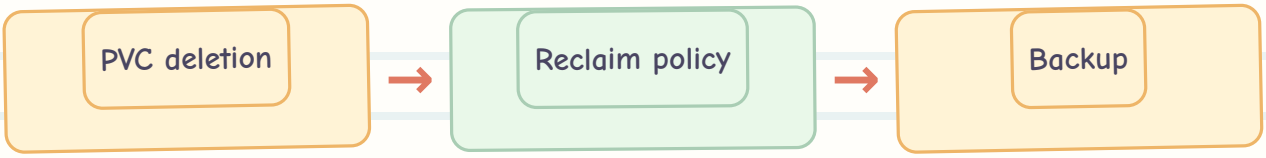
Original guide: page 39 · K08, K24, K25 (source directory).

Q055. Does deleting a PVC always preserve or always delete the data?

20-SECOND ANSWER • IN SIMPLE WORDS

Deleting a PVC can delete or retain backing storage depending on reclaim policy.

Interview precision: The resulting behavior depends on reclaim policy, backend, protection/finalization and workload ownership; deletion is a data-lifecycle decision.



YAML / COMMAND EXAMPLE

```
kubectl get pv
kubectl get pv VOLUME -o yaml
# Inspect persistentVolumeReclaimPolicy and provider policy.
# Test restore before retiring real business storage.
```

WHAT REALLY HAPPENS · Delete can request backend removal, while Retain leaves an administrative recovery and cleanup task. A retained backend is not automatically safe, backed up or reusable without review.

INTERVIEWER FOLLOW-UP

Which recovery objectives should guide your backup and reclaim policy?

Answer: Set recovery-point and recovery-time goals, backups and tested restores independently from reclaim policy.

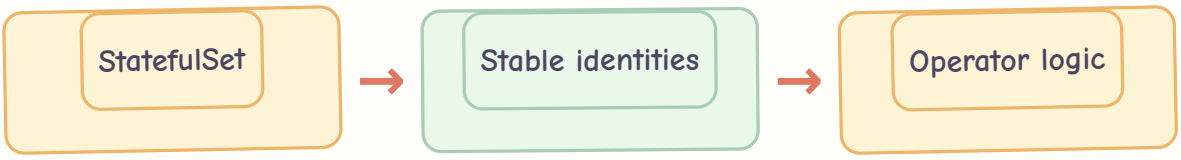
Watch out: Deleting a lab namespace can delete its PVC and trigger backend deletion. StatefulSet claim retention has additional configuration.

Original guide: page 39 · K08, K24, K25 (source directory).

CHAPTER 12 / 22

StatefulSets, DaemonSets, Jobs and CronJobs

Match controller type to workload behavior.



THE QUESTIONS IN THIS CHAPTER

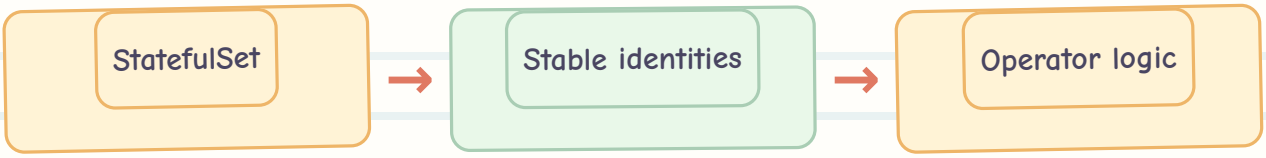
1. When should you choose StatefulSet instead of Deployment?
2. What is a headless Service's role with StatefulSet?
3. What does a DaemonSet do?
4. How is a Job different from a Deployment?
5. Does a CronJob guarantee exactly-once execution?

Q056. When should you choose StatefulSet instead of Deployment?

20-SECOND ANSWER • IN SIMPLE WORDS

Use StatefulSet for stable Pod identities and ordered stateful lifecycle, not as a substitute for replication.

Interview precision: Use StatefulSet when supported stable identities, ordered lifecycle or per-replica storage matter; choose Deployment for interchangeable application replicas.



YAML / COMMAND EXAMPLE

```
orders-api replicas -> Deployment
db-0, db-1 identities -> possible StatefulSet
Per-member PVCs -> explicit stable storage ownership
```

WHAT REALLY HAPPENS · Stable ordinals and suitable discovery can support a clustered application. The database software still needs explicit replication, membership and failover configuration.

INTERVIEWER FOLLOW-UP

What would a database operator add beyond a bare StatefulSet?

Answer: An operator can handle backups, leader changes, upgrades, repair and application-aware failover.

Watch out: StatefulSet does not automatically make a database highly available or synchronize its data.

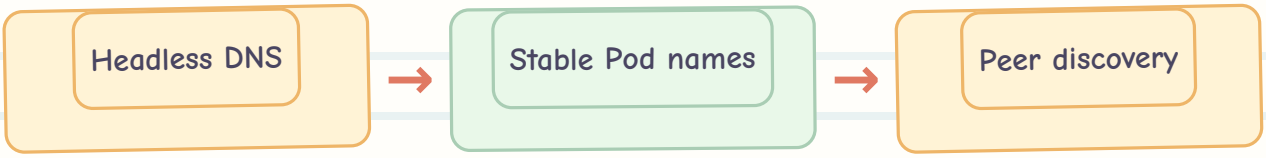
Original guide: page 40 · K14, K15, K16, K17, K20 (source directory).

Q057. What is a headless Service's role with StatefulSet?

20-SECOND ANSWER • IN SIMPLE WORDS

Headless Service exposes individual stable members instead of a single virtual IP.

Interview precision: It supports discovery of individual member identities rather than hiding them behind only a virtual service address.



YAML / COMMAND EXAMPLE

```
spec:
  clusterIP: None
  selector: {app: db}
  ports: [{port: 5432, targetPort: 5432}]
# StatefulSet serviceName must follow the intended discovery contract.
```

WHAT REALLY HAPPENS · Clients or peer nodes can discover particular members through supported DNS patterns. Readiness publication and startup membership requirements must be designed for the application.

INTERVIEWER FOLLOW-UP

How would Kafka member identity and advertised endpoints fit this model?

Answer: Use stable member DNS in Kafka configuration, with correct advertised listener addresses and broker identities.

Watch out: A stable DNS name does not guarantee the member is ready or holds the leader role. Do not hard-code a current Pod IP.

Original guide: page 40 · K14, K15, K16, K17, K20 (source directory).

Q058. What does a DaemonSet do?

20-SECOND ANSWER • IN SIMPLE WORDS

DaemonSet ensures a node-level agent runs on each eligible node.

Interview precision: It maintains a Pod on each eligible node, commonly for node-local infrastructure such as log agents or networking components.



YAML / COMMAND EXAMPLE

```
Node A -> log agent Pod
Node B -> log agent Pod
Node C -> log agent Pod
# Eligibility can be restricted by selection and tolerations.
```

WHAT REALLY HAPPENS · Adding an eligible node can add another agent automatically. Resource usage is multiplied by eligible node count, and required host access must be reviewed carefully.

INTERVIEWER FOLLOW-UP

How would you safely roll out an agent that reads host logs?

Answer: Roll out gradually, limit host permissions, monitor agent overhead and verify log ingestion.

Watch out: A DaemonSet is not a general API horizontal-scaling strategy. Node agents can have substantial privileges.

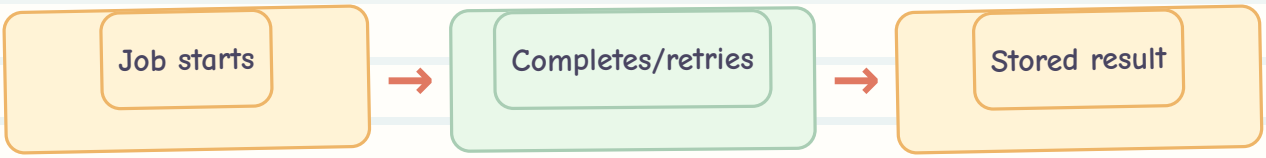
Original guide: page 41 · K14, K15, K16, K17, K20 (source directory).

Q059. How is a Job different from a Deployment?

20-SECOND ANSWER • IN SIMPLE WORDS

A Job finishes bounded work; a Deployment keeps service replicas running.

Interview precision: A Job tracks successful completion of bounded work; a Deployment maintains running application replicas.



YAML / COMMAND EXAMPLE

```
spec:
  backoffLimit: 2
  activeDeadlineSeconds: 60
  template:
    spec:
      restartPolicy: Never
      containers:
        - {name: task, image: busybox:1.37, command: ["echo", "done"]}
```

WHAT REALLY HAPPENS · The Job can retry failed attempts within its policy. Limits bound the exercise, but a retry can repeat effects already performed before failure. Build idempotency into real work.

INTERVIEWER FOLLOW-UP

How would you make a payment reconciliation job safe to retry?

Answer: Use unique reconciliation IDs, durable checkpoints and deduplication so a retry does not pay twice.

Watch out: A successful Pod exit does not prove the business operation was correct unless the job verifies it.

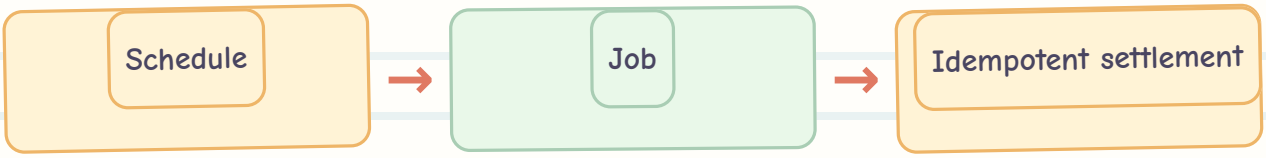
Original guide: page 41 · K14, K15, K16, K17, K20 (source directory).

Q060. Does a CronJob guarantee exactly-once execution?

20-SECOND ANSWER • IN SIMPLE WORDS

A CronJob schedules Jobs but cannot guarantee that the business effect happens exactly once.

Interview precision: No. Scheduling and retry behavior require the underlying job to tolerate missed, duplicated or overlapping execution under the selected policy.



YAML / COMMAND EXAMPLE

```
schedule: "*/5 * * * *"
concurrencyPolicy: Forbid
startingDeadlineSeconds: 120
# Use a supported timeZone field if explicit timezone policy is required.
```

WHAT REALLY HAPPENS · Forbid prevents certain overlaps for that CronJob but does not establish a distributed business lock across unrelated jobs or clusters. Deadline and scheduling policy define what happens after a delay.

INTERVIEWER FOLLOW-UP

How would you recover a missed daily settlement without double processing?

Answer: Track settlement date in durable state, reconcile missed dates and reject duplicate processing.

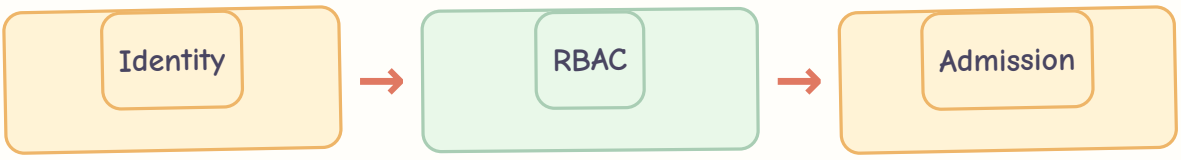
Watch out: A trigger every five minutes is not a transaction guarantee. Track business windows and completed work durably.

Original guide: page 42 · K14, K15, K16, K17, K20 (source directory).

CHAPTER 13 / 22

Authentication, RBAC and workload identity

Least privilege is part of the system design.



THE QUESTIONS IN THIS CHAPTER

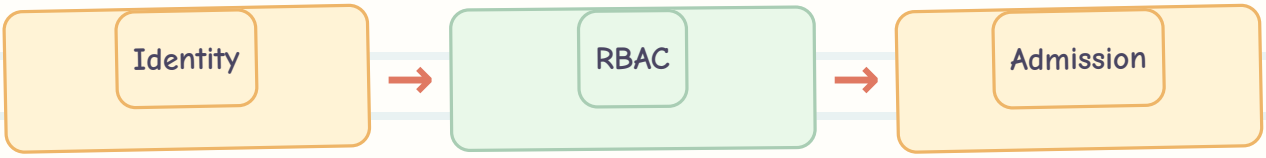
1. How do authentication, authorization and admission differ?
2. How do Role, ClusterRole, RoleBinding and ClusterRoleBinding differ?
3. What is a ServiceAccount used for?
4. How do securityContext and Pod Security Admission work together?
5. Why are Pod creation, exec and cluster-admin sensitive permissions?

Q061. How do authentication, authorization and admission differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Authentication proves identity; authorization checks permission; admission validates or changes a request.

Interview precision: Authentication identifies the caller, authorization decides whether the requested API action is allowed, and admission can validate or modify admitted objects.



YAML / COMMAND EXAMPLE

```
Identity -> permission decision -> admission -> stored object
kubectl auth can-i create deployments -n interview
kubectl auth can-i get secrets -n interview
```

WHAT REALLY HAPPENS · A caller may have permission to create a Pod yet be rejected by resource or security policy. Separating these gates makes Forbidden and admission errors easier to explain.

INTERVIEWER FOLLOW-UP

How would you debug an allowed action whose manifest is rejected?

Answer: Check admission webhook messages or namespace policy after confirming RBAC allowed the API action.

Watch out: A Kubernetes API identity does not automatically define the logged-in user for your HTTP application.

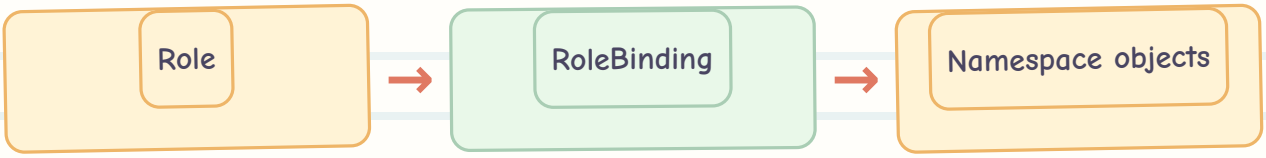
Original guide: page 43 · K09, K26, K27, K33 (source directory).

Q062. How do Role, ClusterRole, RoleBinding and ClusterRoleBinding differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Role/RoleBinding scope permissions in a namespace; ClusterRole may define broader rules.

Interview precision: Roles define namespaced permissions, ClusterRoles define reusable or cluster-scoped permissions, and bindings grant them to subjects at a selected scope.



YAML / COMMAND EXAMPLE

```
# Role rule fragment:
rules:
  - apiGroups: [""]
    resources: [configmaps]
    verbs: [get, list]
```

WHAT REALLY HAPPENS · A RoleBinding can grant a ClusterRole within one namespace. A ClusterRoleBinding grants its permissions cluster-wide. The example allows ConfigMap reads, not Secret reads or Deployment changes.

INTERVIEWER FOLLOW-UP

Can a RoleBinding give access to cluster-scoped nodes?

Answer: A namespaced RoleBinding cannot grant access to cluster-scoped nodes, even if it references a ClusterRole.

Watch out: Wildcards and broad bindings expand authority as APIs grow. Review what a subject can indirectly do, not only explicit Secret reads.

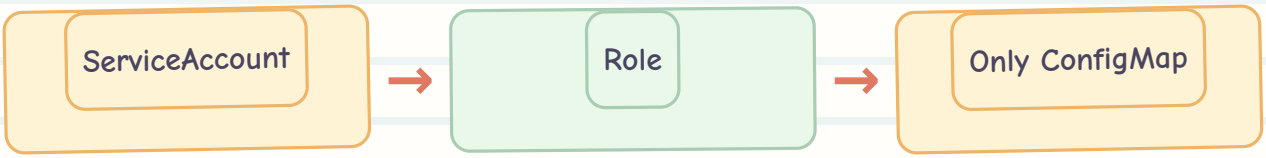
Original guide: page 44 · K09, K26, K27, K33 (source directory).

Q063. What is a ServiceAccount used for?

20-SECOND ANSWER • IN SIMPLE WORDS

A ServiceAccount gives a workload its Kubernetes identity when API access is needed.

Interview precision: A ServiceAccount identifies a workload to Kubernetes API authentication and can participate in supported external workload-identity integration.



YAML / COMMAND EXAMPLE

```
serviceAccountName: orders-api
automountServiceAccountToken: false
# Use false when the application does not need the Kubernetes API.
```

WHAT REALLY HAPPENS · An ordinary order API may need no API token at all. If access is required, give its own account narrowly scoped permissions and use supported short-lived projected-token behavior.

INTERVIEWER FOLLOW-UP

How would you let a job read one configuration object without granting cluster-admin?

Answer: Bind a narrow Role allowing get on only the named ConfigMap to that job's ServiceAccount.

Watch out: Namespace default accounts are not a least-privilege design by themselves. Cloud workload identity also needs provider configuration.

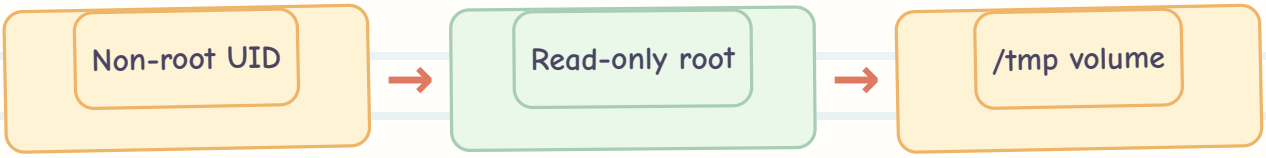
Original guide: page 44 · K09, K26, K27, K33 (source directory).

Q064. How do securityContext and Pod Security Admission work together?

20-SECOND ANSWER • IN SIMPLE WORDS

SecurityContext sets process security; admission policy rejects Pods that violate required controls.

Interview precision: securityContext requests runtime settings; admission policy can reject workloads that do not meet the namespace's security standard.



YAML / COMMAND EXAMPLE

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  seccompProfile: {type: RuntimeDefault}
# Container settings also include allowPrivilegeEscalation: false.
```

WHAT REALLY HAPPENS · Runtime identity and syscall policy constrain execution. A non-root declaration must match the image contract, file permissions and mount access. Pod-level and container-level settings apply to different fields.

INTERVIEWER FOLLOW-UP

What writable path does your Java image need under a read-only root filesystem?

Answer: Mount a writable /tmp emptyDir and avoid writing to the read-only root filesystem.

Watch out: Passing admission does not prove the application is secure or that a mounted host path is harmless.

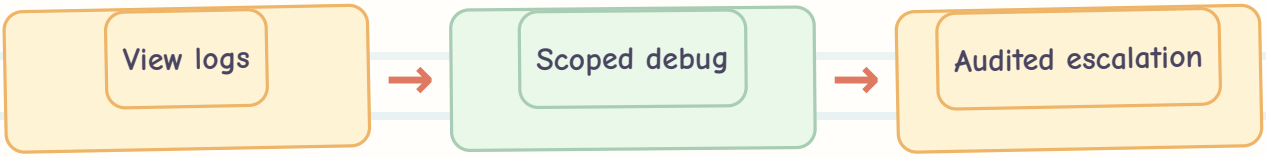
Original guide: page 45 · K09, K26, K27, K33 (source directory).

Q065. Why are Pod creation, exec and cluster-admin sensitive permissions?

20-SECOND ANSWER • IN SIMPLE WORDS

Pod creation, exec and admin privileges can expose data or control other workloads.

Interview precision: They can expose workload credentials, modify execution or control broad cluster resources, including indirect access beyond a simple read rule.



YAML / COMMAND EXAMPLE

```
Review subjects that can:
  create Pods consuming Secrets
  exec into credential-bearing workloads
  bind/escalate privileged roles or alter policy
```

WHAT REALLY HAPPENS · A subject denied direct get secrets may still access a Secret through an allowed Pod. Exec permission can reach an application's files and process environment. Use separate operational identities and audit sensitive actions.

INTERVIEWER FOLLOW-UP

How would you provide production diagnosis access with a reviewed escalation path?

Answer: Use read-only diagnostics by default and time-limited, audited escalation with approval.

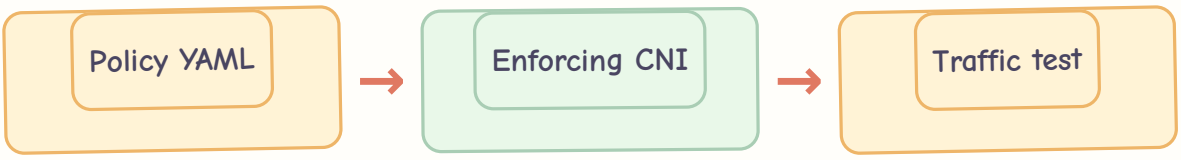
Watch out: A namespace is not a sufficient boundary when a subject can gain node or cluster authority.

Original guide: page 45 · K09, K26, K27, K33 (source directory).

CHAPTER 14 / 22

NetworkPolicy and layered traffic security

Verify both policy definition and actual enforcement.



THE QUESTIONS IN THIS CHAPTER

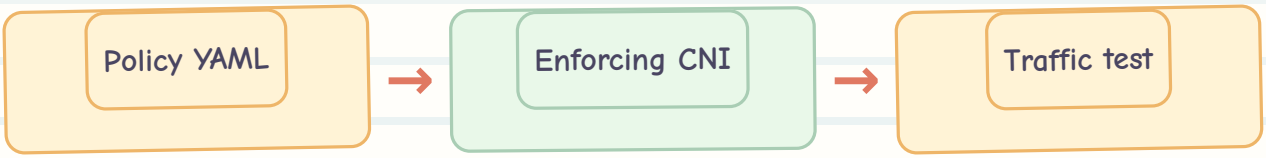
1. What does NetworkPolicy control?
2. Are NetworkPolicy allow rules ordered firewall rules?
3. How do namespaceSelector and podSelector combine?
4. Why does deny-all egress often break otherwise healthy applications?
5. How are NetworkPolicy, TLS, mTLS and application authorization different?

Q066. What does NetworkPolicy control?

20-SECOND ANSWER • IN SIMPLE WORDS

NetworkPolicy selects Pod traffic allowed by an enforcing network implementation.

Interview precision: It restricts supported ingress and egress traffic for selected Pods when the network implementation enforces the policy API.



YAML / COMMAND EXAMPLE

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: deny-incoming}
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

WHAT REALLY HAPPENS · This selects Pods in its namespace and allows no ingress under this policy. It leaves egress unchanged unless other egress-isolating policies apply. Verify the CNI's actual support with traffic tests.

INTERVIEWER FOLLOW-UP

How would you distinguish policy parsing from policy enforcement?

Answer: Create the policy, then run allowed and denied traffic tests; a parsed policy may not be enforced.

Watch out: Creating a policy object on a non-enforcing network does not prove isolation.

Original guide: page 46 · K18, K09 (source directory).

Q067. Are NetworkPolicy allow rules ordered firewall rules?

20-SECOND ANSWER • IN SIMPLE WORDS

Matching policies add allowed traffic together; they are not ordered deny/allow firewall rules.

Interview precision: No. Applicable allow policies are additive; a connection must meet relevant source egress and destination ingress requirements.



YAML / COMMAND EXAMPLE

```
Policy A allows app=api to database TCP 5432.
Policy B allows an approved maintenance client.
Their allowed ingress sets combine for selected destinations.
```

WHAT REALLY HAPPENS · A broad extra policy can reopen access that a narrow policy was meant to restrict. Review all policies selecting each endpoint, including the source's egress isolation.

INTERVIEWER FOLLOW-UP

Why might adding a second policy unexpectedly make a blocked client work?

Answer: A second policy may permit traffic already denied by the first because allow results are additive.

Watch out: NetworkPolicy is not evaluated as first matching deny/allow lines in a conventional ordered ACL.

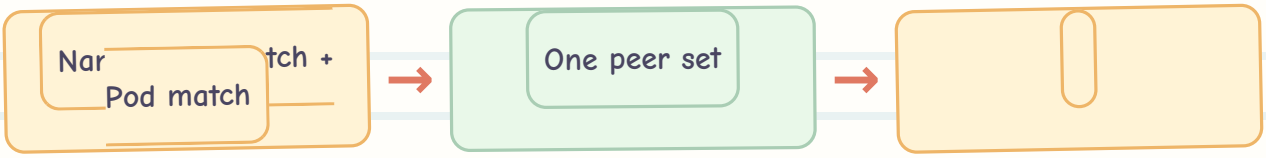
Original guide: page 47 · K18, K09 (source directory).

Q068. How do namespaceSelector and podSelector combine?

20-SECOND ANSWER • IN SIMPLE WORDS

Combining namespaceSelector and podSelector in one rule selects labeled Pods within selected namespaces.

Interview precision: Selectors in the same peer entry are combined; separate peer entries represent alternative allowed sources.



YAML / COMMAND EXAMPLE

```
ingress:
  - from:
      - namespaceSelector:
          matchLabels: {kubernetes.io/metadata.name: apps}
        podSelector:
          matchLabels: {role: api}
    ports: [{protocol: TCP, port: 5432}]
```

WHAT REALLY HAPPENS · The peer must be a role=api Pod in the apps namespace. Splitting the two selectors into separate entries would allow a broader union. This rule fragment needs the destination selection and policy declaration.

INTERVIEWER FOLLOW-UP

What would separate entries allow that this combined entry does not?

Answer: Separate peer entries allow either matching namespaces or matching Pods, a wider traffic set.

Watch out: Label-management authority matters because labels can affect network eligibility.

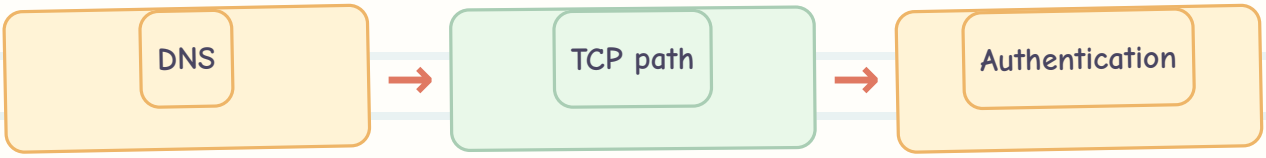
Original guide: page 47 · K18, K09 (source directory).

Q069. Why does deny-all egress often break otherwise healthy applications?

20-SECOND ANSWER • IN SIMPLE WORDS

Denying egress also blocks DNS and required upstream connections unless allowed.

Interview precision: It can block DNS and required dependency paths as well as unwanted traffic; every necessary route needs a supported explicit allowance.



YAML / COMMAND EXAMPLE

```
Required egress may include:
  DNS queries: UDP and TCP 53 to the actual resolver path
  Database: approved destination and port
  HTTPS dependencies: architecture-specific controlled access
```

WHAT REALLY HAPPENS · DNS failure can hide the true policy issue behind an unknown-host exception. Resolver placement, host networking and CNI behavior affect a correct rule. Standard NetworkPolicy does not provide portable domain-name authorization.

INTERVIEWER FOLLOW-UP

How would you verify DNS, TCP connection and application authentication separately?

Answer: Resolve DNS, test TCP to the destination, then check TLS and application credentials independently.

Watch out: Do not permit all egress merely to make tests pass. IP-based controls for changing external services need a deliberate design.

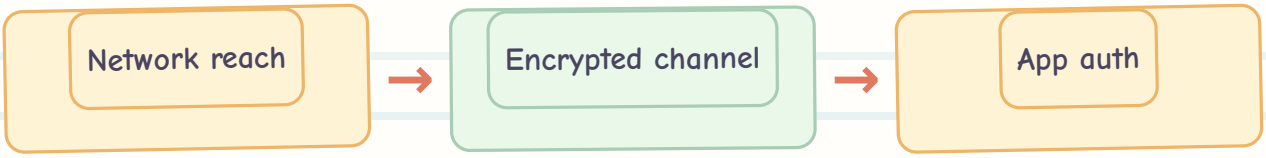
Original guide: page 48 · K18, K09 (source directory).

Q070. How are NetworkPolicy, TLS, mTLS and application authorization different?

20-SECOND ANSWER • IN SIMPLE WORDS

NetworkPolicy limits reachability, TLS encrypts traffic, mTLS verifies peers, and authorization checks users.

Interview precision: NetworkPolicy limits network reach, TLS protects supported transport, mTLS authenticates transport peers, and application policy authorizes business actions.



YAML / COMMAND EXAMPLE

```
Reachability: may this Pod connect to this port?
Transport:   is this encrypted and who is the peer?
Application: may this caller cancel this order?
```

WHAT REALLY HAPPENS · An allowed TCP connection is not authorization to read customer data. A service mesh can automate some transport policy but still needs identity, certificate and operational controls.

INTERVIEWER FOLLOW-UP

Which controls protect an internal Spring endpoint containing personal data?

Answer: Use restricted network paths, TLS/mTLS and app-level object authorization with audited access.

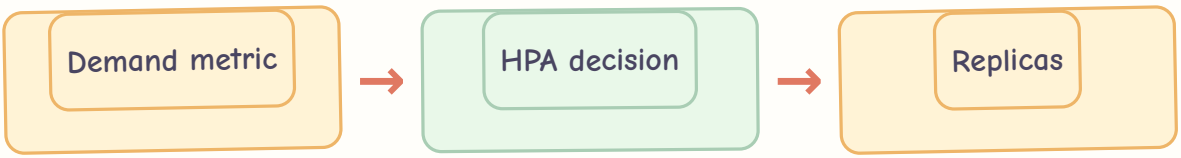
Watch out: Kubernetes does not provide every layer merely because Pods are in separate namespaces.

Original guide: page 48 · K18, K09 (source directory).

CHAPTER 15 / 22

Autoscaling and downstream capacity

Scaling without downstream capacity can be harmful.



THE QUESTIONS IN THIS CHAPTER

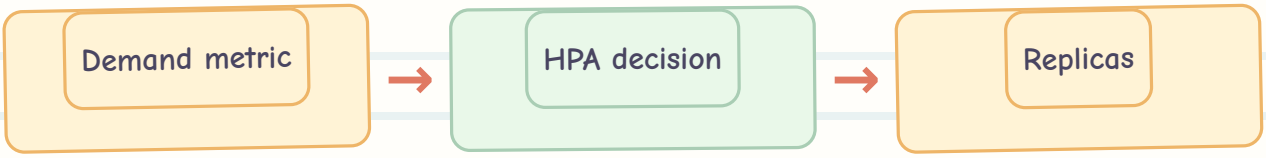
1. What does the Horizontal Pod Autoscaler do?
2. How is CPU utilization calculated for a typical HPA target?
3. When is a queue or demand metric better than CPU?
4. How do HPA, VPA and node autoscaling differ?
5. How do you prevent autoscaling from causing oscillation or overload?

Q071. What does the Horizontal Pod Autoscaler do?

20-SECOND ANSWER • IN SIMPLE WORDS

HPA changes desired replicas using metrics; it does not directly add nodes.

Interview precision: HPA adjusts a target workload's replica count using supported observed metrics and configured scaling policy.



YAML / COMMAND EXAMPLE

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
# Resource metric: CPU utilization relative to requests.
# Custom/external metrics require the relevant metrics integration.
```

WHAT REALLY HAPPENS · CPU-based scaling usually requires Metrics Server and meaningful requests for the relevant containers. HPA controls replicas; it does not itself provision nodes or solve a saturated database.

INTERVIEWER FOLLOW-UP

What would you inspect when the HPA reports an unknown utilization value?

Answer: Check metrics-server/custom metrics availability, Pod requests and HPA conditions/events.

Watch out: Missing or stale metrics can prevent the intended decision. Inspect HPA conditions and events.

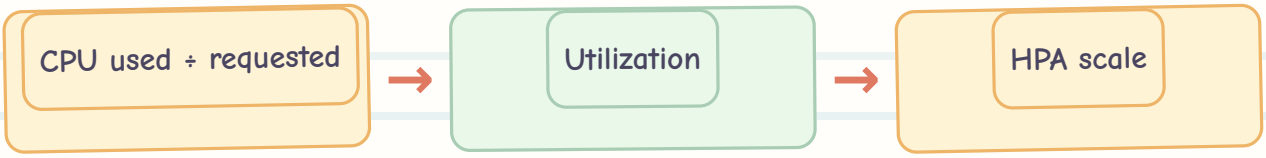
Original guide: page 49 · K19, K37, K38, K07 (source directory).

Q072. How is CPU utilization calculated for a typical HPA target?

20-SECOND ANSWER • IN SIMPLE WORDS

Typical CPU HPA usage compares measured CPU to requested CPU for eligible Pods.

Interview precision: The resource-utilization metric compares observed CPU use against the configured CPU request, not the CPU limit.



YAML / COMMAND EXAMPLE

```
CPU request: 250m
Observed use: 150m
Utilization: 150 / 250 = 60 percent
```

WHAT REALLY HAPPENS · Changing the request changes the metric denominator and scheduling budget. Keep resource tuning and autoscaling policy coordinated. The actual controller also accounts for readiness, missing data, tolerance and policy limits.

INTERVIEWER FOLLOW-UP

Why could lowering CPU requests unexpectedly trigger more replicas?

Answer: Reducing the request raises utilization percentage for identical CPU usage and may scale sooner.

Watch out: Do not interpret 60 percent as 60 percent of an entire physical machine.

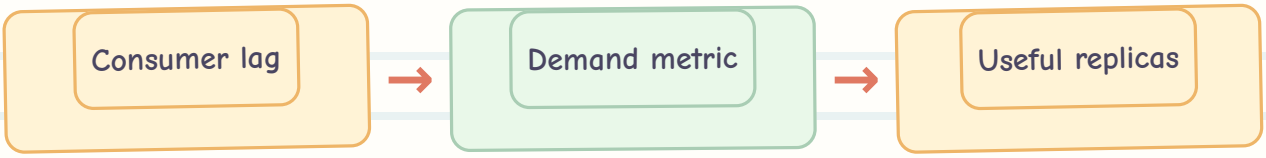
Original guide: page 50 · K19, K37, K38, K07 (source directory).

Q073. When is a queue or demand metric better than CPU?

20-SECOND ANSWER • IN SIMPLE WORDS

Queue size/age can show overload when CPU does not match work waiting.

Interview precision: Use a metric closer to the bottleneck when waiting work, latency or consumption demand is not reflected reliably by CPU use.



YAML / COMMAND EXAMPLE

```
Kafka worker: lag plus processing rate and partition parallelism
HTTP API:     concurrency, latency and CPU under measured demand
Batch work:   queued work with bounded downstream capacity
```

WHAT REALLY HAPPENS · A CPU-idle consumer can still be blocked on a dependency while lag grows. More workers may help only until partition and downstream limits are reached. Any external metric needs a trusted adapter and meaningful stale-data behavior.

INTERVIEWER FOLLOW-UP

What caps useful replicas for a Kafka consumer group?

Answer: Kafka consumer-group partition count and downstream capacity cap useful consumer parallelism.

Watch out: Lag alone can be misleading during rebalances or long tasks. Scaling must not create unbounded database or API concurrency.

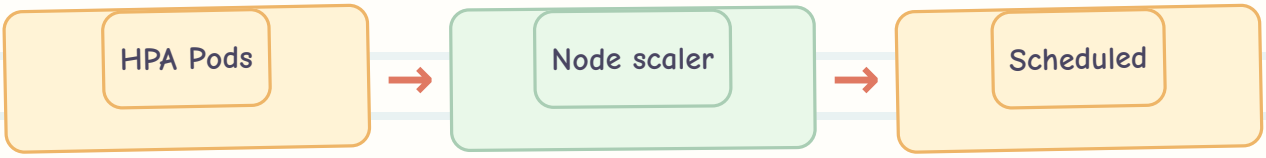
Original guide: page 50 · K19, K37, K38, K07 (source directory).

Q074. How do HPA, VPA and node autoscaling differ?

20-SECOND ANSWER • IN SIMPLE WORDS

HPA changes replica counts, VPA recommends/adjusts Pod resources, and node autoscaling adds compute.

Interview precision: HPA changes replica count, VPA recommends or adjusts workload resource sizing, and node autoscaling changes available node capacity through its integration.



YAML / COMMAND EXAMPLE

```
HPA:  more/fewer application replicas
VPA:  resource sizing for workload execution
Nodes: capacity for eligible scheduled workloads
```

WHAT REALLY HAPPENS · Vertical updates can use different rollout or resize modes depending on the installed implementation and feature support. Coordinate CPU-resource recommendations with CPU-utilization HPA to avoid a moving denominator.

INTERVIEWER FOLLOW-UP

What happens if HPA requests more Pods but no eligible nodes can fit them?

Answer: Extra Pods wait Pending until a node scaler adds eligible capacity or resource constraints change.

Watch out: None of these guarantees storage compatibility, instant startup or a responsive dependency.

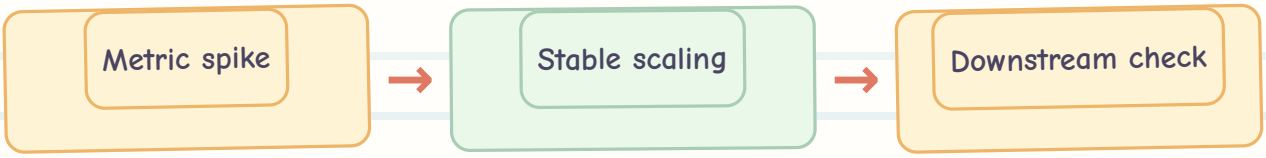
Original guide: page 50 · K19, K37, K38, K07 (source directory).

Q075. How do you prevent autoscaling from causing oscillation or overload?

20-SECOND ANSWER • IN SIMPLE WORDS

Keep scale decisions stable and protect databases, queues and other shared services.

Interview precision: Use measured thresholds, stabilization and rate policy, while bounding expensive concurrency and validating scaling transitions.



YAML / COMMAND EXAMPLE

```
API replicas: 8
DB pool per replica: 20
Potential API connections: 160, plus other clients
# Reserve dependency and node headroom for failure and rollout.
```

WHAT REALLY HAPPENS · A scale-up can stress the database before it reduces HTTP pressure. Scale-down needs connection and worker drain. Test spike response, recovery and sustained load rather than only steady throughput.

INTERVIEWER FOLLOW-UP

Which metrics would stop a harmful scale-up and guide a different mitigation?

Answer: Watch connection saturation, p99 latency, error rate and throttling; cap growth and fix shared bottlenecks.

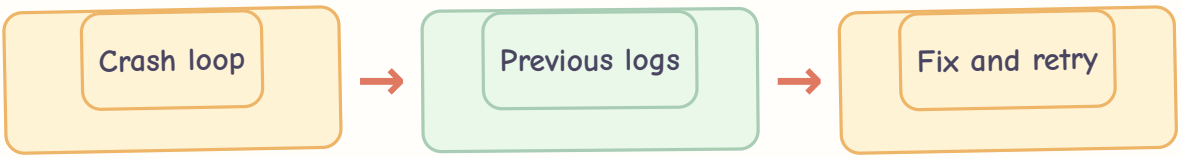
Watch out: Adding replicas is not a repair for a shared lock, schema issue or external outage.

Original guide: page 51 · K19, K37, K38, K07 (source directory).

CHAPTER 16 / 22

Observability and evidence-driven diagnosis

Investigate evidence before restarting everything.



THE QUESTIONS IN THIS CHAPTER

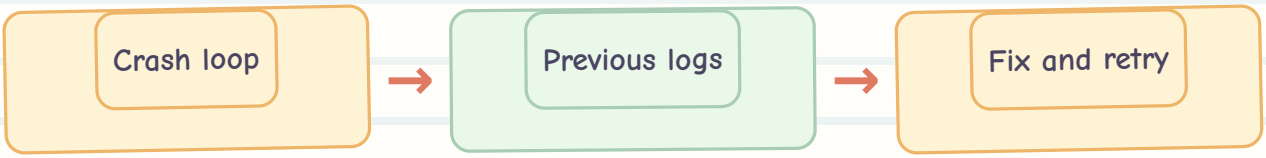
1. How do you diagnose CrashLoopBackOff?
2. How do you diagnose ImagePullBackOff?
3. What should you check for a Service with no working traffic?
4. How do you debug a minimal image without a shell?
5. What is the difference between monitoring and observability in practice?

Q076. How do you diagnose CrashLoopBackOff?

20-SECOND ANSWER • IN SIMPLE WORDS

Find why containers are restarting using events, previous logs, probes and exit codes.

Interview precision: Inspect previous container logs, last termination state, command/configuration and probe events before changing restart behavior.



YAML / COMMAND EXAMPLE

```
kubectl describe pod POD -n interview
kubectl logs POD -n interview -c api --previous
kubectl get pod POD -n interview -o yaml
```

WHAT REALLY HAPPENS · A missing secret, JVM flag error, permission denial or failing liveness probe can cause repeated restarts. Backoff describes retry pacing, not the root cause. Check the exact image and admitted configuration.

INTERVIEWER FOLLOW-UP

How would you prove the next Pod will start with corrected inputs?

Answer: Compare new image digest, config, Secret mount, runtime command and probe behavior before rollout.

Watch out: Deleting the Pod can remove useful local evidence without repairing its template.

Original guide: page 52 · K34, K35, K39, K40, K46 (source directory).

Q077. How do you diagnose ImagePullBackOff?

20-SECOND ANSWER • IN SIMPLE WORDS

ImagePullBackOff means the node could not fetch an image; inspect registry, tag and credentials.

Interview precision: Read pull events and inspect the image reference, registry access, credentials, architecture and pull policy.



YAML / COMMAND EXAMPLE

```
kubectl describe pod POD -n interview
kubectl get secret REGISTRY_SECRET -n interview
# Inspect imagePullSecrets names without printing secret data.
```

WHAT REALLY HAPPENS · Not-found and unauthorized errors require different repairs. A private-registry Secret must be accessible in the workload namespace under the intended reference. A local minikube image has a separate loading and pull-policy contract.

INTERVIEWER FOLLOW-UP

Why might an image visible to Docker on the laptop be invisible to minikube?

Answer: Minikube may use a separate container runtime/image store from laptop Docker; load into the correct node.

Watch out: Retrying an invalid tag does not fix it. Avoid exposing credentials while collecting evidence.

Original guide: page 52 · K34, K35, K39, K40, K46 (source directory).

Q078. What should you check for a Service with no working traffic?

20-SECOND ANSWER • IN SIMPLE WORDS

Trace client name, Service selector, ready endpoints, targetPort and Pod listener.

Interview precision: Trace DNS, selector matches, ready EndpointSlices, port mapping, actual listener and applicable ingress/egress policy.



YAML / COMMAND EXAMPLE

```
kubectl get svc orders -n interview -o yaml
kubectl get pods -n interview -l app=orders -o wide
kubectl get endpointslices -n interview \
  -l kubernetes.io/service-name=orders
```

WHAT REALLY HAPPENS · If no Pod matches the selector, network debugging is premature. If the direct Pod port works but Service access fails, investigate the Service datapath and policy. Compare clients from the intended namespace.

INTERVIEWER FOLLOW-UP

What does an HTTP 401 prove about the network path?

Answer: HTTP 401 proves a response reached an HTTP endpoint; authentication still denied the request.

Watch out: A ping does not verify HTTP readiness, database login or Kafka metadata reachability.

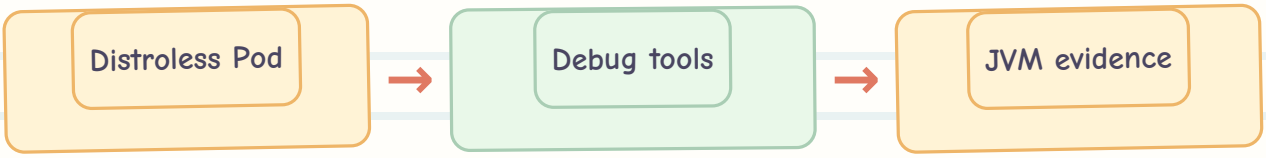
Original guide: page 53 · K34, K35, K39, K40, K46 (source directory).

Q079. How do you debug a minimal image without a shell?

20-SECOND ANSWER • IN SIMPLE WORDS

Use logs, describe, metrics and authorized ephemeral debugging instead of assuming bash exists.

Interview precision: Use logs, status and approved diagnostic tooling such as ephemeral containers, with deliberate permissions and namespace access.



YAML / COMMAND EXAMPLE

```
kubectl logs POD -n interview -c api
kubectl debug POD -n interview -it \
  --image=APPROVED_DEBUG_IMAGE --target=api
# Support, privileges and process visibility depend on the environment.
```

WHAT REALLY HAPPENS · A diagnostic container can supply tools without baking them into the application image. Targeted process visibility and attach permissions require checking. Use the least authority appropriate to the incident.

INTERVIEWER FOLLOW-UP

How would you collect a thread dump while respecting production access policy?

Answer: Use policy-approved JVM diagnostics or a debug image/process with audit and minimal permissions.

Watch out: An ephemeral container is not a normal restartable application sidecar. Do not assume it can read every target filesystem or collect every JVM artifact automatically.

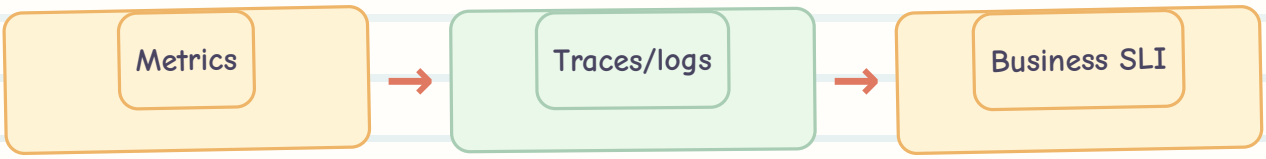
Original guide: page 53 · K34, K35, K39, K40, K46 (source directory).

Q080. What is the difference between monitoring and observability in practice?

20-SECOND ANSWER • IN SIMPLE WORDS

Monitoring checks chosen signals; observability uses logs, metrics and traces to explain failures.

Interview precision: Monitoring tracks known health and performance signals; observability combines evidence to investigate why behavior occurred, including previously unanticipated failures.



YAML / COMMAND EXAMPLE

```
Metrics: request rate, errors, duration, throttling, heap, queueing
Logs:    events with request and instance context
Traces:  request flow and dependency latency
Events:  scheduling, pull, probe and lifecycle transitions
```

WHAT REALLY HAPPENS · Correlate timestamps and release identity across signals. Infrastructure Ready status can coexist with high user error rates. Alerts should lead to a tested investigation or recovery action.

INTERVIEWER FOLLOW-UP

Which user-facing signal would detect a broken order flow that health probes miss?

Answer: Measure completed orders and checkout errors, not just Pod readiness or CPU.

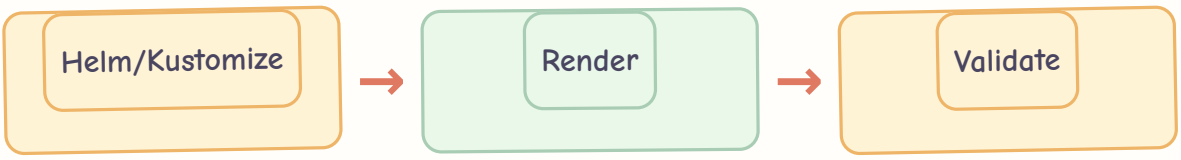
Watch out: kubectl top is not a substitute for historical telemetry. Avoid recording sensitive payloads while adding context.

Original guide: page 54 · K34, K35, K39, K40, K46 (source directory).

CHAPTER 17 / 22

Helm, Kustomize, apply and GitOps

Rendered desired state must be reviewed and deployed.



THE QUESTIONS IN THIS CHAPTER

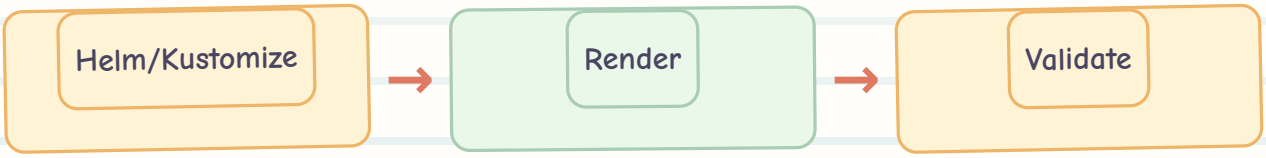
1. How do Helm and Kustomize differ?
2. Why use bases and overlays instead of copying whole manifests?
3. What does server-side apply field ownership mean?
4. What is GitOps reconciliation?
5. Does helm rollback or a Git revert undo every release effect?

Q081. How do Helm and Kustomize differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Helm templates and packages app resources; Kustomize patches/ combines YAML without its own templates.

Interview precision: Helm packages and templates releases with values; Kustomize composes and transforms Kubernetes configuration through bases and overlays without a general template language.



YAML / COMMAND EXAMPLE

```
helm template orders ./chart -f values-prod.yaml
kubectl kustomize overlays/prod
# Review rendered objects before applying them.
```

WHAT REALLY HAPPENS · Both can create environment-specific manifests. Helm also tracks release operations; Kustomize is integrated into kubectl workflows. Choose conventions your team can audit and maintain.

INTERVIEWER FOLLOW-UP

What should CI check after rendering a production overlay?

Answer: Run rendering, schema checks, policy validation, secrets scans and intended-diff checks in CI.

Watch out: Rendering success does not prove API admission, policy compliance or application health.

Original guide: page 55 · H01, K36, K45, A01 (source directory).

Q082. Why use bases and overlays instead of copying whole manifests?

20-SECOND ANSWER • IN SIMPLE WORDS

Shared bases avoid duplicating manifests; overlays add differences for each environment.

Interview precision: A shared base preserves common configuration while focused overlays describe environment-specific differences with less accidental drift.



YAML / COMMAND EXAMPLE

```
# kustomization.yaml sketch
resources: [../../base]
namespace: orders-prod
replicas:
  - {name: orders, count: 3}
```

WHAT REALLY HAPPENS · The overlay can adjust the desired replica count and namespace while inheriting common labels and runtime settings. Inspect the output to ensure patches target the intended resources.

INTERVIEWER FOLLOW-UP

How would you verify that a patch changed only the intended container image?

Answer: Render both old and new manifests and inspect container image, selectors and resource diffs.

Watch out: Overly clever patches can be harder to review than explicit configuration. Keep secret material outside plain committed files.

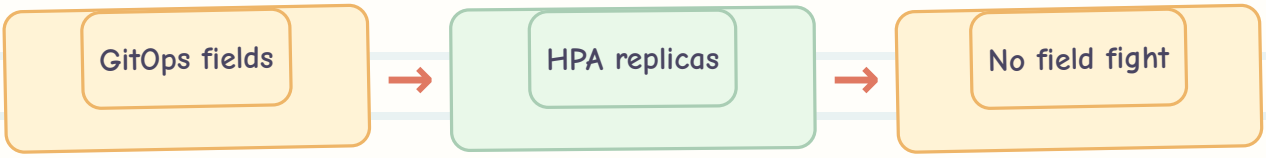
Original guide: page 55 · H01, K36, K45, A01 (source directory).

Q083. What does server-side apply field ownership mean?

20-SECOND ANSWER • IN SIMPLE WORDS

Server-side apply tracks which manager owns fields so controllers can share object updates safely.

Interview precision: The API records managers for managed fields and can report conflicts when an apply tries to take over another manager's field.



YAML / COMMAND EXAMPLE

```
kubectl apply --server-side --field-manager=delivery \
  -f rendered.yaml
kubectl get deployment orders -n interview -o yaml
# Inspect managedFields when ownership conflicts occur.
```

WHAT REALLY HAPPENS · Ownership helps multiple controllers or tools coexist intentionally. Resolve the intended owner rather than blindly forcing every conflict. An autoscaler and a delivery tool can disagree over replica management.

INTERVIEWER FOLLOW-UP

How should a GitOps definition handle replicas managed by HPA?

Answer: Let HPA own replica count; avoid GitOps repeatedly enforcing conflicting spec.replicas.

Watch out: --force-conflicts changes ownership and can override another actor's intent. It is not a universal deployment fix.

Original guide: page 56 · H01, K36, K45, A01 (source directory).

Q084. What is GitOps reconciliation?

20-SECOND ANSWER • IN SIMPLE WORDS

GitOps controllers compare the cluster with reviewed Git configuration and reconcile drift.

Interview precision: A delivery controller compares declared version-controlled desired state with live state and reconciles according to its configured sync policy.



YAML / COMMAND EXAMPLE

```
Git change -> review -> rendered desired objects
Delivery controller -> compare -> authorized synchronization
Live health and drift -> observable status
```

WHAT REALLY HAPPENS · Version history can explain a deployment change and support reproducible rollback decisions. Automatic synchronization, pruning and self-healing are policy choices, not identical defaults across products.

INTERVIEWER FOLLOW-UP

How would you make an emergency change persistent and reviewable?

Answer: Commit the approved emergency change into Git, or it may be reverted by reconciliation.

Watch out: A manual production edit may be reverted by reconciliation. A healthy sync status alone does not prove business behavior.

Original guide: page 56 · H01, K36, K45, A01 (source directory).

Q085. Does helm rollback or a Git revert undo every release effect?

20-SECOND ANSWER • IN SIMPLE WORDS

Helm rollback or Git revert changes manifests; it cannot undo every changed database row or side effect.

Interview precision: No. It changes declared release state under the tool's contract; migrations, external effects, volumes and runtime configuration may need separate recovery.



YAML / COMMAND EXAMPLE

```
Artifact rollback -> reviewed earlier image/template
Schema rollback  -> separate compatibility/recovery decision
External effects -> durable compensation or reconciliation
```

WHAT REALLY HAPPENS · Use backward-compatible migration sequencing where possible. Check hooks, retained revisions, secret versions and data compatibility before treating rollback as safe. Observe restored user behavior after the change.

INTERVIEWER FOLLOW-UP

What would block a rollback after a data transformation?

Answer: Backward-incompatible transformed data may need a forward fix, compatible reader or restored data.

Watch out: Pruning or reverting a storage object can be destructive. A packaging tool is not an application-consistent backup system.

Original guide: page 57 · H01, K36, K45, A01 (source directory).

CHAPTER 18 / 22

Spring Boot and Java workload design

Connect Spring shutdown, probes and resource use.



THE QUESTIONS IN THIS CHAPTER

1. What should a production Spring Boot Deployment define?
2. Why should database pool size account for replica count?
3. How should a Kafka consumer shut down during Pod replacement?
4. Why can a Java image fail only on some cluster nodes?
5. How should business readiness differ from an alive JVM?

Q086. What should a production Spring Boot Deployment define?

20-SECOND ANSWER • IN SIMPLE WORDS

A production Java Deployment needs secure identity, resources, probes, traffic, config and graceful shutdown.

Interview precision: An explicit artifact, runtime identity, resources, probes, configuration, writable paths and termination policy should match the application's verified requirements.



YAML / COMMAND EXAMPLE

```
Image: approved registry digest
HTTP: main listener 8080 with /livez and /readyz
Runtime: non-root, bounded memory/CPU, explicit /tmp
Inputs: database endpoint and protected runtime credentials
```

WHAT REALLY HAPPENS · The manifest is the contract between delivery, platform and application teams. Test the image under the same settings rather than adding them only in production. Record supported Spring and JDK versions.

INTERVIEWER FOLLOW-UP

Which checks would you run before promoting the digest?

Answer: Confirm digest and signatures, manifest/policy checks, staging behavior and business smoke tests.

Watch out: Copying a template without checking the app's endpoint and filesystem behavior can prevent startup.

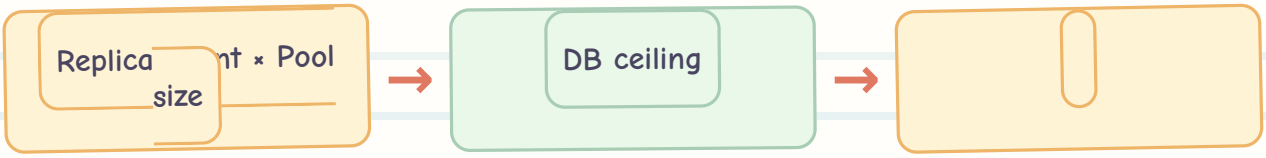
Original guide: page 58 · S01, S02, S03, J01, F01 (source directory).

Q087. Why should database pool size account for replica count?

20-SECOND ANSWER • IN SIMPLE WORDS

Every new API Pod opens database connections, so total pool size must fit database capacity.

Interview precision: Each replica can open its own connections; scaling and rollout overlap multiply the possible total load on the database.



YAML / COMMAND EXAMPLE

```
Steady replicas: 6
Pool maximum per replica: 20
Surge allowance: 1 replica
Possible API pool connections during rollout: 7 x 20 = 140
```

WHAT REALLY HAPPENS · Add jobs, administration and other services before comparing with the database's budget. A small per-instance pool can collectively exceed the safe capacity. Tie HPA bounds to downstream limits.

INTERVIEWER FOLLOW-UP

How would you choose a safe replica maximum for the order service?

Answer: Compute max replicas x per-Pod maximum connections, include surge and other apps, then leave headroom.

Watch out: Virtual threads do not make database connections or CPU unlimited. Idle connections and active query capacity are different concerns.

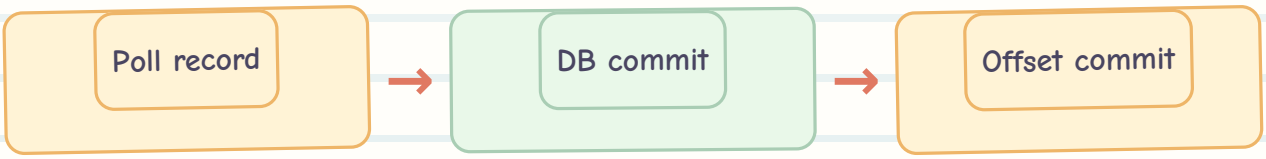
Original guide: page 58 · S01, S02, S03, J01, F01 (source directory).

Q088. How should a Kafka consumer shut down during Pod replacement?

20-SECOND ANSWER • IN SIMPLE WORDS

Stop new consumption, finish or cancel safely, and close Kafka clients during graceful shutdown.

Interview precision: Stop admitting new work, finish or safely abandon bounded processing, coordinate acknowledgment and commit behavior, then close the consumer within the grace budget.



YAML / COMMAND EXAMPLE

```
Pod terminating -> consumer drain policy
Successful durable processing -> corresponding offset policy
Interrupted processing -> safe redelivery and deduplication
```

WHAT REALLY HAPPENS · Kafka group rebalancing and a forced termination can lead to redelivery. Application transactions and external side effects must be designed for that possibility. Measure worst-case processing and use idempotent business operations.

INTERVIEWER FOLLOW-UP

What happens if the database commits but the offset is not committed?

Answer: The message may be processed again; dedupe the effect using a stable event ID or database transaction.

Watch out: Kubernetes does not make external payments exactly once. More consumer replicas are useful only within partition and downstream constraints.

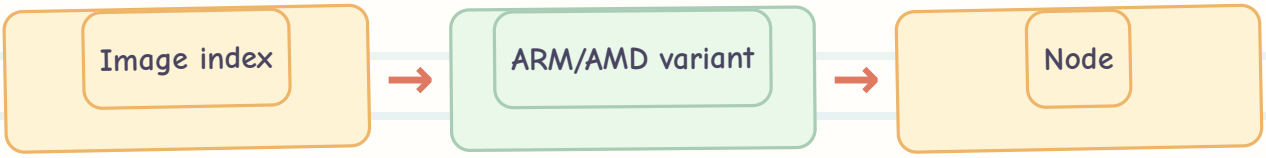
Original guide: page 59 · S01, S02, S03, J01, F01 (source directory).

Q089. Why can a Java image fail only on some cluster nodes?

20-SECOND ANSWER • IN SIMPLE WORDS

The chosen CPU architecture, kernel and native dependencies may vary by node.

Interview precision: Image platform selection, native dependencies, permissions, node features and runtime configuration can differ across eligible nodes.



YAML / COMMAND EXAMPLE

```
kubectl get nodes -L kubernetes.io/arch
kubectl get pod POD -n interview -o wide
kubectl describe pod POD -n interview
# Verify image index variants and any JNI/native-agent requirements.
```

WHAT REALLY HAPPENS · Portable bytecode does not make an AMD64 native library work on ARM64. Select appropriate image variants or constrain placement when a supported runtime requirement demands it. Compare the actual admitted inputs.

INTERVIEWER FOLLOW-UP

How would you test the ARM variant before allowing ARM node placement?

Answer: Build/test the ARM64 image on an ARM-capable runner, verify digest and JNI dependencies, then permit ARM nodes.

Watch out: Architecture labels are not a substitute for testing each published variant. Kernel and storage behavior can also matter.

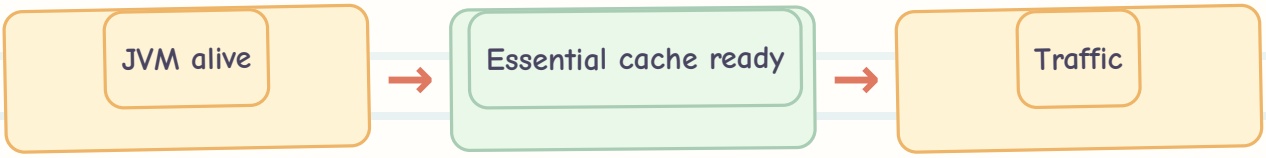
Original guide: page 59 · S01, S02, S03, J01, F01 (source directory).

Q090. How should business readiness differ from an alive JVM?

20-SECOND ANSWER • IN SIMPLE WORDS

Being alive means the JVM responds; being ready means this instance can serve its assigned work.

Interview precision: Readiness must match the traffic the instance can safely accept, while liveness should target local failures where restart is an appropriate repair.



YAML / COMMAND EXAMPLE

```
Alive JVM -> process and local progress
Ready API -> required initialization and traffic contract
Correct order flow -> transaction, authorization and event behavior
```

WHAT REALLY HAPPENS · Warm-up, schema compatibility or intentional drain can make an alive instance unready. A synthetic business check can verify more than health groups, but it needs safe test data and side-effect policy.

INTERVIEWER FOLLOW-UP

Which readiness condition would you use after loading a required reference-data cache?

Answer: Mark ready only when essential reference data has loaded and required local startup steps finished.

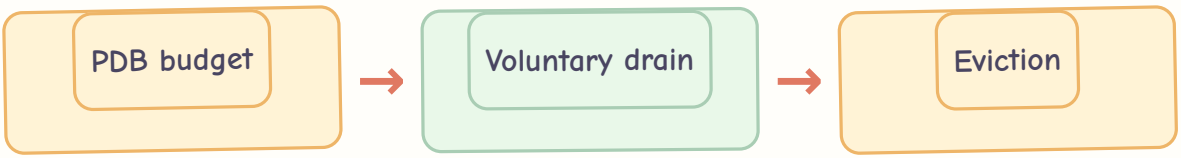
Watch out: Putting a slow full transaction in a frequent liveness probe can create load and restart storms.

Original guide: page 60 · S01, S02, S03, J01, F01 (source directory).

CHAPTER 19 / 22

Availability, disruption and rollout strategy

Protect service availability during changes.



THE QUESTIONS IN THIS CHAPTER

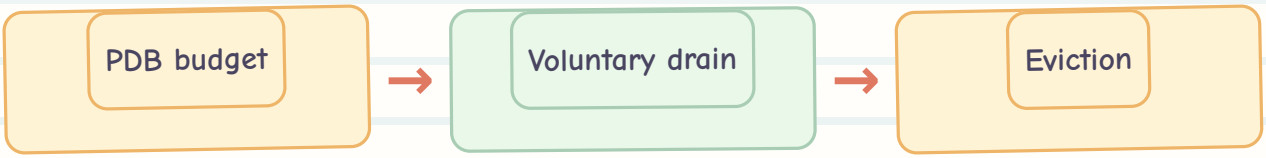
1. What does a PodDisruptionBudget protect?
2. Why are three replicas on one node not sufficient high availability?
3. How do rolling, blue-green and canary deployment differ?
4. How do you design graceful HTTP rollout?
5. How should schema migrations be handled during overlapping versions?

Q091. What does a PodDisruptionBudget protect?

20-SECOND ANSWER • IN SIMPLE WORDS

PDB limits eligible voluntary evictions that would reduce available replicas below its rule.

Interview precision: A PDB limits eligible voluntary disruptions through the eviction mechanism based on workload availability; it does not prevent all failures or direct deletions.



YAML / COMMAND EXAMPLE

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: orders}
spec:
  minAvailable: 2
  selector: {matchLabels: {app: orders}}
```

WHAT REALLY HAPPENS · With three healthy matching Pods, one voluntary eviction may be allowed. With only two healthy Pods, maintenance can be blocked. Rollout strategy has its own availability controls.

INTERVIEWER FOLLOW-UP

Why can a safe drain be blocked while an application rollout still replaces Pods?

Answer: Draining uses the eviction API and honors the PDB; rollout replacement can follow separate controller rules.

Watch out: PDB does not stop a node crash, every direct Pod delete or Deployment-controlled rolling-update deletion.

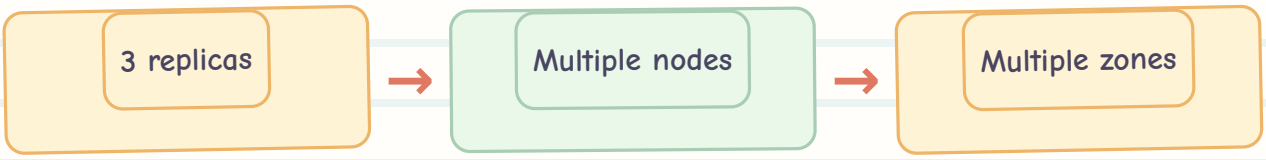
Original guide: page 61 · K03, K23, K28, K06, S02 (source directory).

Q092. Why are three replicas on one node not sufficient high availability?

20-SECOND ANSWER • IN SIMPLE WORDS

Three copies on one node all fail together if that node fails.

Interview precision: They share the same node failure domain and can fail together; topology and independent dependency resilience determine the actual protection.



YAML / COMMAND EXAMPLE

```
Replicas -> spread across eligible nodes
Availability requirement -> consider zone placement
Shared database -> independent failure and recovery design
```

WHAT REALLY HAPPENS · Use supported spread or anti-affinity policy with sufficient capacity. Node diversity is only one layer: shared storage, network and database failures can still affect every API replica.

INTERVIEWER FOLLOW-UP

What would happen if the largest node or one zone disappeared?

Answer: Spread across nodes/zones and test losing one node or zone while preserving minimum capacity.

Watch out: Strict placement without enough eligible domains can reduce running capacity. Design and test the capacity tradeoff.

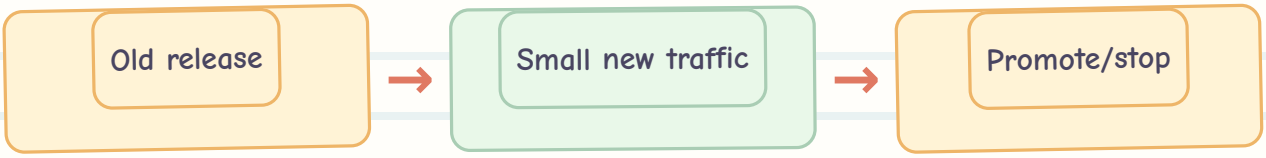
Original guide: page 62 · K03, K23, K28, K06, S02 (source directory).

Q093. How do rolling, blue-green and canary deployment differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Rolling swaps Pods gradually, blue-green switches environments, and canary shifts small traffic fractions.

Interview precision: They replace instances incrementally, switch traffic between deployment groups, or expose a limited candidate share before promotion.



YAML / COMMAND EXAMPLE

```
Rolling:    old/new versions overlap during replacement
Blue-green: switch traffic between prepared groups
Canary:     measure limited candidate traffic before expansion
```

WHAT REALLY HAPPENS · Controllers and routing integrations implement the chosen strategy. Standard Deployment rolling updates do not alone provide every precise traffic-weight policy. Database compatibility must cover coexistence and rollback.

INTERVIEWER FOLLOW-UP

Which metric would stop promotion despite healthy Pods?

Answer: Stop promotion if customer error rate, latency or business success worsens, even if Pods are Ready.

Watch out: A Service selector switch may not instantly end all old connections. Canary evidence needs representative traffic and user-impact signals.

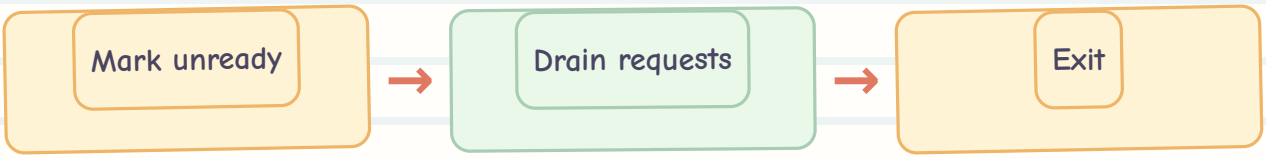
Original guide: page 62 · K03, K23, K28, K06, S02 (source directory).

Q094. How do you design graceful HTTP rollout?

20-SECOND ANSWER • IN SIMPLE WORDS

Withdraw traffic, allow in-flight work to finish, and stop safely before the grace deadline.

Interview precision: Coordinate traffic withdrawal, readiness and endpoint propagation with signal handling, in-flight work and an adequate total termination budget.



YAML / COMMAND EXAMPLE

```
New replica ready -> traffic admitted
Old replica terminating -> stop new admission and drain
Grace expiry -> possible forced termination
# Observe request errors and completion during the transition.
```

WHAT REALLY HAPPENS · Direct Java startup helps signals reach the JVM. The load balancer and clients can still retain connections or stale backend state briefly. Test a realistic long request and dependency interaction during replacement.

INTERVIEWER FOLLOW-UP

How would you test drain without generating real customer side effects?

Answer: Send synthetic requests in staging while terminating Pods; check dropped connections and duplicate effects.

Watch out: A healthy rollout status is not proof of no dropped requests. preStop and Spring shutdown work share the total time available.

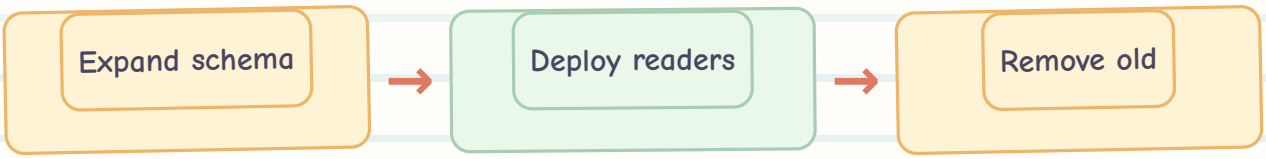
Original guide: page 63 · K03, K23, K28, K06, S02 (source directory).

Q095. How should schema migrations be handled during overlapping versions?

20-SECOND ANSWER • IN SIMPLE WORDS

Keep old and new app versions compatible with the database while a rolling deployment overlaps them.

Interview precision: Use a compatible sequence, controlled migration ownership and verified completion rather than assuming new Pods can immediately require a destructive schema change.



YAML / COMMAND EXAMPLE

```
Expand compatible schema -> deploy interoperable versions
Backfill and verify -> close rollback window -> contract old schema
# Run controlled migration work with observable completion.
```

WHAT REALLY HAPPENS · One job or approved migration controller can own the change while APIs check the required compatible state. Retries must be safe. Separate code rollback from data recovery.

INTERVIEWER FOLLOW-UP

What is your recovery plan if migration succeeds but the new API fails?

Answer: Use expand-migrate-contract, feature flags and a forward recovery plan when rollback is impossible.

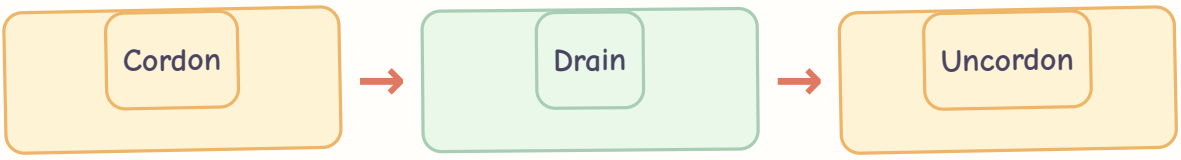
Watch out: Running a destructive migration from every replica's init step can create contention or irreversible effects.

Original guide: page 63 · K03, K23, K28, K06, S02 (source directory).

CHAPTER 20 / 22

Cluster operations and recovery boundaries

Plan node operations and tested data recovery.



THE QUESTIONS IN THIS CHAPTER

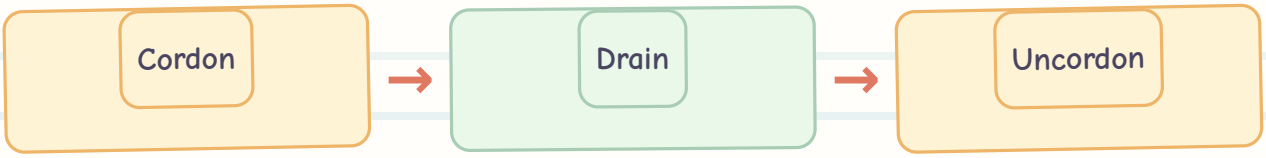
1. What is the difference between cordon, drain and uncordon?
2. What happens when a node becomes unavailable?
3. Why is disk or ephemeral-storage pressure different from application heap pressure?
4. How should Kubernetes upgrades be planned?
5. What should a disaster-recovery plan include?

Q096. What is the difference between cordon, drain and uncordon?

20-SECOND ANSWER • IN SIMPLE WORDS

Cordon blocks new scheduling, drain evicts supported workloads, and uncordon reopens scheduling.

Interview precision: Cordon marks a node unschedulable for normal placement; drain attempts supported workload eviction; uncordon restores normal scheduling eligibility.



YAML / COMMAND EXAMPLE

```
kubectl cordon NODE
# Review workload and data impact before an authorized drain.
kubectl drain NODE --ignore-daemonsets
kubectl uncordon NODE
```

WHAT REALLY HAPPENS · Cordon alone does not move existing Pods. Drain respects applicable eviction constraints and has special handling for DaemonSets, local data and unmanaged workloads. Review failed preconditions rather than adding force blindly.

INTERVIEWER FOLLOW-UP

How could a PDB or node-local volume block maintenance?

Answer: PDB minAvailable or node-local storage may prevent eviction; inspect and plan maintenance accordingly.

Watch out: These are operator commands, not exercises against an unknown cluster. Do not add delete-emptydir-data without understanding disposable-data loss.

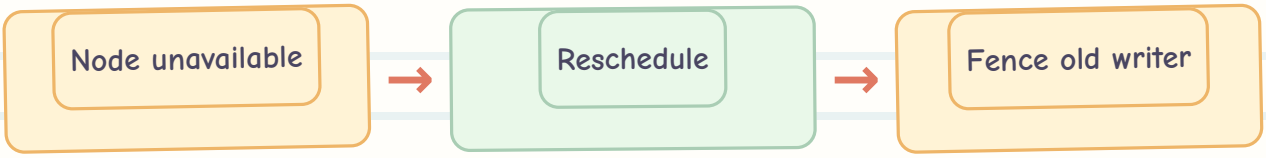
Original guide: page 64 · K41, K43, K44, K32, K08 (source directory).

Q097. What happens when a node becomes unavailable?

20-SECOND ANSWER • IN SIMPLE WORDS

A lost node may trigger Pod replacement elsewhere after control-plane detection and timeouts.

Interview precision: Node conditions, controller detection and configured eviction/replacement behavior guide recovery, while storage and capacity determine where replacements can run.



YAML / COMMAND EXAMPLE

```
kubectl get nodes
kubectl describe node NODE
kubectl get pods -A -o wide
# Correlate node health with network, runtime and storage signals.
```

WHAT REALLY HAPPENS · Recovery is not instantaneous. A process on an isolated node might still execute while the control plane sees failure, so stateful software needs fencing or equivalent safeguards. Replacements need eligible resources and storage.

INTERVIEWER FOLLOW-UP

What prevents two apparent database leaders from writing during a partition?

Answer: Use fencing/leases with storage or database consensus; Kubernetes Pod restarts do not prevent split-brain writes.

Watch out: Restarting an API elsewhere does not restore a failed database or local file.

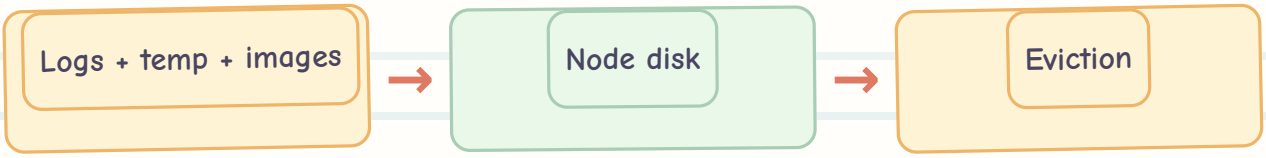
Original guide: page 64 · K41, K43, K44, K32, K08 (source directory).

Q098. Why is disk or ephemeral-storage pressure different from application heap pressure?

20-SECOND ANSWER • IN SIMPLE WORDS

Disk pressure is node-storage exhaustion, not necessarily JVM heap failure.

Interview precision: Node storage pressure concerns local logs, writable layers and temporary data; it can cause eviction even without Java heap exhaustion.



YAML / COMMAND EXAMPLE

```
kubectl describe node NODE
kubectl describe pod POD -n interview
# Inspect node disk, container logs and ephemeral-storage policy.
```

WHAT REALLY HAPPENS · Unbounded logs or temp files can affect neighboring workloads. Volume-backed data has a separate backend capacity contract. Apply rotation and bounded scratch storage, then verify retention and incident evidence.

INTERVIEWER FOLLOW-UP

Which application write paths can consume node-local storage?

Answer: Container logs, emptyDir, writable layers, temp uploads, heap dumps and image layers can fill node storage.

Watch out: More heap is not a repair for a full node disk. An Evicted Pod and OOMKilled container describe different paths.

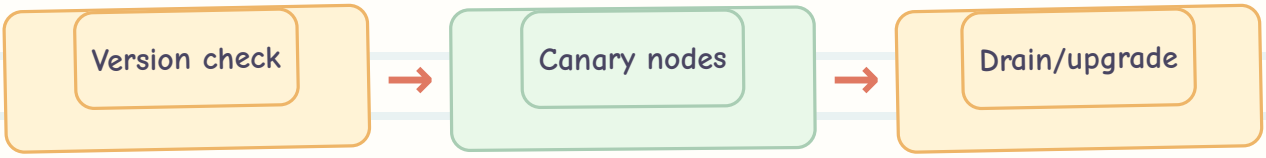
Original guide: page 65 · K41, K43, K44, K32, K08 (source directory).

Q099. How should Kubernetes upgrades be planned?

20-SECOND ANSWER • IN SIMPLE WORDS

Upgrade control plane and nodes using compatibility checks, staged node pools and safe workload drains.

Interview precision: Follow the distribution's supported sequence and version-skew policy, check API compatibility and add-on support, then stage and verify recovery.



YAML / COMMAND EXAMPLE

```
Inventory: control plane, nodes, CNI, CSI, controllers and CRDs
Review: supported skew and removed/deprecated APIs
Rehearse: backup/restore plus workload and traffic checks
Upgrade: approved staged sequence with observation
```

WHAT REALLY HAPPENS · Version compatibility covers more than kubectl and the API server. Webhooks, operators, storage drivers and network controllers can affect admission or workload operation. Use the official policy for the exact versions.

INTERVIEWER FOLLOW-UP

Which application tests should gate a node-pool upgrade?

Answer: Gate on API compatibility, CNI/CSI checks, Java smoke tests, disruption budget and user SLIs.

Watch out: Do not invent a universal safe upgrade order for every managed or self-hosted product. A downgrade may not be a supported rollback.

Original guide: page 65 · K41, K43, K44, K32, K08 (source directory).

Q100. What should a disaster-recovery plan include?

20-SECOND ANSWER • IN SIMPLE WORDS

Disaster recovery must restore cluster configuration and business data within tested goals.

Interview precision: Define acceptable downtime and data loss, then test infrastructure, configuration, credentials, persistent data and application recovery together.



YAML / COMMAND EXAMPLE

```
Control plane: supported state recovery
Application:   reproducible images and manifests
Data:          consistent backups/replication and restore
Traffic:       routing, identity and observed business checks
```

WHAT REALLY HAPPENS · A restored Deployment can still fail because its secret, storage class or database is missing. Recovery evidence should include elapsed time and data correctness. Keep backups in an appropriate independent failure domain.

INTERVIEWER FOLLOW-UP

How would you prove that a recovered order service preserves committed orders?

Answer: Restore a backup into isolation, replay required logs and compare committed order IDs and totals.

Watch out: Replica count and retained PVCs are not substitutes for a recovery rehearsal.

Original guide: page 66 · K41, K43, K44, K32, K08 (source directory).

CHAPTER 21 / 22

Minikube, image loading and Windows/WSL access

Check context, image store and laptop routing.



THE QUESTIONS IN THIS CHAPTER

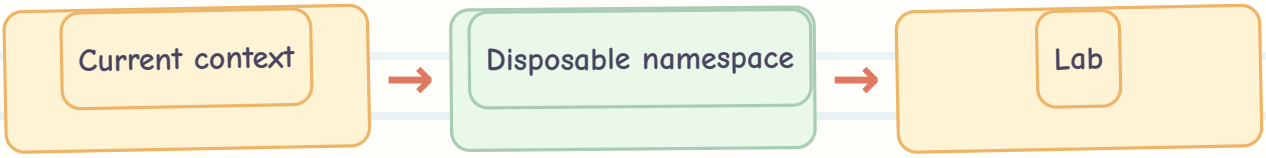
1. How should you establish a local practice cluster?
2. Why is a locally built Docker image unavailable to a Pod?
3. How do Always, IfNotPresent and Never image pull policies differ?
4. Why does NodePort access sometimes fail on Windows/WSL or Docker Desktop?
5. How do you debug kubectl connection refused on a local cluster?

Q101. How should you establish a local practice cluster?

20-SECOND ANSWER • IN SIMPLE WORDS

Choose a disposable cluster, confirm contexts, and keep lab resources isolated.

Interview precision: Use a supported local runtime, an isolated minikube profile and explicit context checks before applying workloads.



YAML / COMMAND EXAMPLE

```
minikube start -p interview-lab --driver=docker
kubectl config current-context
kubectl get nodes
minikube status -p interview-lab
```

WHAT REALLY HAPPENS · The command requires a working supported Docker environment and can select a context. Confirm that commands target the practice cluster before creating or deleting resources. Driver and host resources affect capabilities.

INTERVIEWER FOLLOW-UP

How would you confirm where a kubectl command will run?

Answer: Run kubectl config current-context and kubectl cluster-info before any create/delete action.

Watch out: These setup commands were not executed here. Do not reset an existing useful profile to repair an unrelated context issue.

Original guide: page 67 · M01, M02, M03, K39, K50 (source directory).

Q102. Why is a locally built Docker image unavailable to a Pod?

20-SECOND ANSWER • IN SIMPLE WORDS

A Pod uses the image store on its Kubernetes node, not necessarily the laptop's Docker images.

Interview precision: The cluster's node runtime can use a different image store from the developer's Docker client or daemon.



YAML / COMMAND EXAMPLE

```
docker build -t orders-api:local .
minikube image load orders-api:local -p interview-lab
minikube image ls -p interview-lab
# Alternative: minikube image build -t orders-api:local . -p interview-lab
```

WHAT REALLY HAPPENS · Load or build into the intended profile and verify node-side availability. Then use the exact tag and a compatible pull policy. With multiple nodes, confirm the required image distribution.

INTERVIEWER FOLLOW-UP

What would ImagePullBackOff tell you after loading the image?

Answer: If pull still fails, inspect tag/policy, node image cache, registry access and Pod events.

Watch out: A successful host docker build does not prove Kubernetes can fetch or find the image. Native architecture still matters.

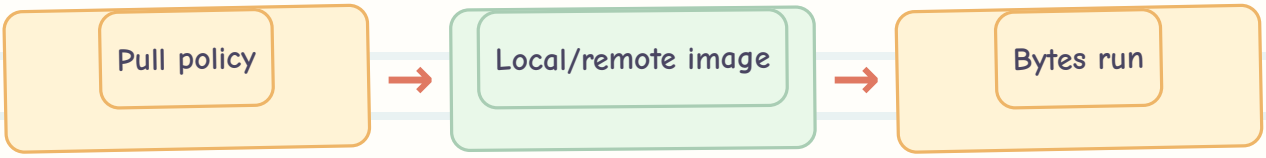
Original guide: page 67 · M01, M02, M03, K39, K50 (source directory).

Q103. How do Always, IfNotPresent and Never image pull policies differ?

20-SECOND ANSWER • IN SIMPLE WORDS

Always requests the image each start; IfNotPresent pulls only if missing; Never uses local only.

Interview precision: They control registry resolution or local-image reliance according to the runtime contract; tags and defaults must be considered deliberately.



YAML / COMMAND EXAMPLE

```
image: orders-api:local
imagePullPolicy: IfNotPresent
# Use an explicitly loaded image for this local exercise.
# Never requires the needed image to be present on the target node.
```

WHAT REALLY HAPPENS · Always can still reuse cached content after resolving the required image identity. IfNotPresent can reuse a node's local tag, so mutable tag updates can produce confusing results. Prefer new tags or digests for repeatable delivery.

INTERVIEWER FOLLOW-UP

Why can two nodes run different bytes behind one reused local tag?

Answer: Different nodes may keep different bytes under one mutable tag; use immutable tags or digests.

Watch out: Pull-policy defaulting occurs when the object is created; changing the image tag does not automatically imply a new default policy.

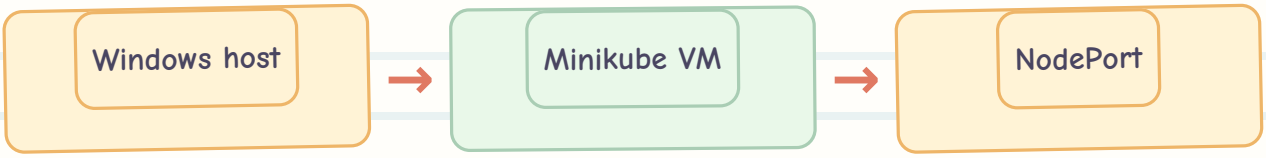
Original guide: page 68 · M01, M02, M03, K39, K50 (source directory).

Q104. Why does NodePort access sometimes fail on Windows/WSL or Docker Desktop?

20-SECOND ANSWER • IN SIMPLE WORDS

Desktop/WSL node addresses or routing may be hidden behind a VM/ NAT layer.

Interview precision: The local driver's node network may not be directly reachable from the chosen host, so a supported proxy or tunnel route may be required.



YAML / COMMAND EXAMPLE

```
minikube service orders --url -p interview-lab -n interview
# Or a stable local development endpoint:
kubectl port-forward -n interview svc/orders 18080:80
# Keep the port-forward process running; test localhost:18080.
```

WHAT REALLY HAPPENS · minikube may print a temporary proxy URL depending on the driver and platform. Service port-forward uses a local listener while selecting a backend Pod through the Kubernetes path; it is a development diagnostic route.

INTERVIEWER FOLLOW-UP

How would you compare host access with a same-namespace Pod client?

Answer: Test through host port-forward and through a same-namespace Pod; differences localize the broken route.

Watch out: The generated proxy URL can change and may require its process to stay alive. Port-forward success does not test the normal Service datapath or Ingress.

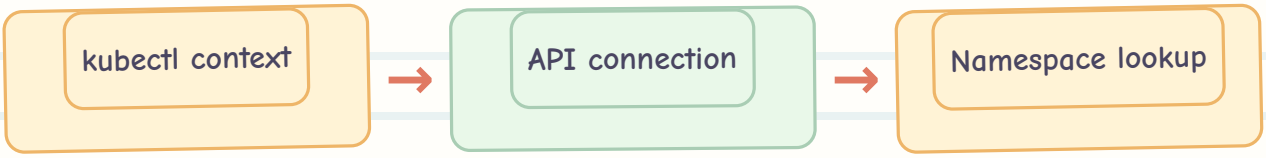
Original guide: page 68 · M01, M02, M03, K39, K50 (source directory).

Q105. How do you debug kubectl connection refused on a local cluster?

20-SECOND ANSWER • IN SIMPLE WORDS

Check the selected context and API endpoint before diagnosing workload resources.

Interview precision: Check profile status, current context, configured API endpoint and local runtime availability before modifying application manifests.



YAML / COMMAND EXAMPLE

```
minikube status -p interview-lab
kubectl config current-context
kubectl config view --minify
kubectl cluster-info
# Redact sensitive data when sharing configuration output.
```

WHAT REALLY HAPPENS · A stale endpoint or stopped local runtime can make every API request fail. Repair the cluster access path first. Missing image or Service errors are later application-layer concerns.

INTERVIEWER FOLLOW-UP

How do you distinguish API connectivity failure from a missing namespace?

Answer: Connection refused means API transport fails; a missing namespace is an API response from a working server.

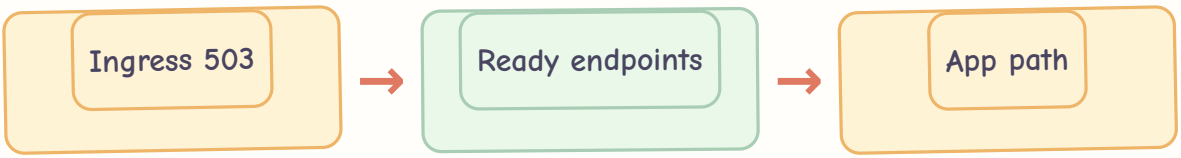
Watch out: Deleting the cluster can destroy local workloads and data; it is not the first diagnostic step. Avoid sharing raw credential-bearing kubeconfig.

Original guide: page 69 · M01, M02, M03, K39, K50 (source directory).

CHAPTER 22 / 22

Senior interview and architecture scenarios

Explain the normal request and the failure path.



THE QUESTIONS IN THIS CHAPTER

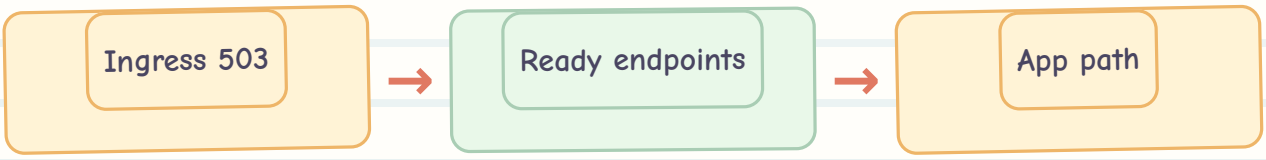
1. A Spring API is Running but Ingress returns 503. How do you investigate?
2. A rollout stalls when readiness is enabled. What is your approach?
3. HPA adds API replicas, but latency gets worse. What could cause it?
4. A database Pod cannot move after node failure. Why?
5. How would you design Kubernetes delivery for an order service?

Q106. A Spring API is Running but Ingress returns 503. How do you investigate?

20-SECOND ANSWER • IN SIMPLE WORDS

A Running Pod can be unready or have a wrong Service/Ingress route.

Interview precision: Locate the response source and trace route ownership, Service selector, ready EndpointSlices, target port, listener and policy in that order.



YAML / COMMAND EXAMPLE

```
kubectl get ingress,svc,pods -n orders
kubectl get endpointslices -n orders \
  -l kubernetes.io/service-name=orders
kubectl logs API_POD -n orders --tail=100
```

WHAT REALLY HAPPENS · Running is not Ready. An empty or unready backend set explains many failures before proxy tuning. If a direct backend test works, compare the controller route and logs. A backend-generated 503 needs application diagnosis.

INTERVIEWER FOLLOW-UP

What observation would shift attention from readiness to path rewriting?

Answer: If direct Service access succeeds and returns 404 only on ingress, inspect path rewrite and Host rules.

Watch out: Preserve the exact request host/path and release identity. Deleting all Pods can worsen availability without repairing selection.

Original guide: page 70 · K34, K35, K19, K08, S01, S02 (source directory).

Q107. A rollout stalls when readiness is enabled. What is your approach?

20-SECOND ANSWER • IN SIMPLE WORDS

Check exact probe path, port, startup time, dependencies and events without removing protection.

Interview precision: Check probe response, startup timing, endpoint access and dependencies, then inspect capacity and the Deployment's progress conditions.



YAML / COMMAND EXAMPLE

```
kubectl describe deployment orders -n orders
kubectl describe pod NEW_POD -n orders
kubectl logs NEW_POD -n orders
# Verify /readyz on the configured port and security policy.
```

WHAT REALLY HAPPENS · A 401, nonexistent endpoint and slow initialization need different fixes. With maxUnavailable=0, an unready new Pod can deliberately preserve old capacity while blocking progress. An unschedulable surge Pod needs placement diagnosis.

INTERVIEWER FOLLOW-UP

When would a startup probe solve the issue without weakening readiness?

Answer: Startup probe helps when healthy apps need extra initialization time before readiness/liveness checks.

Watch out: Disabling readiness can expose users to a broken version. Fix the contract rather than hiding the gate.

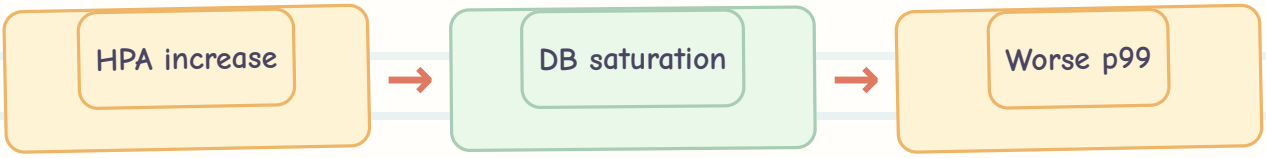
Original guide: page 70 · K34, K35, K19, K08, S01, S02 (source directory).

Q108. HPA adds API replicas, but latency gets worse. What could cause it?

20-SECOND ANSWER • IN SIMPLE WORDS

More API Pods can overload a shared database or connection pool, making latency worse.

Interview precision: Scaling can increase downstream contention, connection demand, rebalances, warm-up work or scheduling pressure when the shared bottleneck does not scale.



YAML / COMMAND EXAMPLE

```
Compare: replicas, pool totals, DB latency, queue depth
Inspect: CPU throttling, request rate, GC and readiness
Check:   scale timing and deployment overlap
```

WHAT REALLY HAPPENS · Use a timeline to connect scale decisions with symptoms. A saturated database may need bounded API concurrency and query repair rather than more consumers. Verify metric meaning and prevent oscillation.

INTERVIEWER FOLLOW-UP

What safe mitigation would you choose while investigating the shared bottleneck?

Answer: Cap replicas/pools and apply admission or backpressure while measuring and fixing the shared limit.

Watch out: A correlation alone is not proof; compare workload changes and controlled test evidence.

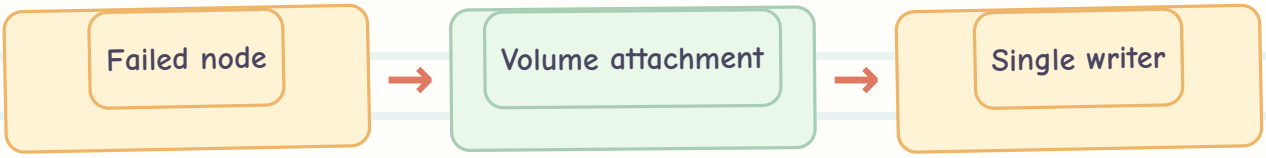
Original guide: page 71 · K34, K35, K19, K08, S01, S02 (source directory).

Q109. A database Pod cannot move after node failure. Why?

20-SECOND ANSWER • IN SIMPLE WORDS

Persistent storage may be pinned to a failed node or zone, or require detach before attachment.

Interview precision: Volume topology, attachment state, access constraints, node-local storage, fencing or lack of eligible capacity can block safe relocation.



YAML / COMMAND EXAMPLE

```
kubectl describe pod DB_POD -n data
kubectl describe pvc DB_CLAIM -n data
kubectl get pv VOLUME -o yaml
kubectl get nodes
```

WHAT REALLY HAPPENS · A stable identity does not guarantee storage is portable. Inspect supported failover and attachment recovery procedures. Ensure the old writer is safely excluded before forcing another writer to use the data.

INTERVIEWER FOLLOW-UP

How does your recovery design handle a partitioned node that is still writing?

Answer: Fence stale writers and ensure the storage/db recovery protocol allows one active owner.

Watch out: Do not delete the PVC or force-detach storage as an unexplained first response. Provider operations can risk data integrity.

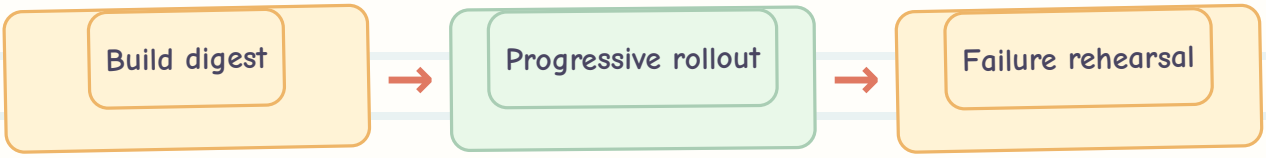
Original guide: page 71 · K34, K35, K19, K08, S01, S02 (source directory).

Q110. How would you design Kubernetes delivery for an order service?

20-SECOND ANSWER • IN SIMPLE WORDS

Ship one reviewed image digest, set secure resource and health contracts, then test rollout and restore.

Interview precision: Define artifact identity, secure runtime inputs, resource and traffic contracts, compatible state changes and tested recovery before choosing individual flags.



YAML / COMMAND EXAMPLE

```
Artifact: approved multi-platform digest and release evidence
Runtime:  non-root, bounded JVM budget, writable temp only
Traffic:  Service, route policy, readiness and graceful drain
State:    database durability, event outbox/idempotency, restore test
```

WHAT REALLY HAPPENS · Select replicas and topology from availability requirements. Bound DB and consumer concurrency, make migration ownership explicit and observe business outcomes during rollout. A platform manifest is one part of the system design.

INTERVIEWER FOLLOW-UP

What failure rehearsal would demonstrate the design to an architect interviewer?

Answer: Kill one API Pod during an in-flight order, repeat its request and prove no duplicate order or payment.

Watch out: Kubernetes does not guarantee exactly-once external effects or recovery from lost business data.

Original guide: page 72 · K34, K35, K19, K08, S01, S02 (source directory).

WORKED ARCHITECTURE

Spring Boot Order API

Seven Kubernetes objects. One complete deployment contract.



Deployment boundaries

Provides: Namespace, ServiceAccount, ConfigMap, Secret placeholder, Deployment, Service, PodDisruptionBudget.

Does not provide: the Spring Boot application JAR, database, production secret delivery, ingress controller or durable database backups.

Replace image digest and Secret values through an approved process. These are teaching manifests, not deployable production credentials.

WORKED EXAMPLE · OBJECT 1/7

Namespace · k8s-orders-example

One part of the existing Spring Boot order API manifest.

```
apiVersion: v1
kind: Namespace
metadata:
  name: k8s-orders-example
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

WORKED EXAMPLE · OBJECT 2/7

ServiceAccount · orders-api

One part of the existing Spring Boot order API manifest.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders-api
  namespace: k8s-orders-example
automountServiceAccountToken: false
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

WORKED EXAMPLE · OBJECT 3/7

ConfigMap · orders-config

One part of the existing Spring Boot order API manifest.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: orders-config
  namespace: k8s-orders-example
data:
  application.properties: |
    server.port=8080
    server.shutdown=graceful
    spring.lifecycle.timeout-per-shutdown-phase=20s
    management.endpoint.health.probes.enabled=true
    management.endpoint.health.probes.add-additional-paths=true
    spring.config.import=configtree:/run/secrets/
    spring.datasource.url=jdbc:postgresql://db.data.svc.cluster.local:5432/orders
    spring.datasource.username=orders_app
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

WORKED EXAMPLE · OBJECT 4/7

Secret · orders-db-auth

One part of the existing Spring Boot order API manifest.

```
# Teaching placeholder. Provision the real Secret through protected delivery.
apiVersion: v1
kind: Secret
metadata:
  name: orders-db-auth
  namespace: k8s-orders-example
type: Opaque
stringData:
  spring.datasource.password: REPLACE_VIA_PROTECTED_SECRET_DELIVERY
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

WORKED EXAMPLE · OBJECT 5/7

Deployment · orders

One part of the existing Spring Boot order API manifest.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders
  namespace: k8s-orders-example
spec:
  replicas: 3
  selector:
    matchLabels: {app: orders}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: orders}
    spec:
      serviceAccountName: orders-api
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 45
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        runAsGroup: 10001
        fsGroup: 10001
        seccompProfile: {type: RuntimeDefault}
      containers:
        - name: api
          image: REGISTRY/TEAM/ORDERS@sha256:DIGEST_HEX
          imagePullPolicy: IfNotPresent
          ports: [{name: http, containerPort: 8080}]
          env:
            - name: SPRING_CONFIG_ADDITIONAL_LOCATION
              value: file:/etc/app/
            - name: JAVA_TOOL_OPTIONS
              value: "-Xms128m -Xmx384m"
          resources:
            requests: {cpu: "250m", memory: "512Mi"}
            limits: {cpu: "1", memory: "768Mi"}
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: [ALL]}
          startupProbe:
            httpGet: {path: /livez, port: http}
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 24
          readinessProbe:
            httpGet: {path: /readyz, port: http}
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 2
          livenessProbe:
            httpGet: {path: /livez, port: http}
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          volumeMounts:
            - {name: scratch, mountPath: /tmp}
            - {name: config, mountPath: /etc/app, readOnly: true}
            - {name: db-secret, mountPath: /run/secrets, readOnly: true}
      volumes:
        - name: scratch
          emptyDir: {sizeLimit: 64Mi}
        - name: config
          configMap: {name: orders-config}
        - name: db-secret
          secret:
```

```
            secretName: orders-db-auth
            defaultMode: 0440
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

WORKED EXAMPLE · OBJECT 6/7

Service · orders

One part of the existing Spring Boot order API manifest.

```
apiVersion: v1
kind: Service
metadata:
  name: orders
  namespace: k8s-orders-example
spec:
  selector: {app: orders}
  ports: [{name: http, port: 80, targetPort: http}]
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

WORKED EXAMPLE · OBJECT 7/7

PodDisruptionBudget · orders

One part of the existing Spring Boot order API manifest.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: orders
  namespace: k8s-orders-example
spec:
  minAvailable: 2
  selector:
    matchLabels: {app: orders}
```

> This YAML is a teaching fixture. Review Pod security, application compatibility, credentials, resource sizing and cluster admission before real deployment.

HAND-ON PRACTICE

12 Kubernetes labs

Safe scope starts with the correct context.



Run one script at a time

Create a fresh empty working directory. Keep the script outside that folder. Read the Watch for notes first. The scripts target and delete generated namespaces.

Verification: Bash syntax checked, not cluster-executed. Storage, RBAC, NetworkPolicy and networking behavior depend on your practice environment.

HANDS-ON LAB 01 / 12

L01 · Declare and scale a Deployment

Goal: The controller maintains Pods for its updated replica goal. Scaling changes the replica count without requiring a different image template.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l01-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > web.yaml <<'YAML'
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports: [{name: http, containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: http}
            periodSeconds: 2
YAML
kubectl apply -n "$K_NS" -f web.yaml
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
kubectl scale deployment/web -n "$K_NS" --replicas=3
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
test "$(kubectl get deploy web -n "$K_NS" -o jsonpath='{.spec.replicas}')" = 3
test "$(kubectl get deploy web -n "$K_NS" -o jsonpath='{.status.readyReplicas}')" = 3
printf 'PASS: Deployment desired and ready replicas are three\n'
```

Watch for: Needs capacity for three small nginx Pods. Image-pull access and namespace admission must work.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 02 / 12

L02 · Running versus Ready

Goal: The main process runs continuously. Creating the probe file changes traffic readiness independently from process existence.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l02-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > pod.yaml <<'YAML'
apiVersion: v1
kind: Pod
metadata: {name: worker}
spec:
  containers:
    - name: worker
      image: busybox:1.37
      command: ["sh", "-c", "sleep 600"]
      readinessProbe:
        exec: {command: ["test", "-f", "/tmp/ready"]}
        periodSeconds: 2
YAML
kubectl apply -n "$K_NS" -f pod.yaml
kubectl wait pod/worker -n "$K_NS" --for=jsonpath='{.status.phase}'=Running --timeout=120s
kubectl wait pod/worker -n "$K_NS" --for=condition=Ready=false --timeout=30s
kubectl exec -n "$K_NS" worker -- touch /tmp/ready
kubectl wait pod/worker -n "$K_NS" --for=condition=Ready --timeout=60s
test "$(kubectl get pod worker -n "$K_NS" -o jsonpath='{.status.containerStatuses[0].restartCount}')"
= 0
printf 'PASS: readiness changed without restarting the container\n'
```

Watch for: This is an artificial probe fixture. A production readiness check must represent the actual admission contract.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 03 / 12

L03 · Service DNS and ready backends

Goal: The client uses the Service name and port. EndpointSlices provide a backend-selection view independent of memorizing Pod IPs.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l03-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > web.yaml <<'YAML'
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports: [{name: http, containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: http}
            periodSeconds: 2
YAML
cat > service.yaml <<'YAML'
apiVersion: v1
kind: Service
metadata: {name: web}
spec:
  selector: {app: web}
  ports: [{name: http, port: 80, targetPort: http}]
YAML
kubectl apply -n "$K_NS" -f web.yaml -f service.yaml
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
kubectl run client -n "$K_NS" --image=busybox:1.37 --restart=Never \
  --command -- sh -c 'sleep 600'
kubectl wait pod/client -n "$K_NS" --for=condition=Ready --timeout=120s
kubectl exec -n "$K_NS" client -- sh -ec \
  'wget -qO /tmp/page http://web; grep -q "Welcome to nginx" /tmp/page'
kubectl get endpointslices -n "$K_NS" -l kubernetes.io/service-name=web
printf 'PASS: namespace Service DNS routed HTTP to a ready backend\n'
```

Watch for: Depends on cluster DNS and working service networking. The nginx page check belongs to this disposable fixture.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 04 / 12

L04 · ConfigMap environment snapshot versus file update

Goal: Projected configuration can refresh eventually; the container environment remains the startup snapshot. No subPath mount is used.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l04-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > config.yaml <<'YAML'
apiVersion: v1
kind: ConfigMap
metadata: {name: settings}
data: {MODE: old}
YAML
cat > pod.yaml <<'YAML'
apiVersion: v1
kind: Pod
metadata: {name: worker}
spec:
  containers:
    - name: worker
      image: busybox:1.37
      command: ["sh", "-c", "sleep 600"]
      env:
        - name: MODE
          valueFrom:
            configMapKeyRef: {name: settings, key: MODE}
      volumeMounts:
        - {name: settings, mountPath: /config}
  volumes:
    - name: settings
      configMap: {name: settings}
YAML
kubectl apply -n "$K_NS" -f config.yaml -f pod.yaml
kubectl wait pod/worker -n "$K_NS" --for=condition=Ready --timeout=120s
test "$(kubectl exec -n "$K_NS" worker -- printenv MODE)" = old
kubectl patch configmap settings -n "$K_NS" --type=merge \
  -p '{"data":{"MODE":"new"}}'
K_UPDATED=0
for K_N in $(seq 1 36); do
  if test "$(kubectl exec -n "$K_NS" worker -- cat /config/MODE)" = new; then
    K_UPDATED=1; break
  fi
  sleep 5
done
test "$K_UPDATED" = 1
test "$(kubectl exec -n "$K_NS" worker -- printenv MODE)" = old
printf 'PASS: projected file updated; environment remained old\n'
```

Watch for: Propagation timing is cluster-dependent. The bounded wait can fail on unusually slow settings. This does not test application hot reload.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 05 / 12

L05 · Narrow RBAC scope for a ServiceAccount

Goal: The Role contains only selected ConfigMap read verbs, and the namespaced binding grants that scope to the lab account.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l05-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
kubectl create serviceaccount reader -n "$K_NS"
kubectl create role config-reader -n "$K_NS" --verb=get,list --resource=configmaps
kubectl create rolebinding reader -n "$K_NS" --role=config-reader \
  --serviceaccount="$K_NS:reader"
K_SUBJECT="system:serviceaccount:$K_NS:reader"
test "$(kubectl auth can-i get configmaps -n "$K_NS" --as="$K_SUBJECT")" = yes
if kubectl auth can-i get secrets -n "$K_NS" --as="$K_SUBJECT"; then
  printf 'FAIL: unexpected Secret access\n' >&2; exit 1
fi
printf 'PASS: account can read ConfigMaps but not Secrets\n'
```

Watch for: The caller needs permission to create these objects and impersonate the account for auth can-i. Existing broad grants or group bindings can change the result.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 06 / 12

L06 · Bounded Job completion

Goal: The Job records successful completion rather than maintaining a permanent server. The command is harmless and deterministic.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l06-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > job.yaml <<'YAML'
apiVersion: batch/v1
kind: Job
metadata: {name: task}
spec:
  backoffLimit: 1
  activeDeadlineSeconds: 120
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: task
          image: busybox:1.37
          command: ["sh", "-c", "printf 'complete\n'"]
YAML
kubectl apply -n "$K_NS" -f job.yaml
kubectl wait job/task -n "$K_NS" --for=condition=Complete --timeout=150s
test "$(kubectl logs -n "$K_NS" job/task)" = complete
test "$(kubectl get job task -n "$K_NS" -o jsonpath='{.status.succeeded}')" = 1
printf 'PASS: Job completed once with the expected output\n'
```

Watch for: This observed success does not prove exactly-once execution for a real business task. Retryable jobs need durable idempotency.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 07 / 12

L07 · PVC persistence across Pod replacement

Goal: The second Pod mounts the same claim. Pod deletion removes its ephemeral execution state but not this still-existing PVC.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l07-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > claim.yaml <<'YAML'
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests: {storage: 1Gi}
YAML
cat > pod.yaml <<'YAML'
apiVersion: v1
kind: Pod
metadata: {name: worker}
spec:
  containers:
    - name: worker
      image: busybox:1.37
      command: ["sh", "-c", "sleep 600"]
      volumeMounts:
        - {name: data, mountPath: /data}
  volumes:
    - name: data
      persistentVolumeClaim: {claimName: data}
YAML
kubectl apply -n "$K_NS" -f claim.yaml -f pod.yaml
kubectl wait pod/worker -n "$K_NS" --for=condition=Ready --timeout=180s
kubectl exec -n "$K_NS" worker -- sh -c 'echo retained > /data/note'
kubectl delete pod worker -n "$K_NS" --wait=true
kubectl apply -n "$K_NS" -f pod.yaml
kubectl wait pod/worker -n "$K_NS" --for=condition=Ready --timeout=180s
test "$(kubectl exec -n "$K_NS" worker -- cat /data/note)" = retained
printf 'PASS: retained file survived Pod replacement\n'
```

Watch for: Needs a usable default StorageClass and storage permissions. Namespace cleanup deletes the disposable claim; backend retention follows reclaim policy. This is not a database backup test.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 08 / 12

L08 · Verify ingress NetworkPolicy enforcement

Goal: Both clients first have an accessible Service path. The applied ingress policy permits only the approved peer label. The final positive check guards against merely breaking the backend.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l08-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > web.yaml <<'YAML'
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports: [{name: http, containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: http}
            periodSeconds: 2
YAML
cat > service.yaml <<'YAML'
apiVersion: v1
kind: Service
metadata: {name: web}
spec:
  selector: {app: web}
  ports: [{name: http, port: 80, targetPort: http}]
YAML
cat > policy.yaml <<'YAML'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-approved}
spec:
  podSelector: {matchLabels: {app: web}}
  policyTypes: [Ingress]
Script continues on the next page. Save both code blocks as one script.

  ingress:
    - from:
        - podSelector: {matchLabels: {access: approved}}
      ports: [{protocol: TCP, port: 80}]
YAML
kubectl apply -n "$K_NS" -f web.yaml -f service.yaml
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
kubectl run approved -n "$K_NS" --image=busybox:1.37 --restart=Never \
  --labels=access=approved --command -- sh -c 'sleep 600'
kubectl run blocked -n "$K_NS" --image=busybox:1.37 --restart=Never \
  --command -- sh -c 'sleep 600'
kubectl wait pod/approved pod/blocked -n "$K_NS" --for=condition=Ready --timeout=120s
kubectl exec -n "$K_NS" blocked -- wget -qO- -T 5 http://web >/dev/null
kubectl apply -n "$K_NS" -f policy.yaml
K_DENIED=0
for K_N in $(seq 1 12); do
  if ! kubectl exec -n "$K_NS" blocked -- wget -qO- -T 3 http://web >/dev/null 2>&1; then
    K_DENIED=1; break
  fi
```

```
  sleep 2
done
test "$K_DENIED" = 1
kubectl exec -n "$K_NS" approved -- wget -qO- -T 5 http://web >/dev/null
printf 'PASS: approved client succeeded and blocked client lost access\n'
```

Watch for: Requires a NetworkPolicy-enforcing CNI and no conflicting broad allow rules. DNS egress is unchanged. Failure on a non-enforcing local network is useful evidence, not a passed policy test.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 09 / 12

L09 · Diagnose an unsatisfiable node selector

Goal: The lab first confirms that no node has the requested label. The scheduler cannot satisfy the selector, independent of current CPU utilization.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l09-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > pod.yaml <<'YAML'
apiVersion: v1
kind: Pod
metadata: {name: worker}
spec:
  nodeSelector:
    interview.example.com/eligible: never-match
  containers:
    - name: worker
      image: busybox:1.37
      command: ["sh", "-c", "sleep 600"]
YAML
test -z "$(kubectl get nodes -l interview.example.com/eligible=never-match -o name)"
kubectl apply -n "$K_NS" -f pod.yaml
kubectl wait pod/worker -n "$K_NS" --for=jsonpath='{.status.phase}'=Pending --timeout=30s
test -z "$(kubectl get pod worker -n "$K_NS" -o jsonpath='{.spec.nodeName}')"
kubectl describe pod worker -n "$K_NS"
# Verify scheduler evidence rather than changing shared node labels.
printf 'PASS: Pod is Pending and has no assigned node\n'
```

Watch for: Pending alone does not prove the cause. Read Events for the actual scheduling message; admission or cluster failure can produce other symptoms.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 10 / 12

L10 · Observe a stalled rollout and explicit undo

Goal: A missing nginx path makes new replicas unready while the previous template can still provide capacity. The exercise chooses an explicit undo after observing the failure.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l10-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > web.yaml <<'YAML'
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports: [{name: http, containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: http}
            periodSeconds: 2
YAML
kubectl apply -n "$K_NS" -f web.yaml
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
K_PATCH='{"spec":{"template":{"spec":{"containers":[
  {"name":"web","readinessProbe":{
    "httpGet":{"path":"/missing-lab-path","port":"http"}}}
]}}}}'
kubectl patch deployment web -n "$K_NS" --type=strategic -p "$K_PATCH"
if kubectl rollout status deployment/web -n "$K_NS" --timeout=30s; then
  printf 'FAIL: broken readiness unexpectedly passed\n' >&2; exit 1
fi
kubectl describe deployment web -n "$K_NS"
kubectl rollout undo deployment/web -n "$K_NS"
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
test "$(kubectl get deploy web -n "$K_NS" -o jsonpath='{.status.readyReplicas}')" = 2
printf 'PASS: rollout stalled; explicit undo restored ready replicas\n'
```

Watch for: The 30-second client timeout is not an automatic rollback or necessarily the Deployment progress deadline. Real rollbacks require schema/config compatibility review.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 11 / 12

L11 · Kustomize a base for one replica

Goal: The Kustomization transforms a base that declared two replicas. The rendered file and live object provide separate views of the desired result.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l11-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > web.yaml <<'YAML'
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports: [{name: http, containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: http}
            periodSeconds: 2
YAML
cat > kustomization.yaml <<'YAML'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [web.yaml]
replicas:
  - name: web
    count: 1
YAML
kubectl kustomize . > rendered.yaml
kubectl apply -n "$K_NS" -f rendered.yaml
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
test "$(kubectl get deploy web -n "$K_NS" -o jsonpath='{.spec.replicas}')" = 1
test "$(kubectl get deploy web -n "$K_NS" -o jsonpath='{.status.readyReplicas}')" = 1
printf 'PASS: rendered overlay requested one ready replica\n'
```

Watch for: The v1beta1 version belongs to the Kustomization configuration format, not to a Deployment API version. Rendering does not validate every cluster policy.
Status: Bash syntax passed; cluster result remains to be tested.

HANDS-ON LAB 12 / 12

L12 · Temporary host access with port-forward

Goal: An omitted local port requests an available one. The script discovers the reported loopback port and tests HTTP while the forwarding process is alive.

SOURCE LAB · RUN ONLY ON A DISPOSABLE LOCAL CLUSTER

```
set -eu
test -z "$(ls -A)"
kubectl config current-context
K_NS="k8s-l12-$$"
! kubectl get namespace "$K_NS" >/dev/null 2>&1
kubectl create namespace "$K_NS"
trap 'kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
cat > web.yaml <<'YAML'
apiVersion: apps/v1
kind: Deployment
metadata: {name: web}
spec:
  replicas: 2
  selector: {matchLabels: {app: web}}
  template:
    metadata: {labels: {app: web}}
    spec:
      containers:
        - name: web
          image: nginx:stable
          ports: [{name: http, containerPort: 80}]
          readinessProbe:
            httpGet: {path: /, port: http}
            periodSeconds: 2
YAML
cat > service.yaml <<'YAML'
apiVersion: v1
kind: Service
metadata: {name: web}
spec:
  selector: {app: web}
  ports: [{name: http, port: 80, targetPort: http}]
YAML
kubectl apply -n "$K_NS" -f web.yaml -f service.yaml
kubectl rollout status deployment/web -n "$K_NS" --timeout=120s
kubectl port-forward -n "$K_NS" svc/web :80 > forward.log 2>&1 &
K_FORWARD_PID=$!
trap 'kill "$K_FORWARD_PID" >/dev/null 2>&1 || true;
      kubectl delete namespace "$K_NS" --wait=false >/dev/null 2>&1 || true' EXIT
K_PORT=""
for K_N in $(seq 1 30); do
  K_PORT=$(sed -n 's/.*127.0.0.1:\([0-9]*\) -> 80.*/\1/p' forward.log | head -n 1)
  if test -n "$K_PORT"; then break; fi
  sleep 1
done
test -n "$K_PORT"
curl -fsS "http://127.0.0.1:$K_PORT" -o page.html
grep -q 'Welcome to nginx' page.html
printf 'PASS: temporary local port reached the selected backend\n'
```

Watch for: Requires host curl and standard sed/head. Port-forward uses a Kubernetes diagnostic connection, not the ordinary Service virtual-IP or external-ingress path.
Status: Bash syntax passed; cluster result remains to be tested.

LAST-MINUTE REVISION

Object chooser

Desired result	Start with
Stateless API replicas	Deployment + Service
Stable database member identity	StatefulSet + storage
Node log/metrics collection	DaemonSet
Finish bounded work	Job
Run work on a schedule	CronJob
Internal stable endpoint	ClusterIP Service
HTTP routes from outside	Ingress/Gateway + controller
Storage beyond Pod lifetime	PVC + supported StorageClass
Limit API actions	RBAC + ServiceAccount
Limit pod-to-pod traffic	NetworkPolicy + enforcing CNI
Scale on demand	HPA + useful metrics
Protect voluntary maintenance	PodDisruptionBudget

Critical rule: An object existing does not prove the traffic path, data safety, or application correctness. Verify real behavior.

TROUBLESHOOT LIKE AN ARCHITECT

Symptom → evidence

Pending Pod
kubectl describe pod; inspect scheduling Events, PVC and constraints

CrashLoopBackOff
kubectl logs --previous; examine last termination reason and probes

ImagePullBackOff
describe Pod; check image ref, node store and imagePullSecrets

Service returns errors
selectors, EndpointSlices, readiness, targetPort, app listener

Ingress 404
Host, Prefix/Exact path, rewrites and application handler

Ingress 503
controller logs, ready upstreams, endpoints and app health

Pods running but slow
CPU throttle, memory, DB pool, queue age and p99

Rollout stalled
readiness, surge capacity, image, Secrets and progress conditions

Node down
replacement timing, storage attach, fencing and data leader safety

14-DAY PRACTICE PLAN

Prepare without memorizing

1–2	Chapters 01–04: explain control plane, Pod and rollout with a picture
3–4	Chapters 05–08: Service, Ingress, config, probes; try L01–L04
5–6	Chapters 09–12: resource budget, placement, PVC and Jobs; L05–L07
7–8	Chapters 13–15: RBAC, NetworkPolicy, HPA; L08
9–10	Chapters 16–18: diagnoses, GitOps and JVM pool limits; L09–L11
11–12	Chapters 19–21: rollout, disaster recovery, minikube, WSL; L12
13	Architecture mock: walk through Spring Boot order-service YAML
14	Explain ten random questions, answer follow-ups and rehearse one outage

For each topic: explain the resource owner, run one command, predict a failure, and say what evidence would prove recovery.

FURTHER LEARNING

Primary documentation

Use the official resources below to verify exact behavior for your cluster version. This edition preserves original question wording and adds new simplified teaching explanations and follow-up answers.

https://kubernetes.io/docs/concepts/overview/components/

https://kubernetes.io/docs/concepts/workloads/pods/

https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

https://kubernetes.io/docs/concepts/services-networking/service/

https://kubernetes.io/docs/concepts/services-networking/ingress/

https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

https://kubernetes.io/docs/concepts/storage/persistent-volumes/

https://kubernetes.io/docs/concepts/security/rbac-good-practices/

https://kubernetes.io/docs/concepts/configuration/configmap/

https://kubernetes.io/docs/concepts/configuration/secret/

https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/

https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/

https://kubernetes.io/docs/concepts/workloads/controllers/job/

https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

https://kubernetes.io/docs/concepts/services-networking/network-policies/

https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/

https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/

https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/

https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

https://kubernetes.io/docs/concepts/storage/storage-classes/

https://kubernetes.io/docs/concepts/storage/volumes/

https://kubernetes.io/docs/concepts/security/pod-security-standards/

https://kubernetes.io/docs/reference/access-authn-authz/rbac/

https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/

https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/

https://kubernetes.io/docs/concepts/policy/resource-quotas/

https://kubernetes.io/docs/concepts/policy/limit-range/

https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/

https://kubernetes.io/docs/concepts/security/service-accounts/

https://kubernetes.io/docs/tasks/debug/debug-application/debug-pods/

https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/

https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/

https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/

https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/

https://kubernetes.io/docs/concepts/containers/images/

https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/

https://kubernetes.io/docs/tasks/debug/debug-cluster/

https://kubernetes.io/docs/concepts/security/secrets-good-practices/

https://kubernetes.io/docs/tasks/administer-cluster/safely-drain-node/

https://kubernetes.io/releases/version-skew-policy/

https://kubernetes.io/docs/reference/using-api/server-side-apply/

https://kubernetes.io/docs/concepts/cluster-administration/logging/

https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/

https://kubernetes.io/docs/concepts/workloads/pods/downward-api/

https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/

https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/

https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/

https://gateway-api.sigs.k8s.io/guides/getting-started/introduction/

https://helm.sh/docs/intro/using_helm/

https://argo-cd.readthedocs.io/en/stable/core_concepts/

https://minikube.sigs.k8s.io/docs/start/

https://minikube.sigs.k8s.io/docs/handbook/accessing/

https://minikube.sigs.k8s.io/docs/handbook/pushing/

https://docs.spring.io/spring-boot/3.5/reference/actuator/endpoints.html

https://docs.spring.io/spring-boot/3.5/reference/web/graceful-shutdown.html

https://docs.spring.io/spring-boot/3.5/reference/features/external-config.html

https://docs.oracle.com/en/java/javase/21/docs/specs/man/java.html

https://kafka.apache.org/42/configuration/broker-configs/